SOCIAL NETWORKS

NPTEL Online Course | Prof. Sudarshan Iyengar, IIT Ropar

Week 2 - Lecture Transcripts

Lectures 14-26 | 13 lectures | 126 pages (source transcript pages 159-283)

INDEX

LECTURE	TOPIC	PAGE IN THIS FILE
L14	Introduction to Datasets	2
L15	Ingredients Network	9
L16	Synonymy Network	13
L17	Web Graph	16
L18	Social Network Datasets	19
L19	Datasets: Different Formats	22
L20	Datasets: How to Download?	36
L21	Datasets: Analyzing Using Networkx	50
L22	Datasets: Analyzing Using Gephi	78
L23	Introduction: Emergence of Connectedness	91
L24	Advanced Material: Emergence of Connectedness	96
L25	Programming Illustration: Emergence of Connectedness	104
L26	Summary to Datasets	125

Social Networks
Prof. S. R. S. Iyengar
Department of Computer Science
Indian Institute of Technology, Ropar

Lecture – 14
Handling Real-world Network Datasets
Introduction to Datasets

(Refer Slide Time: 00:20)



So, here we are with a brand-new chapter on datasets.

(Refer Slide Time: 00:26)



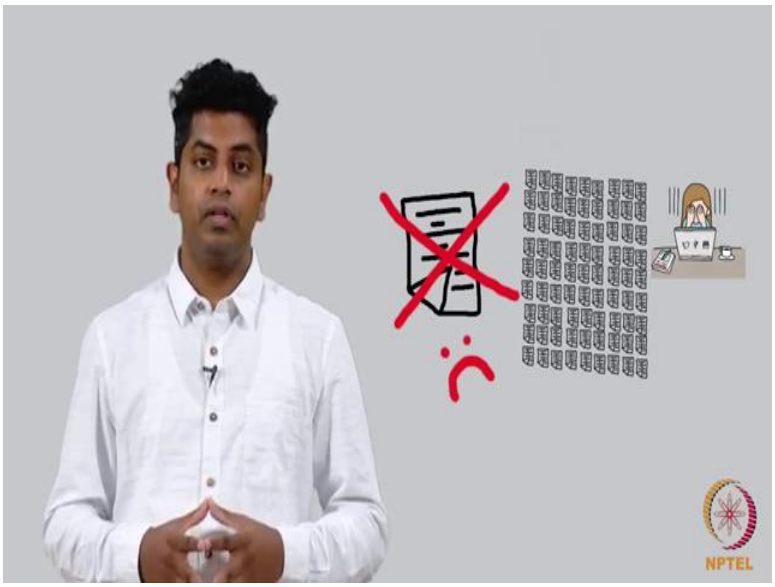
So, we are going to teach you the importance of crunching datasets and how to go about crunching them.

(Refer Slide Time: 00:30)



So firstly, what are datasets? Especially datasets that are relevant to social networks.

(Refer Slide Time: 00:38)



You see there was a time when we did not have any data and today we are in that era where it is all about data we have so much of data that we do not know how to make sense out of this data that a complete contrast over it was let us say some 30-40 years back.

(Refer Slide Time: 00:53)



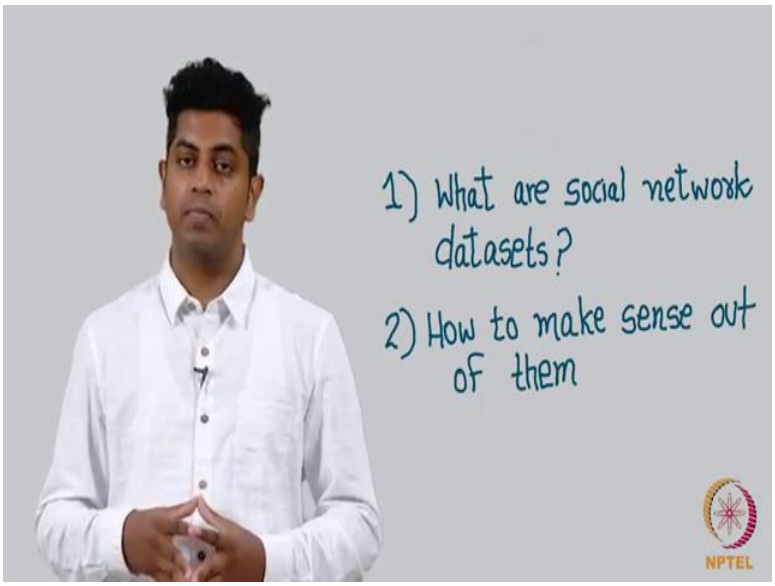
For example there was an; there was time when people look at the friendship network on 30 to 35 nodes, this is called to the Zachary Karate network, it had some 32 nodes and some edges on it, it is very popularly studied, very widely studied and are very popular social network, but today; back then we did we probably did not have a way in which we could get a bigger internet.

(Refer Slide Time: 01:28)



But today with the advent of internet and higher computing and storage facilities, we can crunch a whole lot of data whether we can crunch or not we have a whole lot of data.

(Refer Slide Time: 01:42)



So, this week this a entire module is all about understanding what are social network datasets and how one can make sense out of these things.

(Refer Slide Time: 01:52)



Please ensure that you have understood Python and networkx and another APIs on Python really well, especially the things that we thought in the last week.

(Refer Slide Time: 02:03)



I am sure you all know of this Play-Doh; Play-Doh is where you have this sand like thing that is sort of you can make any shape out of it and you realize that the options are multitude, you can go on making a small box out of it or a cute animal out of it, I mean we are in with limit is just in your imagination.

(Refer Slide Time: 02:28)



So, similarly when people got hold of this big network data set they realized that one can actually asked the whole lot of questions on it. Let us say first question that one can ask

on our friendship network, there is a classroom full of 100 students and data is available to me of their friendship network, who is friends with whom?

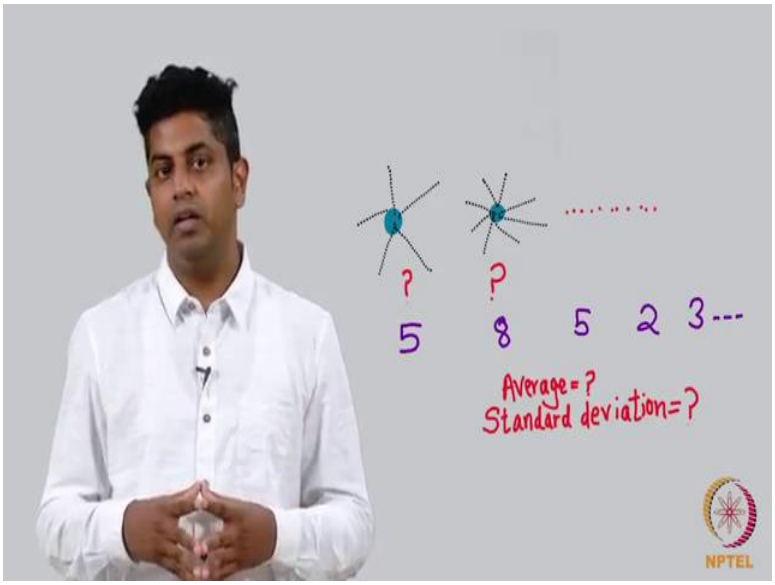
What kind of questions can we ask the obvious question is the graph connected we have discussed in a on this topic before most all graphs are connected, fine?

(Refer Slide Time: 03:06)



Second question; what can you say about the degree of the nodes?

(Refer Slide Time: 03:10)



By degree you mean; pick a person look at how many friends he has, pick another person look at how many friends he or she has so on and so forth, make a list of all these friendships of individual people, what is the average here and what is the standard deviation of this list what are the mean by this? By this I mean is the average sort of do everyone agree with the average or is it very widely spread of what uses is such that kind of research?

(Refer Slide Time: 03:45)



Data sets gave us this ability to question random stuff like this just like we would build a random stuff with Play-Doh and then played around with his datasets and arrived at fantastic conclusions. So, now, what we are going to do is we are going to pick some random datasets for a discussion right now. I am going to go one by one go through some 8 to 10 datasets and explain what these datasets and what kind of question are can one ask and answer.

Social Networks
Prof. S. R. S. Iyengar
Department of Computer Science
Indian Institute of Technology, Ropar

Lecture – 15
Handling Real-world Network Datasets
Ingredients Network

Let us now look at some networks as examples; I just told you that the advent of internet era has resulted in; we having a whole lot of plethora of datasets. Let me start discussing them one by one first and the most interesting dataset at least to me is the following.

(Refer Slide Time: 00:30)



Assume I prepare a dish and I look at all the ingredients in this dish; let say there are 5 items in this particular dish I am preparing.

(Refer Slide Time: 00:41)



And I take all possible ingredients in the world and I put a edge between two ingredients if they are part of a dish.

So, let us say there are some 200 ingredients and I start putting a link between two things if they are part of one dish and I keep doing this and I finally, observe how does the graph look like, maybe you think there is definitely some dish where any two ingredients are part of it. So, what I will do is I will look at only those cases which are popular.

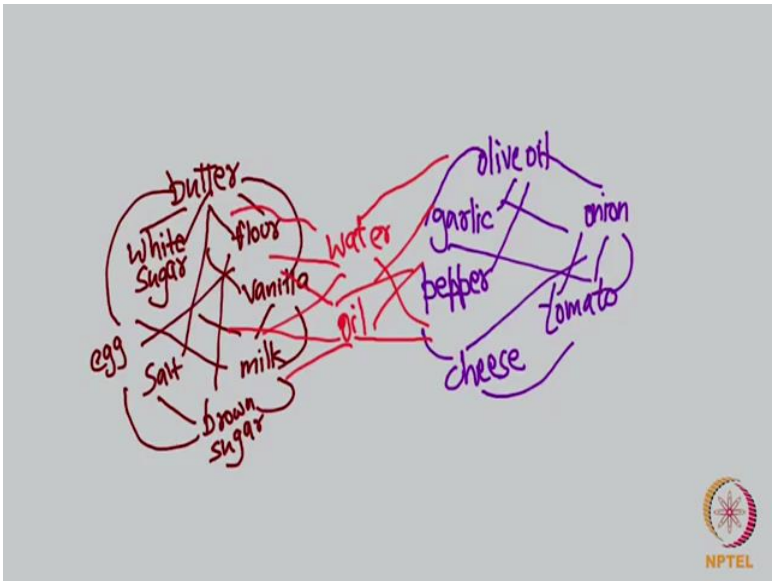
(Refer Slide Time: 01:20)



For example, there are dishes where salt and both are added many dishes one can name. So, I am going to put an edge between salt and sugar, but chili and chili powder and sugar probably there are not many dishes where you have both the both these things one exception could be chili chocolate, but one or two with one or two exceptions it is never used together.

So, there is no edge between chili powder and let say sugar.

(Refer Slide Time: 01:55)



If I look at the entire big network of all these ingredients it is a big network, now what do you expect? Anything interesting; anything at all that is interesting well you will observe that there will be strong communities in this network by communities I mean a bunch of people, bunch of ingredients that are mostly connected to each other and again a bunch of ingredients that are mostly connected to each other, but no connections across.

(Refer Slide Time: 02:27)



Now why even study this? It is plain interesting number one and secondly, may not be in this network in any other network such community structure has a lot of hints on the overall behavior of the system, so will see more such networks and more such properties of these networks in the forth coming examples.

Social Networks
Prof. S. R. S. Iyengar
Department of Computer Science
Indian Institute of Technology, Ropar

Lecture – 16
Handling Real-world Network Datasets
Synonymy Network

I told you a sort of a weird example where one can see networks; I am going to tell you another example which is equally weird.

(Refer Slide Time: 00:18)



So, assume I take all words in English, words that are available from thesaurus and I put an edge between two words if they are synonyms so; obviously, a love and let say passion has an edge because they are synonyms. Let us say one can think of obtain such examples of let us say I say hate synonym for hate is bad, so and so forth.

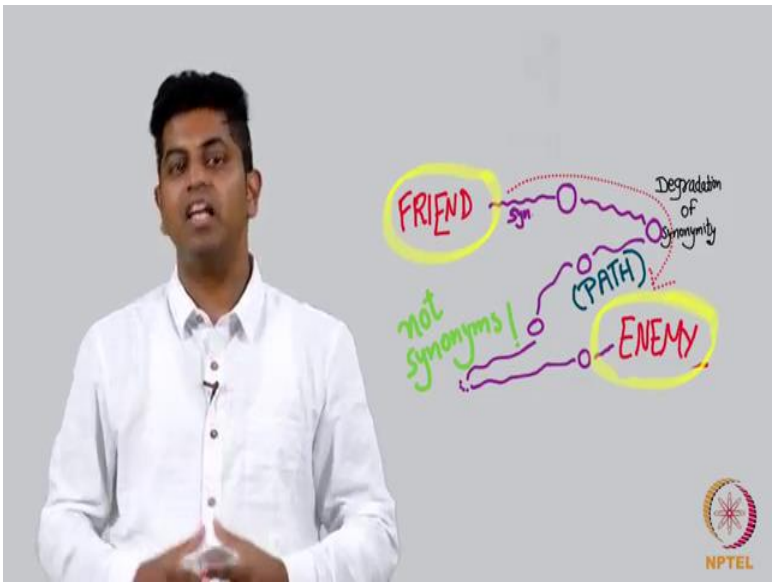
Now, the same old question; once you have a network, we will ask the following question; is it connected? So, it is surprisingly true that this network is connected, at least it as a huge component; large component. What by mean by this? By this I mean you take all words in English thesaurus and you put an edge between two words, if one is a synonym of the other as you know if A is synonym of B then B is synonym of A; it is symmetric you draw the graph I am telling you its connected, so what? Something surprising happens if it is connected; think about it for a minute.

(Refer Slide Time: 01:31)



There is this word love there is this word hate they are not synonyms given that the graph is connected there has to be some word which love is synonym to and this words is synonym to some other words, so on and so on and finally, you will be getting the word hate. Graph theoretically speaking there should be a path from love to hate and you move synonymously, how do I mean by this? By this I mean do you see that if a is synonym of B and B is a synonym of C, A should be a synonym of C that is what it means, synonym means what these two mean the same if the word A means the same as B, if the word B means the same as C then the word A must mean the C same as C, but you observe that from love we can go to hate through synonyms which means love is synonyms to hate.

(Refer Slide Time: 02:28)



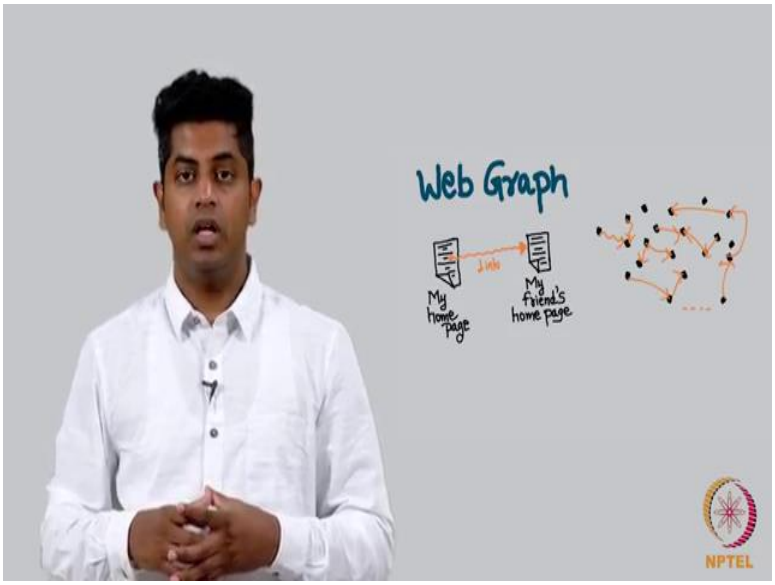
We can go from friend to let say enemy synonymously which means friend is synonyms to enemy I means the how weird is that; a graph theoretic analysis network theoretic analysis tells you of such weird retain English language not necessarily English language may be in many such many well known languages and what does this denote? It denotes that this synonym network synonym relation is not actually symmetric; if A is synonymous to B and B is synonymous to C, you cannot say A and C are actually synonymous, maybe there is a degradation of the synonyms as you proceed because from love you very beautifully get to hate, from enemy you very nicely get to friends synonymously. I just gave you an example of how a network theoretic analysis of question or an entity give some eyebrow raising results which are mostly generally is unexpected.

Social Networks
Prof. S. R. S. Iyengar
Department of Computer Science
Indian Institute of Technology, Ropar

Lecture – 17
Handling Real-world Network Datasets
Web Graph

So, let us look at this seemingly useless example, I say useless because it is appearingly useless, but it is extremely useful, I have been highlighting this fact from almost the first lecture. The example is this. It is called the network of the World Wide Web also called the web graph.

(Refer Slide Time: 00:24)



So, what I do is the following, I have my home page and I link the homepage of my friend on my homepage. I say for more details about these scores, please refer to my friend's website here and that here is a hyperlink to the other web page. We have discussed this before if you remember. So, now, if I look at the entire World Wide Web it is a bunch of pages; simply a bunch of pages, is it not. So, what I do is I go to a page, call it a node in my graph, in my graph every node corresponds to a page.

So, take 2 nodes means take 2 web pages, put an edge between them if one points to the other. So, my home page is a node, my friend's home page is another node, I put an edge from my page to my friend's home page saying A; my homepage is adjacent to B; my

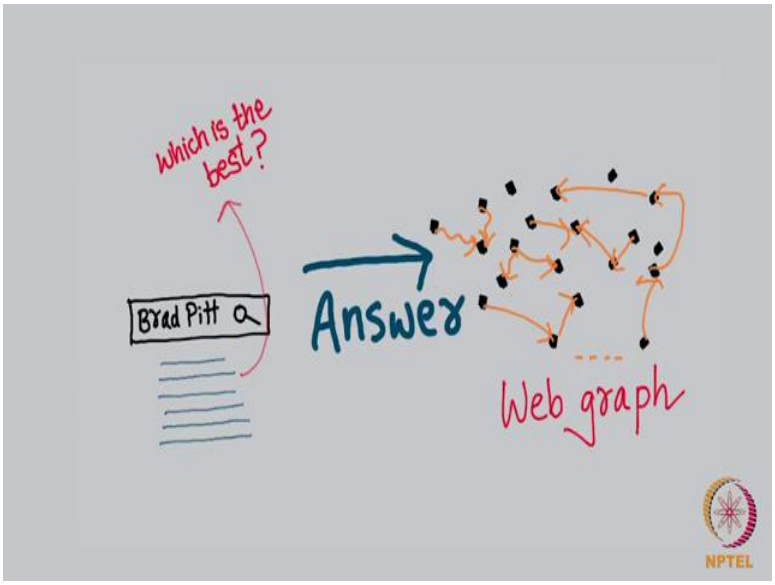
friend’s home page and here is the edge. Assume I do this to all possible pages in the world. How does the graph look like? Very weirdly collecting such a graph looks like complete waste of time of what use would such a graph be it does not sound like it is going to be of any use, but as I told you people this resulted in a multibillion dollar industry which today we call google.

(Refer Slide Time: 01:42)



So, google basically harness the fact, but deep within this connections is the clue to the answer; answer to this question as to how can one get the best search result. If someone types a celebrity’s name; let us say a Brad Pitt, it has a whole lot of their whole lot of pages on the internet which has the word Brad Pitt which is the best possible hit amongst these; the answer is in this web graph and google unlisted it.

(Refer Slide Time: 02:22)

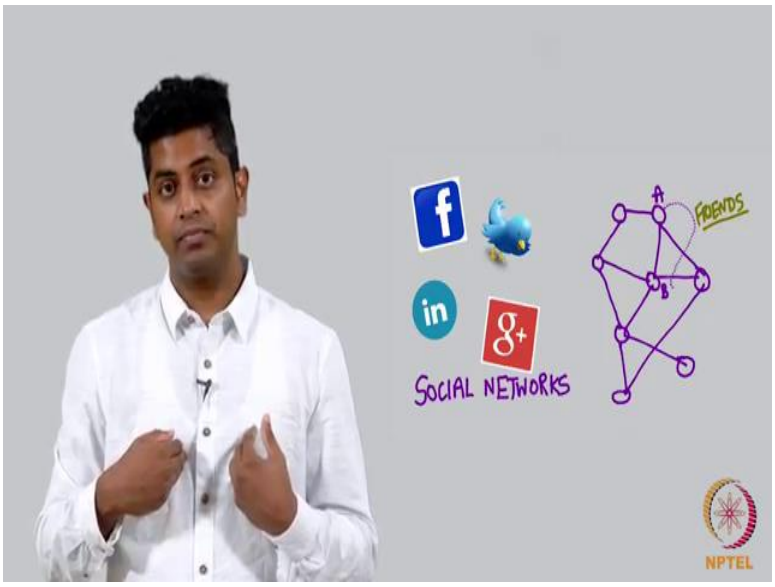


So, now here is an example of a network dataset called the web graph which is appearing of no use, but a very small process of mining, it gives you an answer to a huge question, but people did not realize is a that important in the 90s, but in the late 90s to Larry Page and (Refer Time: 02:50) came out with this beautiful algorithm with says this is where one can search for an answer for the question, how does one rank once search keywords once you type in a keyword whatever you get how do you rank the output of your search.

Social Networks
Prof. S. R. S. Iyengar
Department of Computer Science
Indian Institute of Technology, Ropar

Lecture – 18
Handling Real-world Network Datasets
Social Network Datasets

(Refer Slide Time: 00:14)



Let us look at literally a good social network that we all are familiar with. We all know a Facebook or let us say LinkedIn, Twitter, Google Plus, they are all what is called the social networks where a bunch of people get together and then the declared he is my friend he she is my friend and so on. If you look at such a network, what do you mean by such a network? By that I mean you put an edge between 2 nodes; nodes are people here, if one is in the friend list of the other I have you in my Facebook friend list, you have been in my Facebook friend list, you know its mutual and you put an edge between these 2 people and you look at the entire friendship network this way again appearing is useless dataset.

But then Facebook again made good use of this dataset by asking this question, they said fine here is a bunch of friends of this person who could be is next friend no, now here is a very important topics that requires some explanation.

(Refer Slide Time: 01:09)

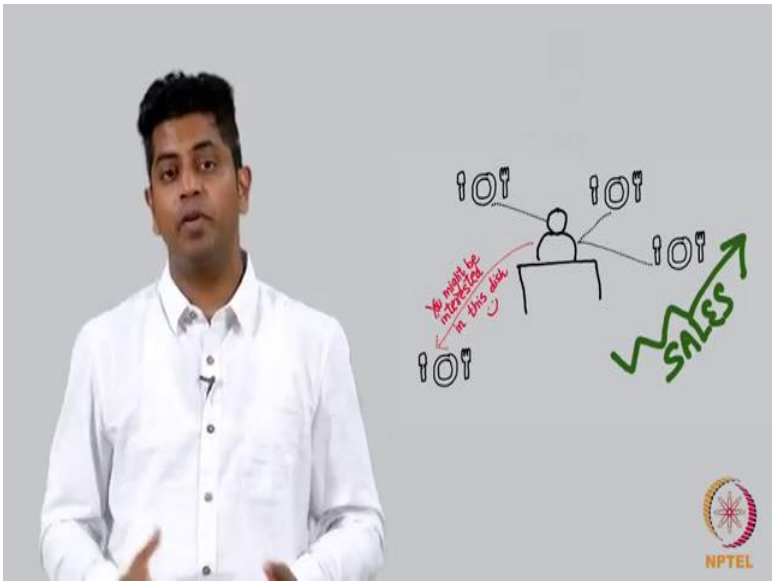


Let us take a pause and try to analyze this question properly, here is a friendship network of who is friends with whom and you would like to say I recommend Sudarshan a new friend who is this, out of let say 7 billion possible friends in this world for me I have some thousand friends on Facebook, anybody outside this thousand friends can be my prospective friend.

Now, how will you choose one most probable friend for me out of this big lot of people of what use is this to a social networking site like let say LinkedIn, Facebook, Google plus, all these sites work based on the numbers the more friends you have more busy you get glue to the monitor being on the social networking sites. So, they want to increase the number of friends they can only increased when the show you a prospective friend. This prospective friend let us say he is your long lost childhood friend or this prospective friend is a friend you find very interesting for what is ever reasons how will Facebook know about it well the answer for this is hidden in the network structure there were few things that you must observe not just between you your friends, but in the entire graph which will give you a hint as to who will be your most probable next friend.

If you show me the most probable next friend I will probably the friend that person make friends with that person correct it is just like saying what dishes I eat and in a hotel in a restaurant and is an automated recommending system which says you may possibly be interested.

(Refer Slide Time: 03:00)



In this particular dish that will increase the sales of the restaurant if they are able to predict properly what do you generally like eating similarly who will be your next friend in case it is predicted can result in increasing your friend list not only will you will it increase your friend list it will also populate your friend list with good friends good whatever you define it as the most important friends whatever I define it as. So, this is this goes into the name link prediction and Facebook friend list is a great playground for one to investigate on who could be the next probable friend.

Social Networks
Prof. S. R. S. Iyengar
Department of Computer Science
Indian Institute of Technology, Ropar

Lecture – 19
Handling Real-world Network Datasets
Datasets: Different Formats

(Refer Slide Time: 00:05)

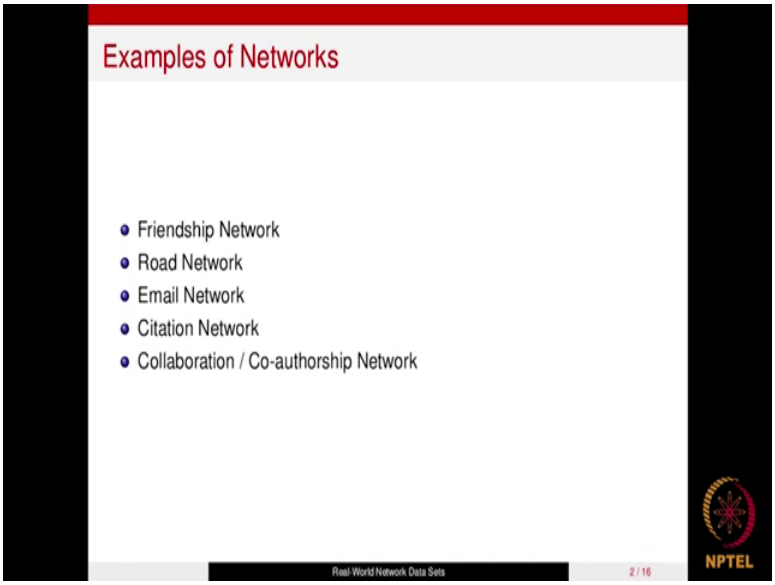


Hi everyone! So, after having got introduced to the course and after having seen the various packages that you can use for the energy the social networks, I am sure you might be all geared up to learn the concept from social networks. I would say if you only learn the concepts from this course and do not apply them to nearby situations, you may not be able to get an enough understanding. So, it is important that whatever knowledge you required you apply them to real world scenarios and for that purpose you obviously, need a dataset from the real world situations.

Now, how do we get that data? Fortunately there are various individuals and organizations who have gathered network datasets for different situations and topics and the good thing is that they have kept this data online publicly available for anyone to access and analyze more over these datasets are available in different formats. So, in this video, we are first going to take a look at the formats into which the datasets are available and after that we are going to look at the sources from which we can download

these datasets and then in the subsequent videos we are going to analyze these datasets which we will download from the internet.

(Refer Slide Time: 01:31)



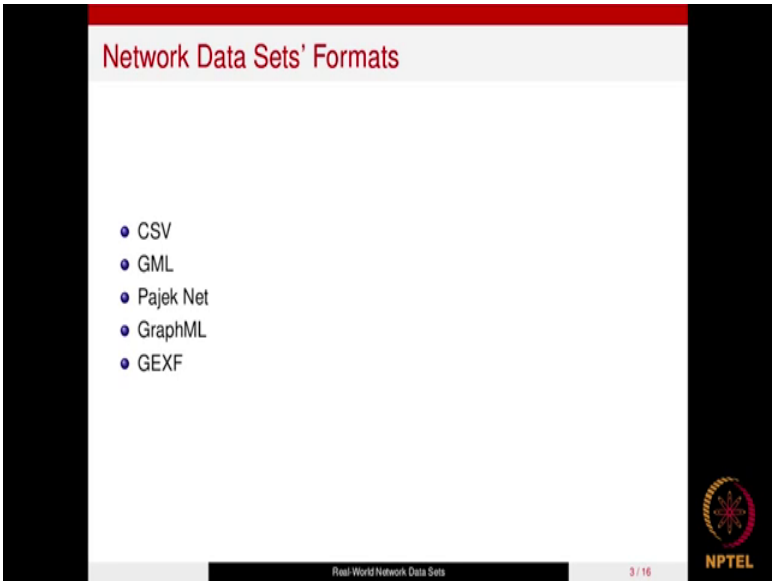
So, let us get started and before we get into the formats of these data sets I want you to take a look at the networks on which data is available of course, they are not limited we have data sets available on diverse range of topics, I am just going to give you a few examples of networks over here. So, we have friendship networks where the nodes are people and there will be a link from one node to the other node if these 2 nodes are if these 2 people are friends with each other similarly we have road networks where the nodes will be cities and there will be link from one node to the other node if these 2 cities are connected to a road.

So, usually these friendship networks and road networks are undirected networks and if you look at directed networks we have email network where the nodes will be the people and there will be a directed edge from node A to node B, if person is sent an email to person B, another example of a director network is citation network. So, you might be knowing that in scientific scenario if a paper uses some information from another paper then this paper sites the second paper. So, that is what information is captured by a citation network. So, this is of course, directed network.

Another example of a network from scientific domain is collaboration network. So, scientists collaborate with each other for some research project or some scientific

development. So, this sort of information is captured by collaboration or co authorship network which captures which 2 people collaborated with each other or co authored for a people. So, these where just few examples of networks we are going to come across a number of them in on different topics. So, as he said these networks are available in different formats.

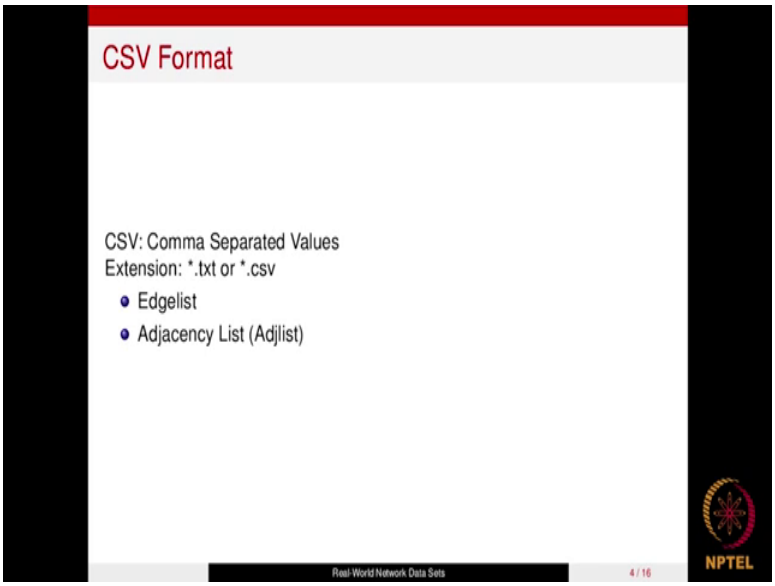
(Refer Slide Time: 03:27)



Let us take a look at these formats one by one. So, we have CSV format, we have GML format, we have Pajek format, we have GraphML format, we have GEXF format.

So, I am going to take these networks one by one, I am going to tell you the basic features of these formats and the basic structure that these that the files of these networks carry. So, apart from these formats you might also come across from two three more formats, but these 5 are the most commonly used formats when you download some data sets from internet that might most likely been one of these formats. So, we are going to take a look at these formats in detail will take them one by one.

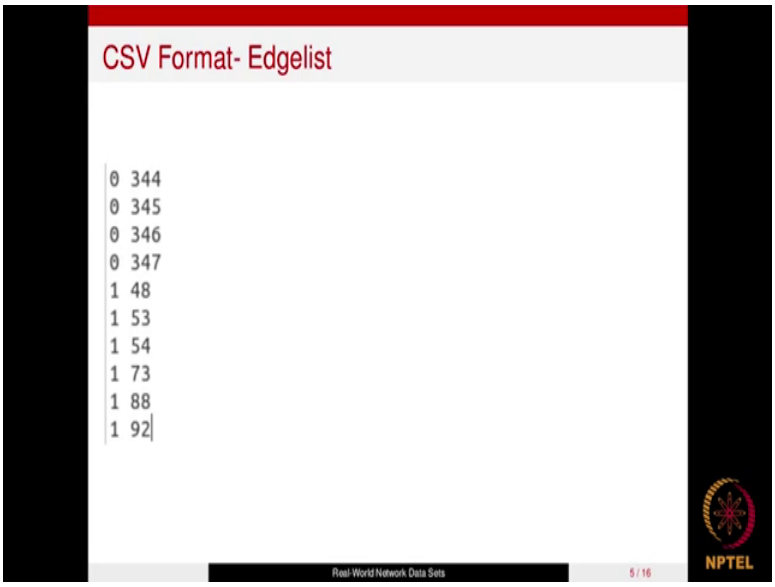
(Refer Slide Time: 04:16)



Let us see; what is CSV format? So, we might be knowing that CSV stands for comma separated values this file will be in form having the extension either dot txt or dot csv.

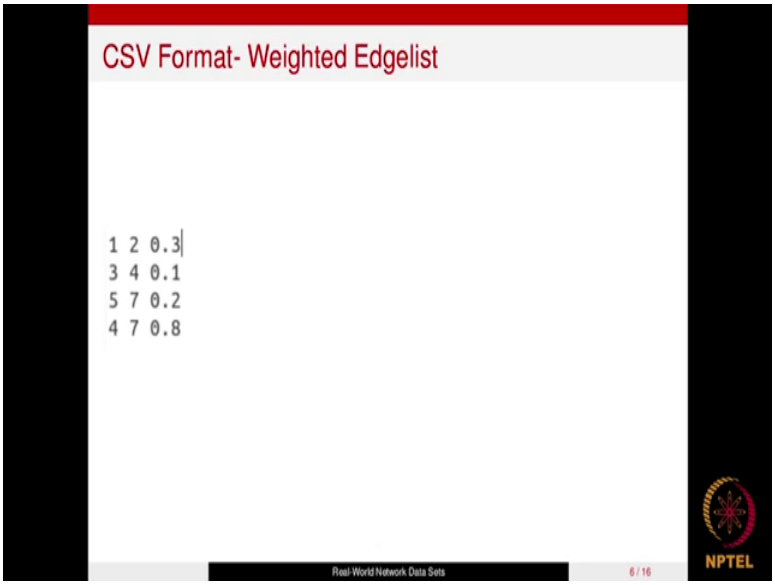
Now, CSV format file can have 2 more types it could either be in Edgelist list format or it could be in adjacency list format. So, if you are from graph background you might know what an adjacency list means and what an Edgelist mean any anyway I am going to give an example of both these formats.

(Refer Slide Time: 04:54)



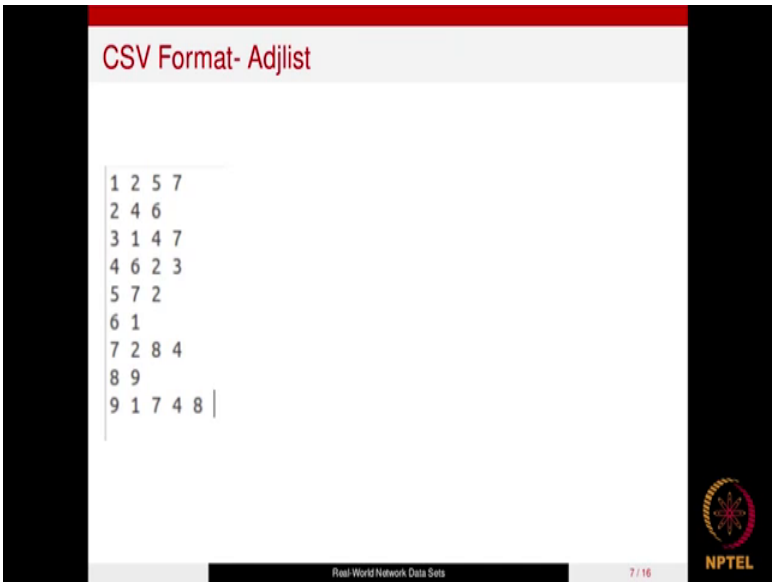
So, let us see what is an edge list format you can look at the simplicity of this file this is a snapshot from one of the files which is in CSV format which is in Edgelist type here every row contains two values first value is the source node and the second value is the target node. So, this indicates the row indicates that is going to be an edge from 0 to 344, 0 to 345, 0 to 346 and so on.

(Refer Slide Time: 05:38)



So, basically the file will contain a list of the edges and in case you want to add some weight to these edges you can also do that for example, you want to add weights to these edges like this you can add third value to every row. So, for example, the edge which is going to go from 1 to 2 should have an 8.3. So, you can add it like this; however, there is one limitation that the edge list format carries and that you cannot add non numeric weights to the to the edges you cannot change the labels you cannot change any you cannot add an attribute to the nodes and edges. So, that is a limitation that this format carries. So, there is a trade. So, you can see that there is a trade of between the simplicity and the flexibility that a format provides.

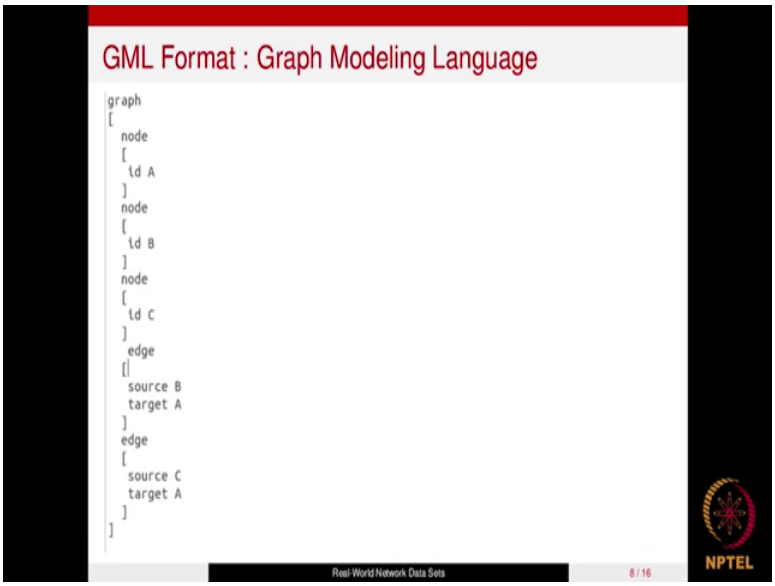
(Refer Slide Time: 06:22)



Let us now look at the adjacency list format. So, this is a snapshot from a file which is in adjacency list format you see every row is having more than 2 values, now what is this indicate the first value in every row is basically a source node and the subsequent values in every row are the nodes which are adjacent to this source node. So, this first row indicate that there is going to be an edge from one to 2 this going to be an edge from 1 to 5 and1 to 7 and if you go to the second row it indicates that there is going to be an edge from 2 to 4, 2 to 6, 3 to 1, 3 to 4 and 3 to 7 and so on.

So, basically every row tells all the nodes that are adjacent to a given node. So, this is the adjacency list format again you can see the simplicity that this format provides; however, you as I already told you this format does not give you much flexibility for example, you just cannot add any sort of attribute or label to the nodes or edges if we want to add some color attribute to the nodes or edges you cannot do that. So, it all depends on your requirement.

(Refer Slide Time: 07:38)

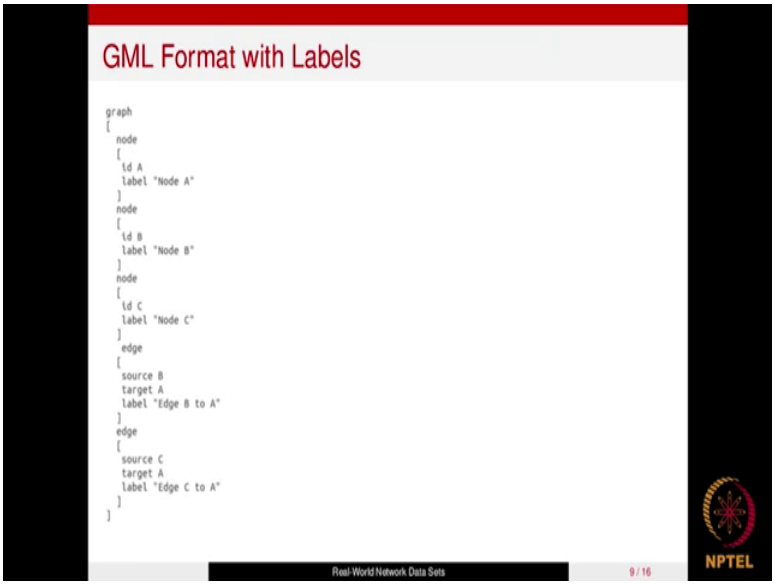


Let us go to the next format the next format is GML format with stand for graph modeling language now this is one of the most commonly used format for network datasets the reasons include the flexibility that this format provides when it comes to labeling or assigning attributes to the nodes and edges the also this format is not very complex.

Let me explain you the format of this file. So, you start with the graph keyword and when you start the square bracket and inside that you have node keyword and when you again this square bracket and we write the keyword id and then you write the id of that node we close the node and so on you keep adding the nodes. So, if there are some n nodes in the network, they are going to be n nodes keyword once the nodes are done you start with the edge keyword. So, again you start the edge square bracket and then you write the source keyword and this is the source node the idea of the source node and then you write the target keyword and then the idea of the target node and we close it and so on.

So, if there are n edges there will be m edge keywords like this that is that the simplest format of a file which is in GML format in case you want to add some labels to these nodes and edges you can do that as well in this format.

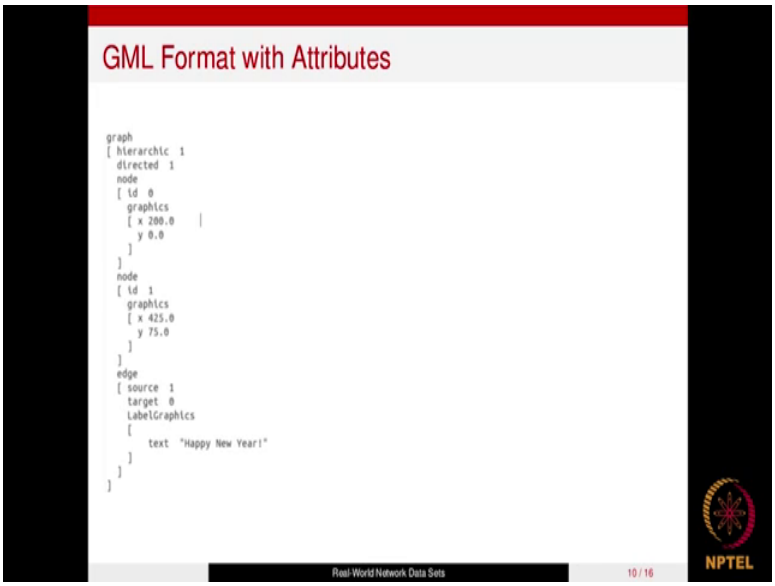
(Refer Slide Time: 09:09)



Let me show you an example for that. So, here you see we are adding a label to this node A. So, you can see this syntax you go inside the node keyword and there you write this keyword label and then you write the label of that node which is node a in this case similarly to write the label of the next node you go inside that in you write the label keyword and then you write the label that is node B and so on you do this for all the nodes individually and similarly you can do the labeling for the edges again you use the label keyword and then you write the label for the edge A which is edge B to A in this case and so on.

So, this is how to add labels and in case you want to add some attributes to the nodes and edges you can do that as well let me show you an example for that.

(Refer Slide Time: 10:00)

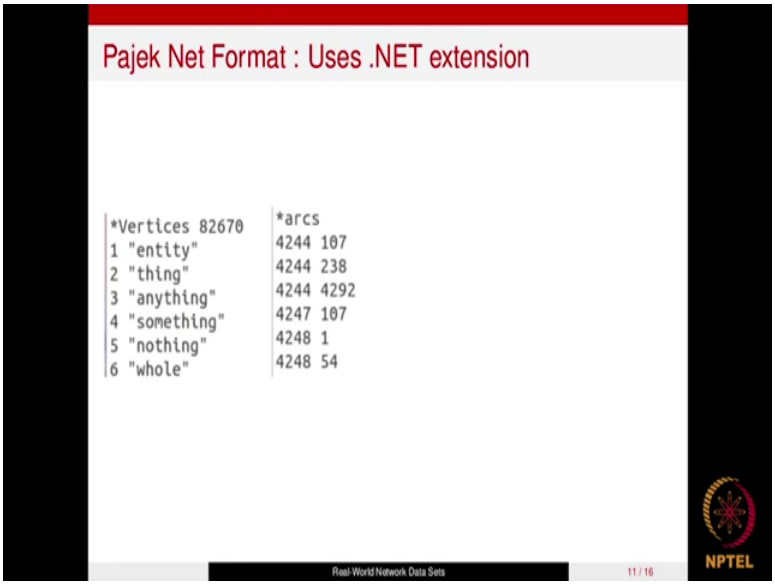


Here you see this is a graph; this is the starting with graph and inside this, there are 2 attributes here which are the graph attribute. So, apart from nodes node attributes and edge attributes there are graph attributes as well in this case there are 2 attributes hierarchic and directed and these are the values of these attributes to said the node attributes you go inside the node and when you start this attribute that you want to assign and then the name of that attribute and the value of that attribute inside the square brackets. So, this is the syntax. So, basically here they are assigning x and y positioning to the node.

Similarly, for the next node they are assigning x and y positions of the second node and here for adding an attribute to the edge again you start another attribute and start square brackets and then assign the value of that attribute now here at this point let me tell you one thing in case you are just making use of the data set download from the internet you might not need to go into the details whatever I am just now explaining. So, only in case you want to create your own datasets only in that case you should know these integrate details that I am telling you otherwise you should not worry about these details I just want to you to be aware of the syntax. So, that in case you come across some file should be able to mix sense sort of it you should be able to know what these a labels and keywords mean.

In other cases, you just want to download the files from internet, you might not need to go into all these details, let us go to the next format now.

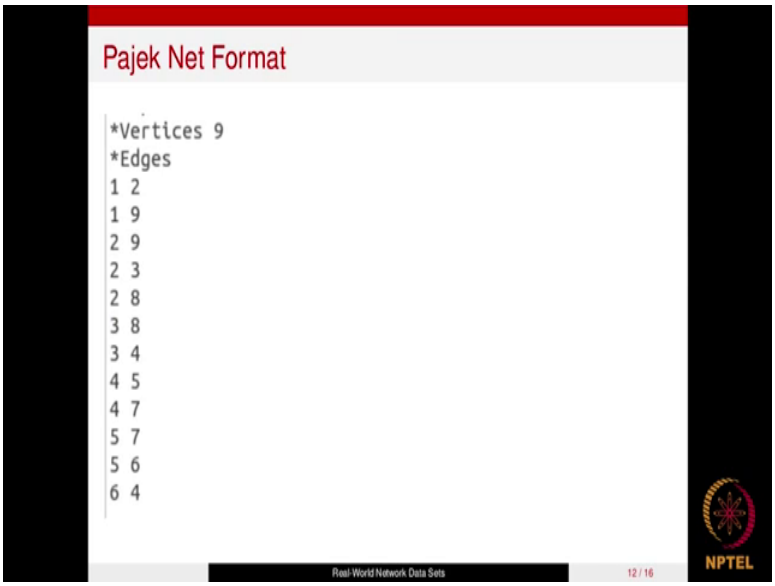
(Refer Slide Time: 11:46)



We have Pajek format which many people use for network datasets this kind of format has extension.net and let me tell you that in some cases you might also find networks with an extension.paj that also indicates a Pajek format network. So, it does not make any difference. So, you have either dot net or dot PNG format networks and the way the way of handling and the syntax is completely the same. So, let me show you the syntax of the file that is a structure of the file. So, you will start with the keyword star vertices and after that will be a number return which basically mean a number of vertices in the graph or the network after that you would have on every row you have these numbers one 2 up to n and after the number you would have some string written which basically indicates the label of that node.

So, you see that for every row you have every node return and once all the nodes are done you start with the information about the edges by writing with a star arcs or star edges this will basically come below it. So, every row contains the source node and the target node and so on.

(Refer Slide Time: 13:16)



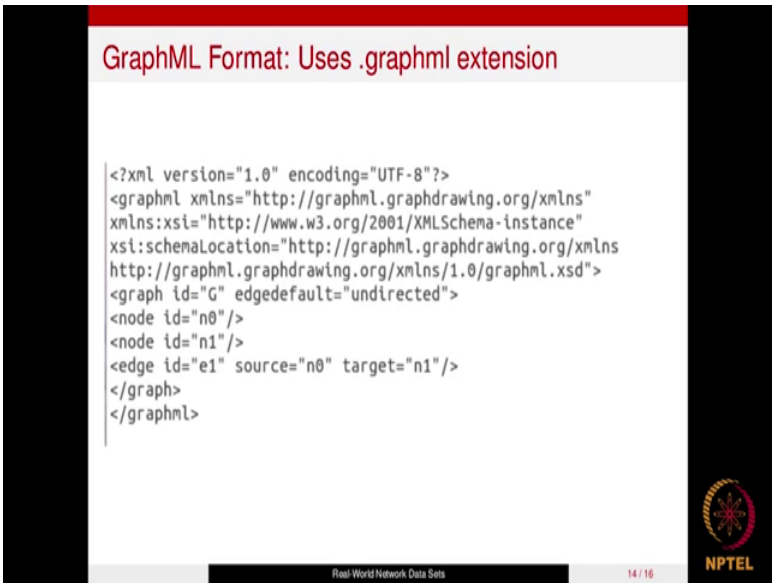
So, this is the basic syntax of a Pajek file and there is one more thing that I should tell you in case there is no labeling assign to the vertices of the network you just do not write anything below this vertices keywords because that does not make any sense you just write the there is no need to write the nodes consecutively one to n. So, you just leave that blank you just write the number of vertices that the graph contains and afterwards you start with the details on the edges and that is the same and in case you want to add some attribute to the edges you can do that as well.

(Refer Slide Time: 13:49)



Let me show you an example. So, here this is a source node and the target node and then you have value written this value basically is an attribute for this edge. So, in this case it is sort of a weight assign to this edge. So, that was about the Pajek format.

(Refer Slide Time: 14:08)

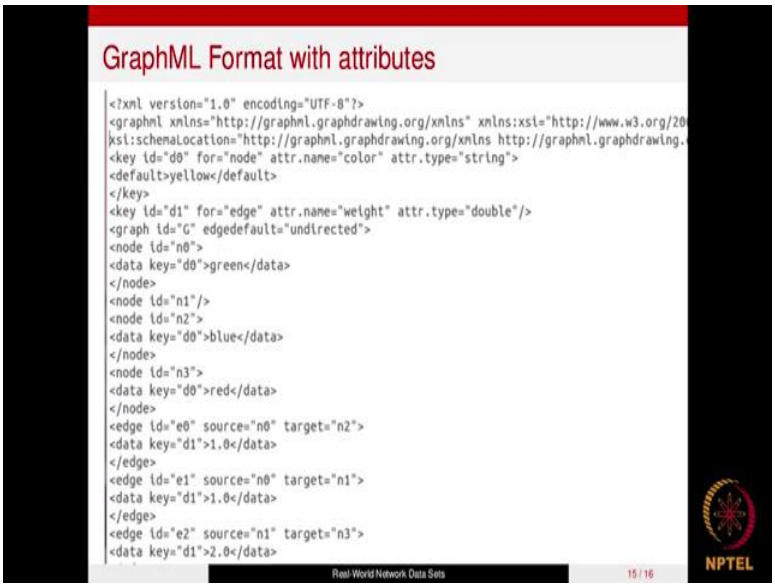


So, the next format is graph ml format ml here stands for xml basically you might be knowing what xml format is this basically provides the hierarchical structure and it makes use of various tags just like html and that is how you store the details in the form of tags. So, in a graph ml file you would have several tags let me quickly tell you about these tags. So, initially there will be an xml tag and after that you would have this graph ml tag and an inside the graph ml you would have a graph tag right and after the graph tag you would have number of node tags once the node tags are done you would have number of edge tags. So, this is the basic structure.

Now, let me get back here to the graph ml tag here you see some namespaces details basically you see some metadata you might be knowing the advantages of using namespaces and all that they allow different users to use different names and as long as I making use of different namespaces. So, I do not think you need to get into all that this is just a metadata about this network you can start from here you see a graph tag and then you have an attribute of graph which is edge default which is undirected and then you have node you have 2 nodes here and you have one edge here. So, this is a source and target keyword that we used inside the edge tag this is the basic structure of a graph ml

file you might you might see the structure you might feel that structure is little complex, but the flexibility that it provides in terms of assigning attributes to the nodes and edges is quite better than asking by 2 other formats.

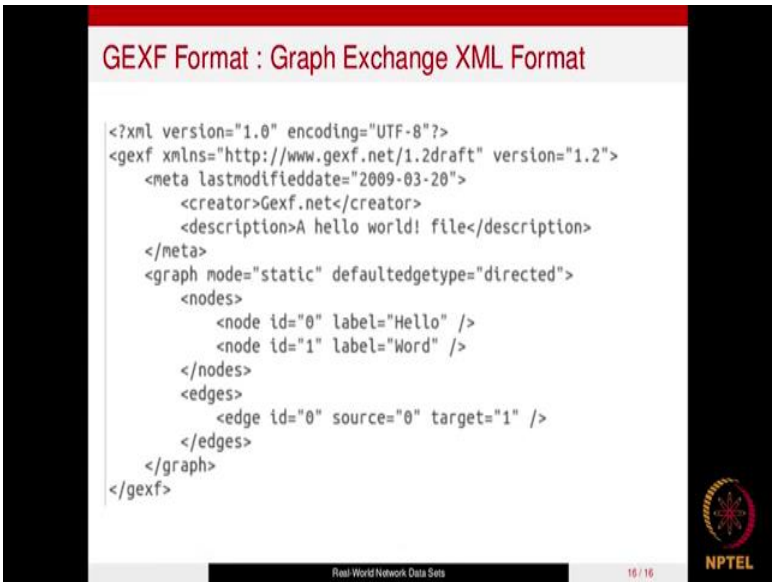
(Refer Slide Time: 16:02)



So, let us look at one more example from graph ml where there assigning attributes to the nodes and edges. So, after you done with this GraphML tag you start a key tag this key tag for example, is for nodes and this is for assigning colors and after that you have another key tag which is for edges and that is for assigning weights and once you done with the key tags you start the graph tag as usual and when you want to assign an attribute to a node you start a data tag and inside this data tag you have you make use of this key that is d0 you are making use of this key and you are assigning green color to this node in the next one you are assigning blue color to this node and red color and so on.

Similarly, you are assigning a weight 1.0 to the edge by making use of this d1 key, you are assigning a weight 1.0 and so on. So, this is how you assign attributes to the nodes and edges. So, that was about the GraphML format.

(Refer Slide Time: 17:07)



Let us go on to the last format in our list which is GEXF format this stands for graph exchange xml format this format was basically created by Gephi people Gephi is an open source software which is used for visualizing and analyzing social networks. So, this format is again inspired from xml as you can see it is composed of many xml tags. So, you start with an xml tag and we start with GEXF tag and inside that you have this meta tag to store the meta data about the network and then you start the graph tag inside the graph tag you have nodes tag and inside the nodes tag you have all the nodes and then you start the edges tag inside the edges tag you have the edge tag.

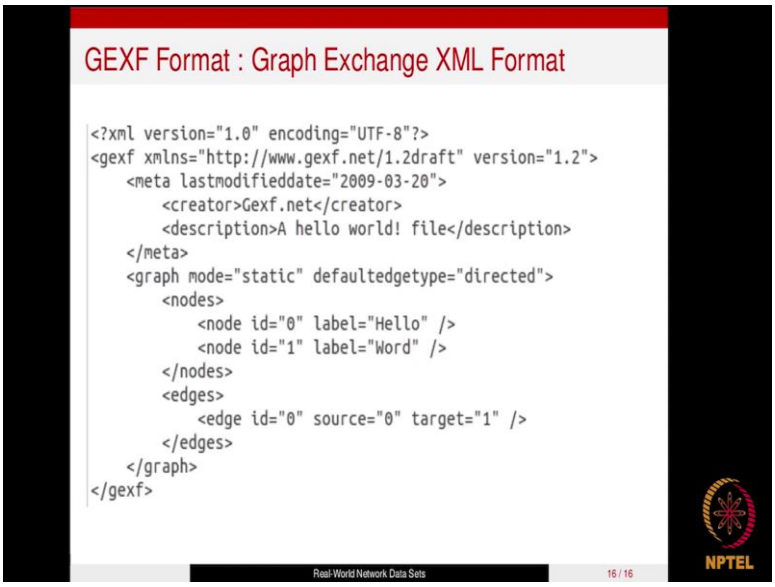
So, the format is little similar if it actually quite similar to graph ml format the previous one this is a little cleaner in terms of assigning various attributes to the nodes and edges. So, there is one thing that I would like to point out here. So, I want to say that do not worry if you not understanding anything out of this video because the aim of this video was just to introduce to you the main formats into which the datasets are available and in case you are not creating your own dataset you might not able to me to you know get into also integrate details in case you are just using the datasets downloaded from the internet you might not need to yourself write all these details that I just explain to you.

So, the aim was just for you to know you now get introduced to these statements and in case you get file you would be able to make some sense out of it that was your only aim. So, do not worry if you do not understand these integrate details.

Social Networks
Prof. S. R. S. Iyengar
Department of Computer Science
Indian Institute of Technology, Ropar

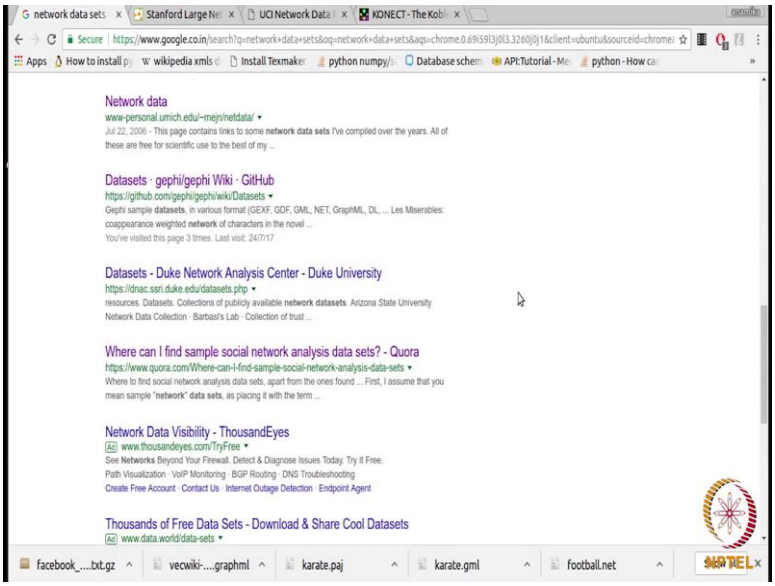
Lecture – 20
Handling Real-world Network Datasets
Datasets: How to Download?

(Refer Slide Time: 00:05)



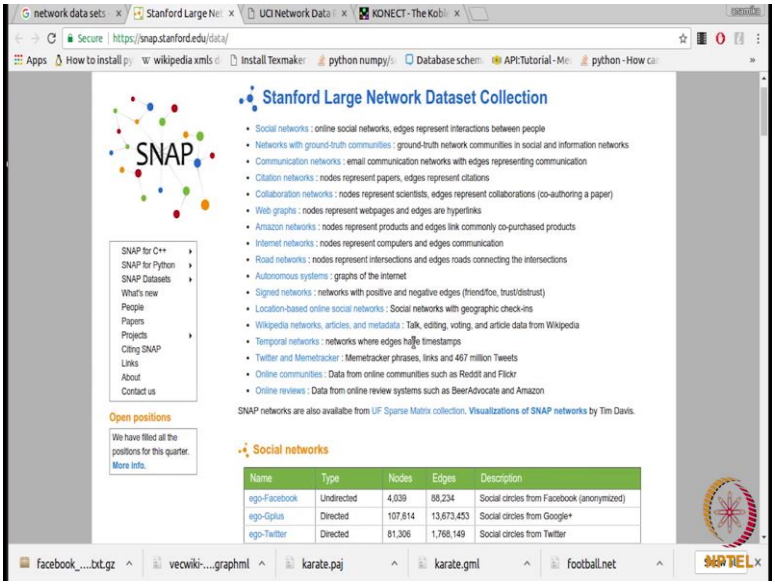
There is one more point to note here, networkx provides various functions through which we can read a network in one format and we can write the network in another format for example, we can read a network in Gmail format and we can write the network in GEXF format. So, those functions available and we can make use of them now let see how we can download these networks in different formats.

(Refer Slide Time: 00:34)



So, when we google for network dataset let us see which options do we get. So, here we get repository wise Stanford university which is called snap dataset let me open this and you also get a repository by UCI that is maintained by university of California and we also get this connect repository which is maintained by Koblenz university as you go down you get a number of more resources.

(Refer Slide Time: 01:05)

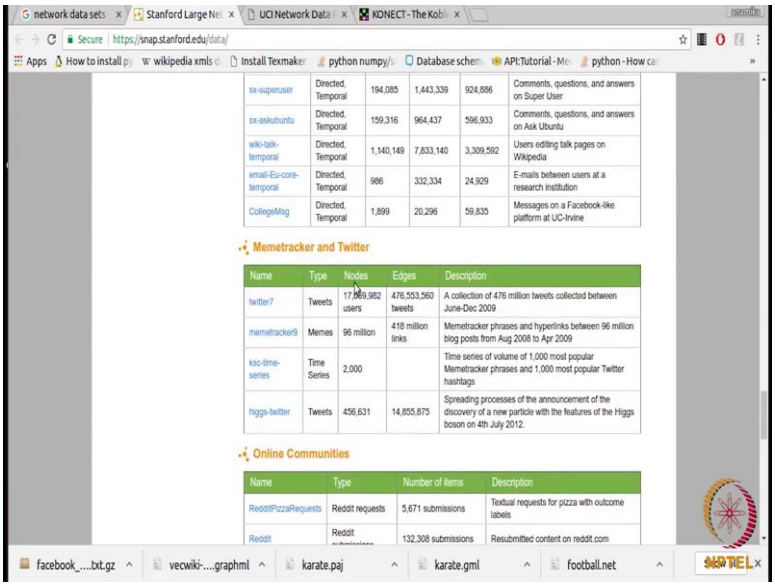


Let me show you this one first. So, snap dataset repository is the most commonly used repository for accessing the network data sets. So, here you get a number of networks of

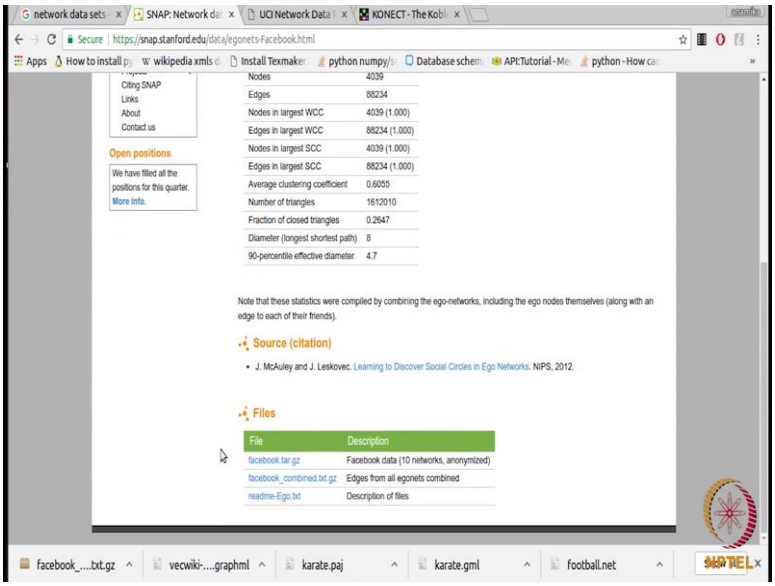
different types for example, you get social network like you like the networks on Facebook, Twitter, Slashdot, Google plus and you also get networks which have ground truth community what do you mean by that you know there are various communities of nodes in the networks. So, if there is a community detection algorithm you will come across these algorithms in the next videos. So, if you want to check whether a community detection algorithm is working fine or not, we might need to verify it.

So, these are the networks where the community details are given as ground truth values. So, they are useful in such scenarios then we have communication networks for example, networks e for example, email networks and we have citation networks and collaboration networks we already discussed about these networks and then we have web graphs we have Amazon networks, we have road networks and we have Wikipedia networks as in the networks between the articles and we have temporal networks now temporal networks are the ones which also contain the temporal information of the events. So, timing details will also be there, and we also have networks on stack over flow here. So, you see there are so many networks available over here and then the basic details you can see the number of nodes number of edges etcetera.

(Refer Slide Time: 02:45)

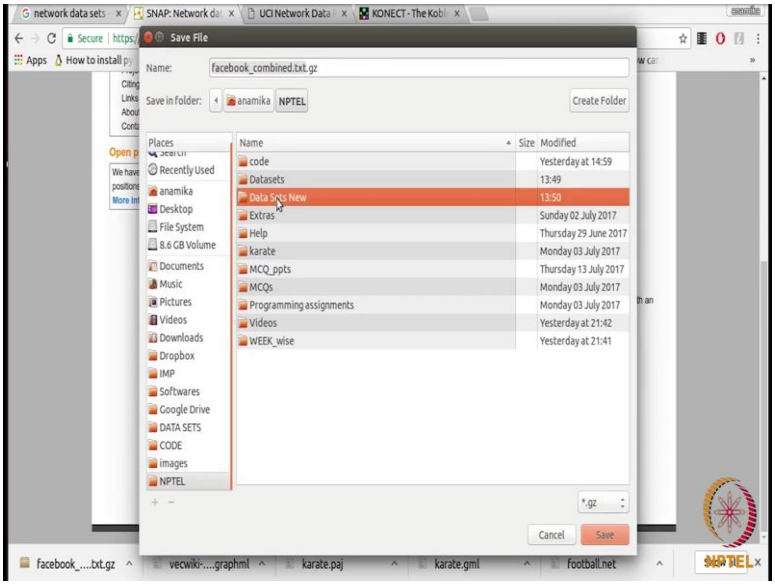


(Refer Slide Time: 02:52)



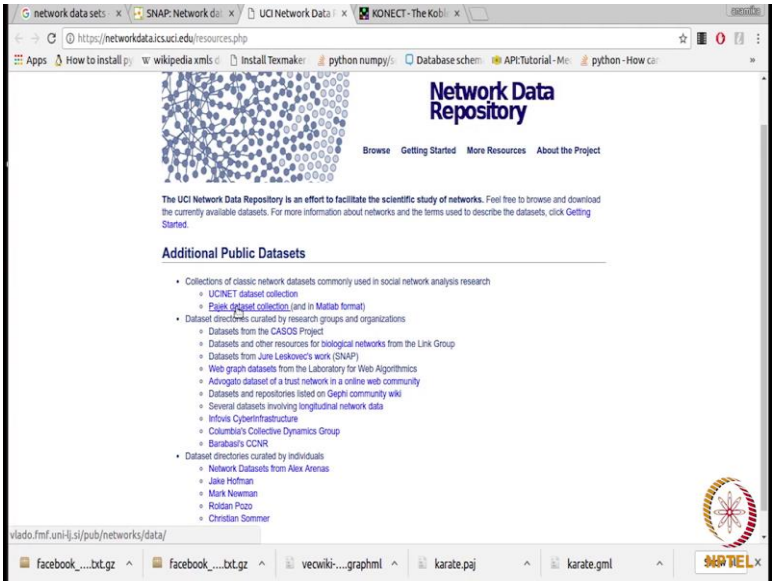
Now, let me show you how we can download one of the networks our; of these network. So, let me let me first open this and see the details. So, here you can read the details about the network and then you have basic statistics about the network and as you go down you have different files regarding the network. So, here we see 3 files one is readme file and the first file is having ten networks, the second file is having edges from all the networks you know combined. So, let me download this second file.

(Refer Slide Time: 03:31)



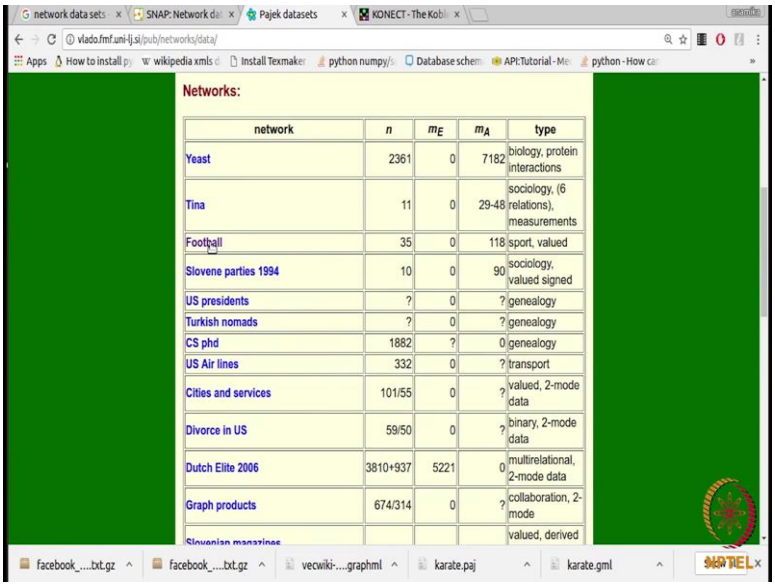
So, let me save this file and we will open it later to see its structure. So, this is how you can download the networks.

(Refer Slide Time: 03:43)



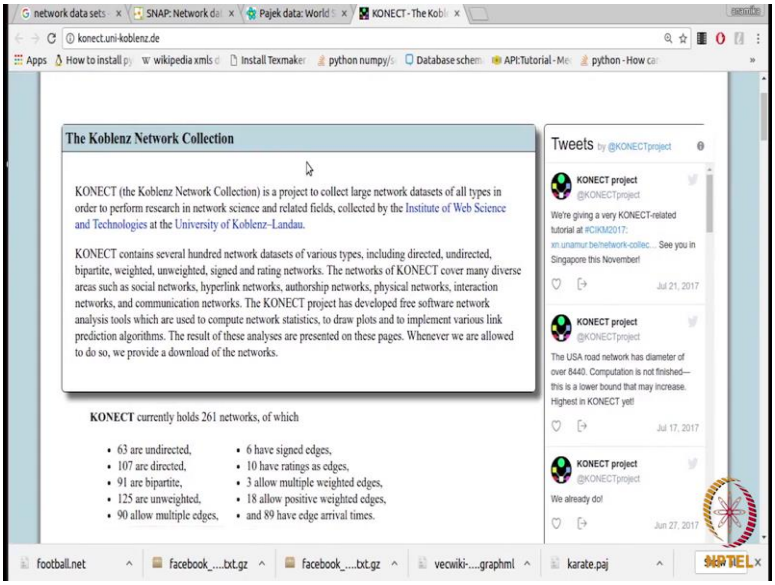
Let me show another repository this is maintained by university of California and they are also you see their in a given additional public datasets you can you can explore all these lines and get different kinds of network since we also have to download Pajek network and click this link. So, here we have all the Pajek datasets.

(Refer Slide Time: 04:02)



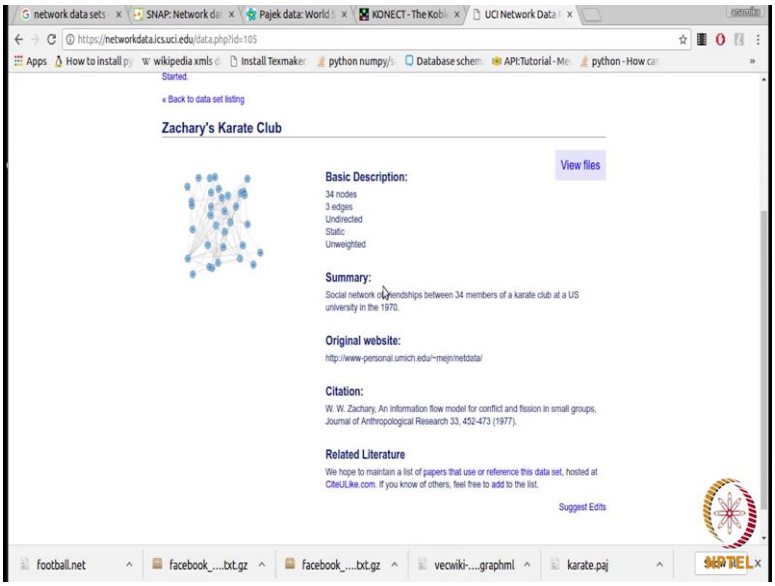
Let me download the small network just to show you the kind of structure that Pajek file contains. So, I am saving this I am sorry let me first see this. So, we have to save this file. So, this is a dot net file I told you previously that the Pajek files will be available in one of the 2 formats either it will be dot net, or it will be dot paj. So, this one is dot net a very small network just to show you the structure.

(Refer Slide Time: 04:43)



Let us go to the next one. So, here you see this repository again is having a lot of networks on different topics you see Wikipedia site you like and there are air traffic control there are different topics and you can just choose based on your requirement there is twitter dataset. So, you can download the data set based on the resource that you want to access or you can download dataset based on the format that you want for example, I want the data from say UCI.

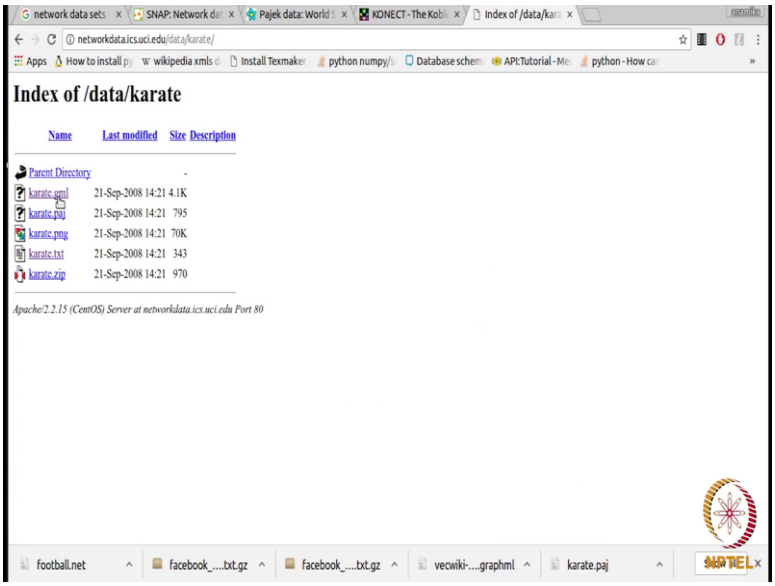
(Refer Slide Time: 05:24)



So, I click this and here you have all the network from this repository there is an interesting network here which is the Zachary Karate, let me download this. So, there is an interesting history attached to this network there was a karate club in us university and it had 34 participants, there was a fight that happened between 2 important people of that club and they were the instructor and the administrator.

So, after a period of 3 year the due to this fight the network got divided into 2 communities Zachary was a person who analyze this network over a over this period and he Indian basically predicted the communities that are going to form. So, this network is well known and it is called the Zachary's karate club network and it is very small as well its use for the community detection algorithms as well and it is also a nice starting step for you to start your analysis on the networks. So, we are going to download this network.

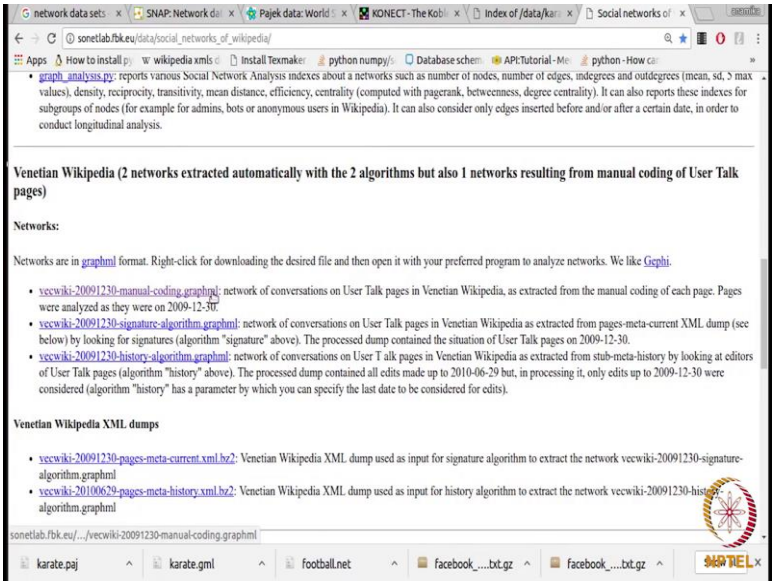
(Refer Slide Time: 06:35)



This network is available in GML format as well as Pajek format. So, let me download both of them and then we have this Pajek. So, we have done with Ajelius format and dot net format and dot gml format and dot Pajek format.

So, we are left with GraphML. So, let me download one network in that format I know one resource a sonnet is a repository that contains data in graph ml format.

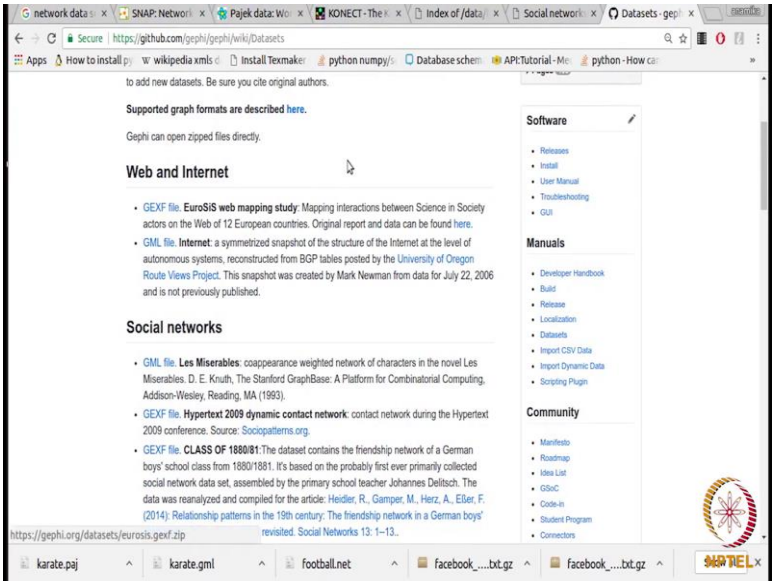
(Refer Slide Time: 07:12)



So, let me just open that only here these are the Wikipedia network which are in graph ml format I had already downloaded this once. So, I will access that only. So, you can

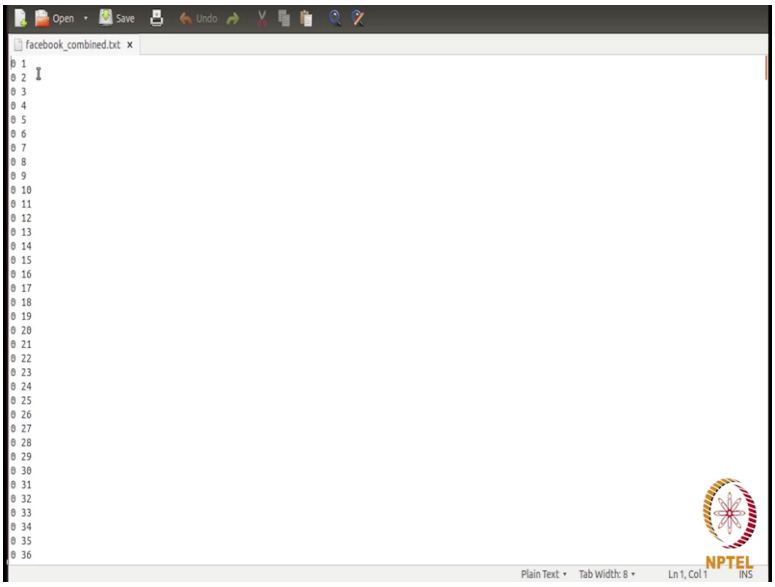
download all any of these networks and you can basically read the details about the network and then accordingly download if it chooses your purpose.

(Refer Slide Time: 07:47)



Now, let see GEXF format let me show you how we can download that let me open this link. So, here you can see various networks in GML as well as GEXF format. So, you can download any of these after reading the details I have already downloaded one in GEXF files. So, I will show you that only.

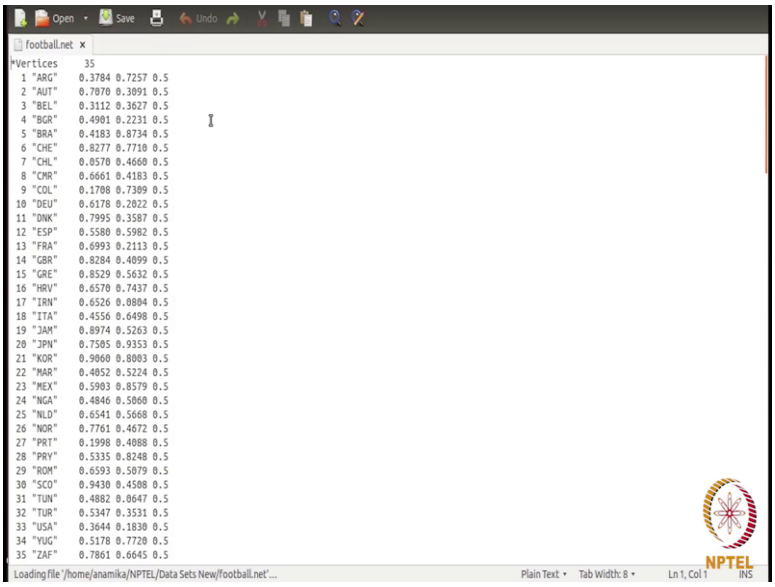
(Refer Slide Time: 08:05)



So, let us look at what we have downloaded we have all these networks let me extract this first. So, here we have six network GEXF format txt format dot net format dot gml format dot Pajek format and dot graph ml format. So, I am quickly going to show you the structure of these files although I had already introduce the formats to you let me first open the txt format which is in edge list format.

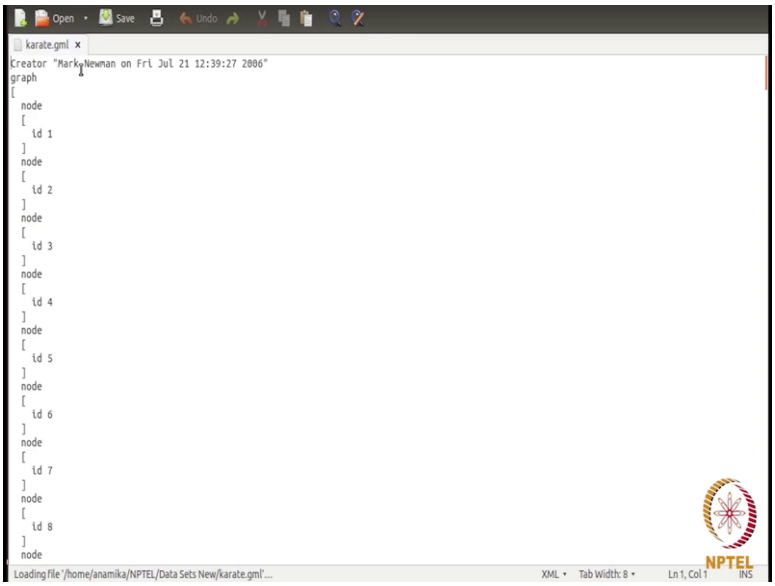
So, you can see this simplicity of the format again you just have 2 things in every row and these 2 things are the source and the target of the edges. So, there will be a link from 0 to 1, 0 to 2, basically this is undirected. So, there will be an edge between 0 and 3, 0 and 4 and so on. So, this is the edge list format.

(Refer Slide Time: 08:59)



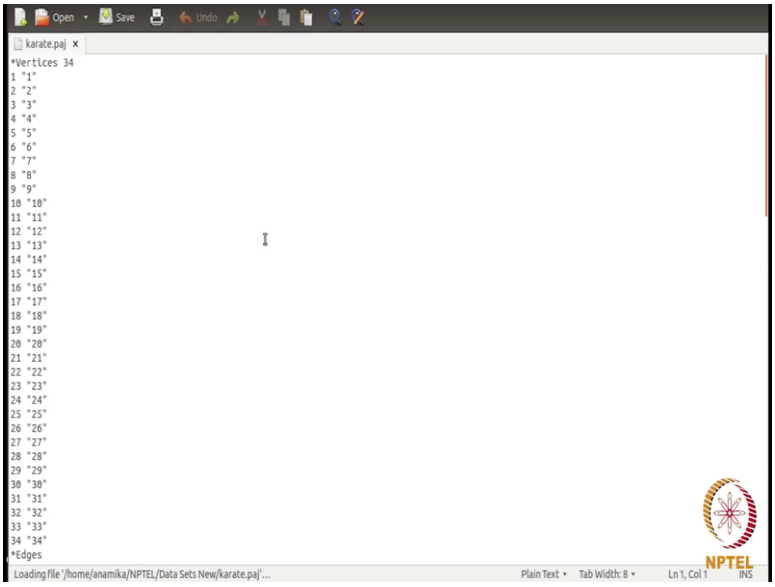
Next let us check the dot net format. So, as I told you this starts with the key words star vertices and then you have the number of vertices in that network. So, we these are the ids and these are the labels of the vertices and then we have 3 details attached to every vertex which are the attributes and you can basically see the documentation to see what these attributes mean then after the nodes are done you have these arcs which are basically the edges and for every edge you have this attributes attached.

(Refer Slide Time: 09:40)



So, this is an example of dot net file and then we have GML. So, you see there is a graph keyboard and then you have square brackets and then you have all the nodes. So, first all the nodes will be there and then once the nodes are done you have these edge details. So, this is a Gmail format. So, this is Zachary karate network which had thirty-four nodes.

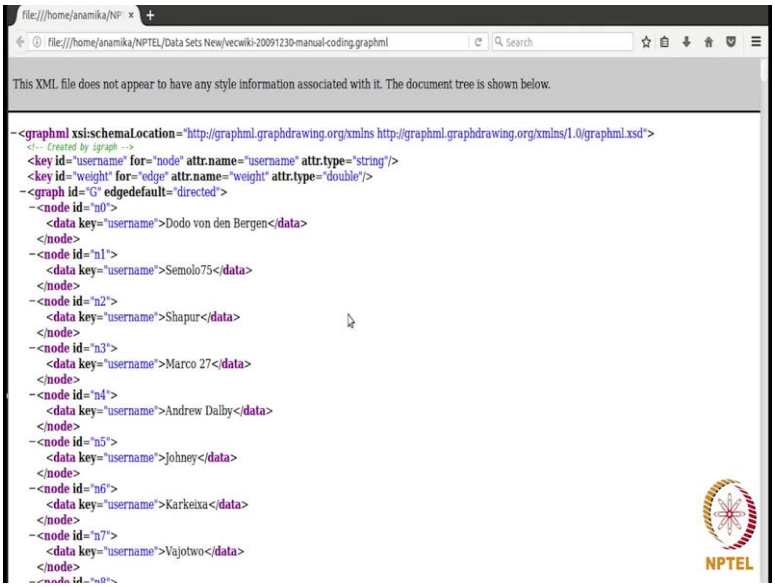
(Refer Slide Time: 10:05)



Now, this is a karate network in Pajek format, let me show you that here there is no attribute for the vertices and there is no attribute for the edges as well. So, this is again a very simple network in Pajek. Now let me show you graph ml format. So, this is an

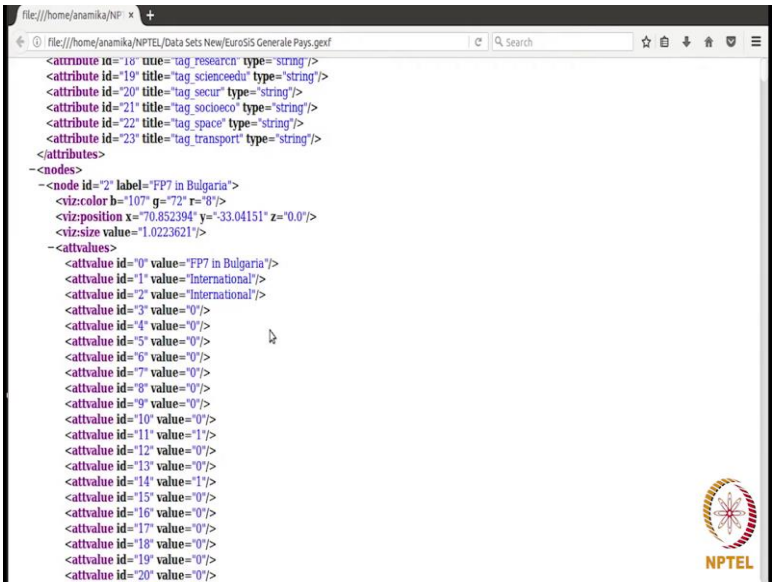
example of a GraphML network. So, it as the number of tags the first tag is GraphML xl and after that we have 2 key tags. I told you that key tags for adding the attributes to nodes and edges.

(Refer Slide Time: 10:26)



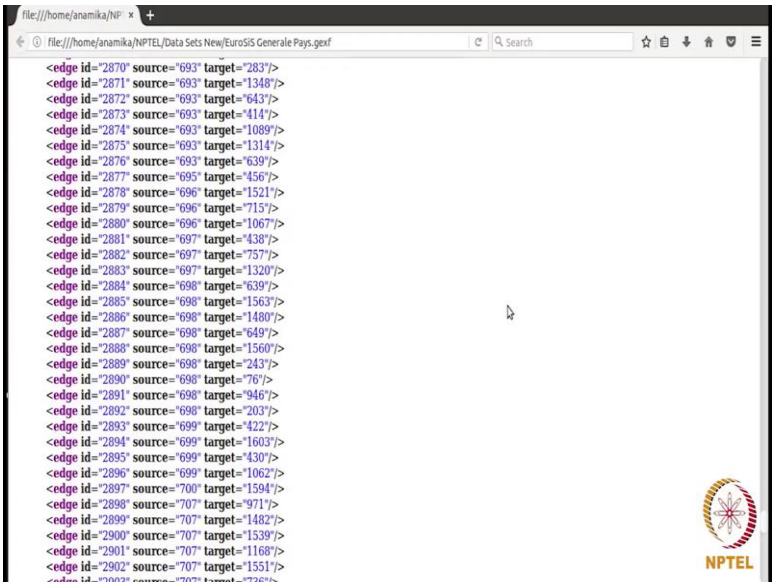
So, first one is for nodes and the second one is for edges and then we start the graph tag and inside graph we have a number of node tags and the nodes are given attributes the key using the you know data tag and we are making use of this key and as you go down after the nodes are done you can go down gap after the nodes are done you have the edge stacks. So, again the edges are given attributes using the data tag.

(Refer Slide Time: 11:21)



Now, let us check the GEXF format. So, here you see there are again because this is also based on xml there is GEXF tag inside that we have graph tag and there are. So, many attributes for this graph and after that we have nodes tag inside these nodes there will be all the nodes. So, the first node is here and then there are. So, many attributes for this node and then second node comes with all these attributes. So, basically here in this network they are assigning a lot of attributes for every node as you can see. So, as you go down once the nodes are done the edges will be there it should be the yeah. So, here you see once the nodes are done, we have edge tags they have not assigned any attributes for the edges as you can see. So, this is an example of GEXF format.

(Refer Slide Time: 12:01)



So, you saw there are so many resources available, you can just explore and download the once which suit your purpose for an analysis. So, this is the basic introduction to how we can download datasets. Next, we will see how we can analysis these network datasets that we have downloaded.

Social Networks
Prof. S.R.S. Iyengar
Department of Computer Science
Indian Institute of Technology, Ropar

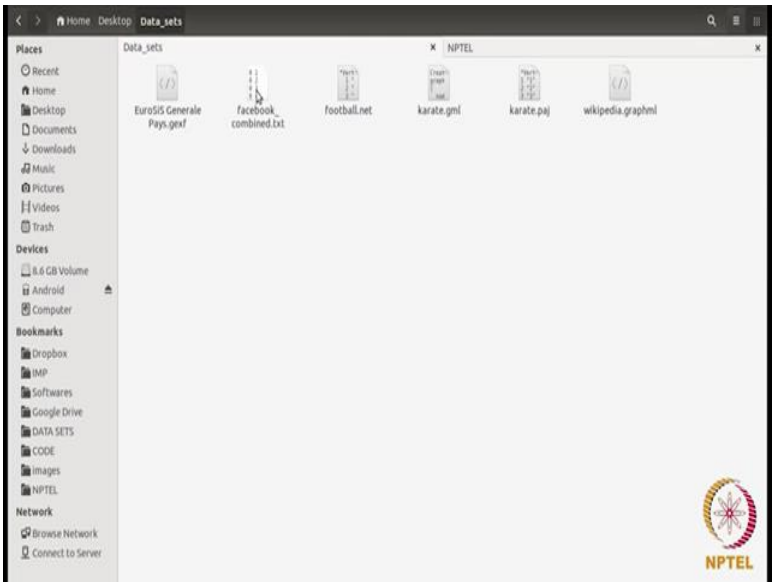
Lecture - 21
Handling Real-world Network Datasets
Datasets: Analyzing Using Networkx

(Refer Slide Time: 00:06)



Hey everyone in the previous video we had downloaded a number of network datasets in different formats, in this video we are going to see how we can analyze them using networkx package of python; let us take a look at the datasets that we have downloaded.

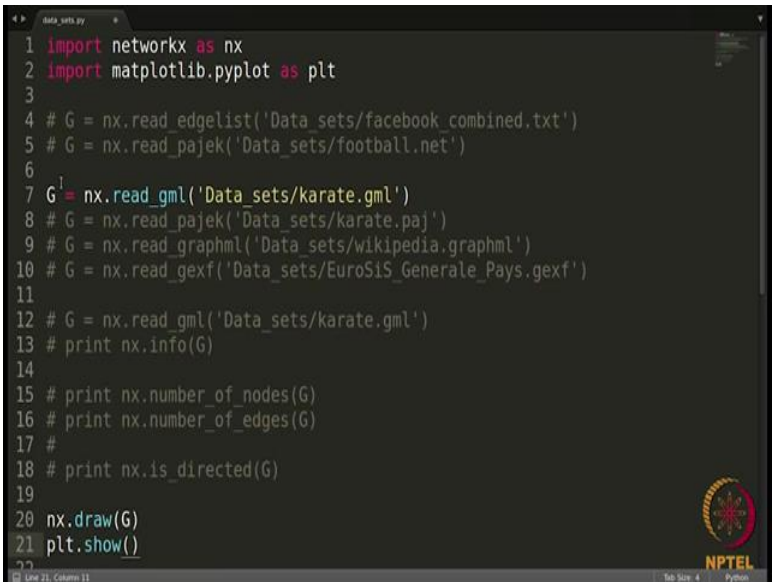
(Refer Slide Time: 00:25)



So, these are the datasets that we downloaded in the previous video. So, we had 6 networks, we have a network in gexf format, we also had a Facebook network in Edgelist format, we had a football network in dot net format which it is equivalent to Pajek format and we had karate club network in gml format, we also had karate network in Pajek format and we also had a Wikipedia network in GraphML format.

So, these were the 6 network datasets that we had, now we going to see how we can analyze them. So, I have all the datasets in this folder, I am going to create a new python file here where I will be writing all the 4. So, datasets.py I am going to open in it in an editor I am using sublime text here can use any editor for that matter.

(Refer Slide Time: 01:26)

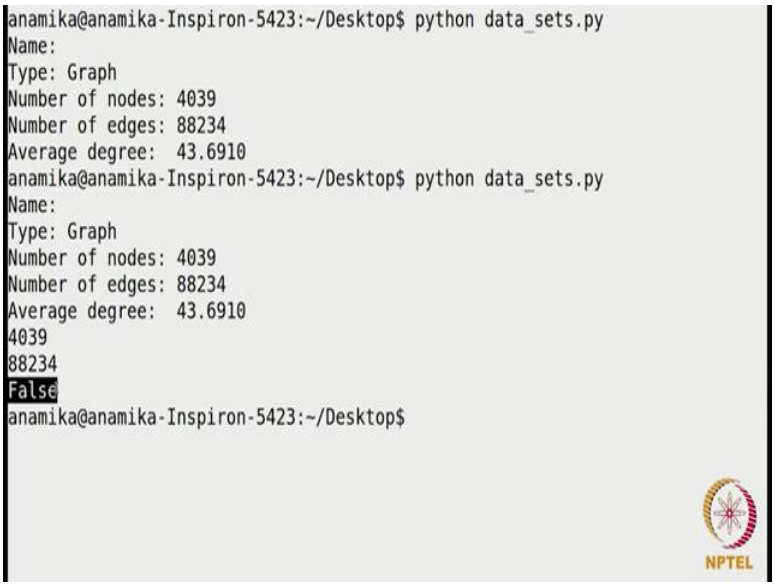


Since we are going to make use of networkx package and I am going import it, we are also going to visualize the networks. So, I am going to import matplotlib as well. Now let us take the first network in this folder we have let us make use of this Facebook combined dot txt network which is in Edgelist format let me copy this name.

So, now since the Facebook network is in Edgelist format, the function that we are going to make use of is read Edgelist. So, I am going to write g = nx.read_edgelist and here I am going to get the name of the network. Now our datasets are kept in a folder. I am sorry I think that is the name (Refer Time: 02:30). So, in that we have this is the name of the network. So, the you see the function that we are using is read_edgelist function, which is present in networkx package, and as a parameter we are giving the name of the network. Now this function basically takes the network in a in the Edgelist format and returns a graph object; and then we can apply any function on this graph object.

For example, if you want to look at basic information of this network, we can write nx.info on as a parameter here give G. So, info is function which provides a basic detail as to the number of nodes number of edges etcetera about to graph. So, let us save this file.

(Refer Slide Time: 03:26)



And I am going to open my terminal here, I am going to run this file. So, all right. So, here you see firstly, it tells us the type; the type is graph as in it basically tells us whether it is digraph or directed graph or it is multi graph and it tells us the number of nodes, it also tells us the number of edges and the average degree.

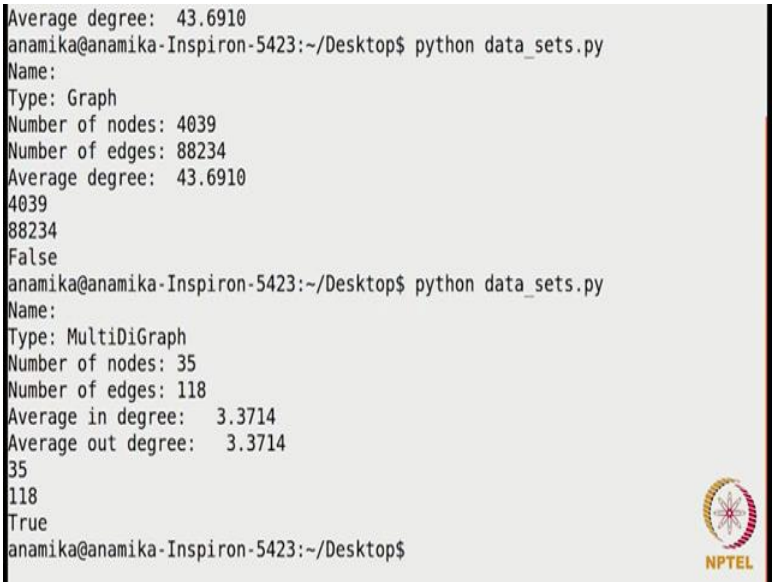
So, these are just a basic detail about the graph, now let us go back to our file and add some more things. So, this is what the basic things if you just want to get the number of nodes you can write. So, now, nx.number_of_nodes is a function which returns you the number of nodes if you have to just get the number of nodes, you can use this function and similarly if you want the number of edges if you do not want all the other information you can use this. And in case you want to know whether the graph is directed or not, then you can make use of this function print nx.is_directed. So, as a parameter you pass the graph.

So, this should tell us whether the graph is directed or not, let us go back and try to run this. So, here you see after the basic statistics it told us the number of nodes and edges and it is false that is it is non directed. So, this Facebook network that was a friendship network it is basically undirected network. So, that was about how we can we read and Edgelist network into networkx object. Now let us see the other kinds of networks that we have, we also have here dot net format as I told you in the previous video that dot net

format is basically the Pajek format. So, in order to read this network into the networkx object we will use Pajek functions.

So, let me show you how to do it. So, what I am going to do is I am going to use this function read Pajek, which is a function that is used to read a dot net or dot paj file. So, let me check the name of the network football dot net. So, change it. So, I am reading this dot net network through this function read Pajek into a graph object g, and then I applying all these operations let us see and we run this.

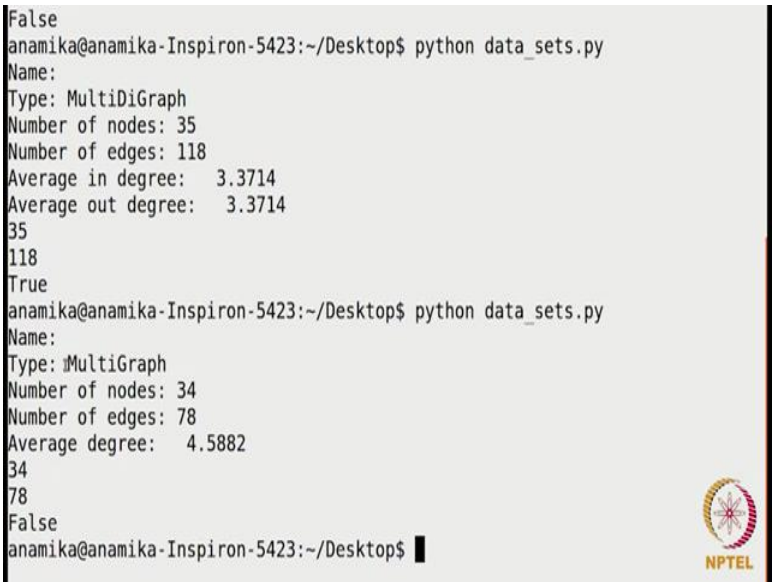
(Refer Slide Time: 06:10)



So, here you see that the type of graph is multi digraph; that means, there are multiple edges between the nodes, and it is also directed graph and it is telling us a number of nodes number of edges, and since it is a directed graph it is telling us the average in degree and average out degree, and after that again it is giving the result of the functions number of number of nodes number of edges, and it is true which means it is a directed graph.

So, this these are just the basic functions, let us see what other kinds of networks we have. We have gml network we have a Pajek format as well let me show you that for reading the Pajek files as well, you use the same function that is read Pajek. So, the network name is karate dot p a j. So, I will replace this ok.

(Refer Slide Time: 07:15)



So, when I run this, I am getting that this is a multi graph and the number of nodes is 34 number of edges is 78 and this is a average degree and it is not a directed graph. So, it is. So, I am getting false here ok.

(Refer Slide Time: 07:35)



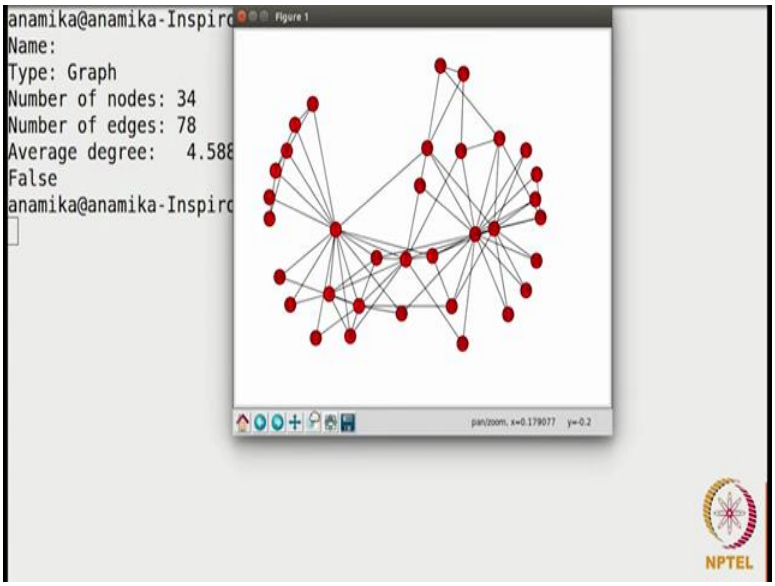
Now, 2 more network formats that we have are GraphML and gexf. So, let me show you quickly show you how we can read them as well. So, graph m l how do we handle it we. So, let me the function that we use it read graph m l and the file name is Wikipedia dot graph m l. So, this is how we will use it; since this is written information, I am going to I

am going to comment this, now let me run this all right. So, what we are getting here is that it is a digraph Wikipedia because this is graph between that the nodes are the articles. So, is it basically tells us whether an article is referred to in the other article or not? So, it is basically a directed graph 921 are the number of nodes number of edges are giving and since it is directed we are getting in degree as well as the out degree, the average in degree and out degree and it is true that means, it a directed graph.

So, let us go back here and the only format that is left is gexf, let me quickly show you how to convert into graph object as well and copying it is name and here let us go back. Since this is a gf; gexf format will make use of function read_gexf and (Refer Time: 09:24) name of the function let me rename this, so that it does not create any problem and let me rename this is well. So, it should work let us see. So, this is how we can read various networks in different formats and convert them into a graph object; once they are converted into graph object, we can apply various functions on them, and we can play around with them. Now let also show you how we can visualize a network in networkx package.

So, let me take a small network, let me take karate network itself and we had it in gml format. I think we have in that video we can quickly add it. So, the function is read gml. So, if you have a graph which is in gml format, the function that you will used is nx.read_gml. So, all right. So, we read a karate network which was in gml format, we made use in function nx.read_gml.

(Refer Slide Time: 11:05)



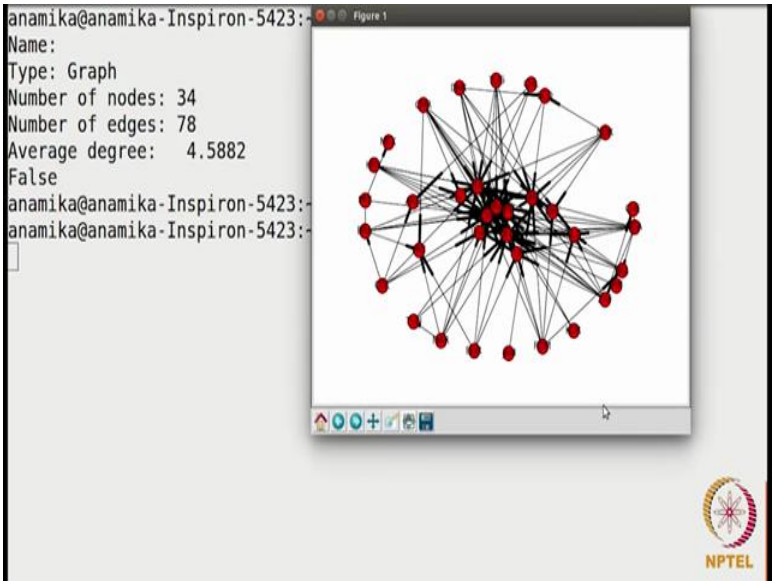
So, let us execute this program. So, this is a simple graph, and this is a number of nodes and edges and it is and undirected graph. Now let me show you how we can visualize this graph? I am going to comment this; I am going to comment this as well.

Now, the function that we used to visualize the graph is nx.draw, and the parameter we will do the graph that we have to draw in order to see that graph we have to use this function plt.show, basically the show function which is available in matplotlib. So, that is how we will be able to see this graph, now let me run this all right. So, this is all graph the karate network, and the labels are given here. Let me also show you a few features that this interface provides for example, this when you click this you can just move the graph the way you want, and the next option is zoom to rectangle. So, when you click that if you want to carefully observed some part of the graph you can just zoom it and see and if you further want to do that you can do the way you want and then you can go back then you go then go back. So, these are few functions that few features that you can make use of, and this is configures subplots this will be used when we plot something this is just a graph, I will show you the functionality of this later then this is how you can save the figure ok.

Let me close this window now let me go back to the program; let me also show you how directed graph in networkx looks like. Since this karate is in undirected graph let me

comment this and if I am not wrong this football network was a directed network. So, I am doing the same analysis on football dot net.

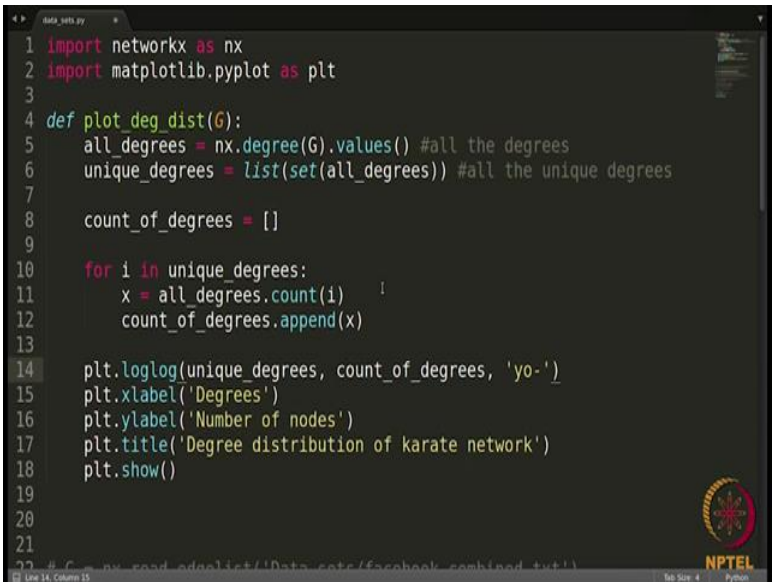
(Refer Slide Time: 13:13)



Let me execute it again, you see this is how directed graph and networkx looks like. So, you see the arrows are represented like this, if you want to zoom it again you can make use of this feature.

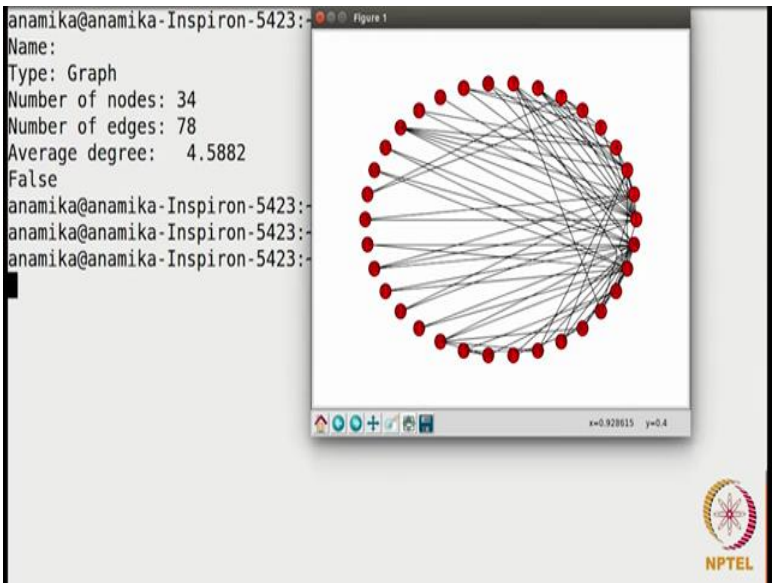
You want to zoom this area you can further zoom it is well. So, this is how you can closely analyze whatever you want in the graph right and you can then go back, or you can just press home here. Then you can save the graph is well let me close this now I am going to continue the rest of the analysis on karate networks.

(Refer Slide Time: 13:51)



So, I am going to on comment object back. Let me also show you that there are different layouts which are available in networkx for example, we have as if now use this function nx.draw, we can use another function if you want to different layout. So, we can use nx.draw_circular. So, this is one of the layouts there are various layouts available, let me show you the output that we get in this case.

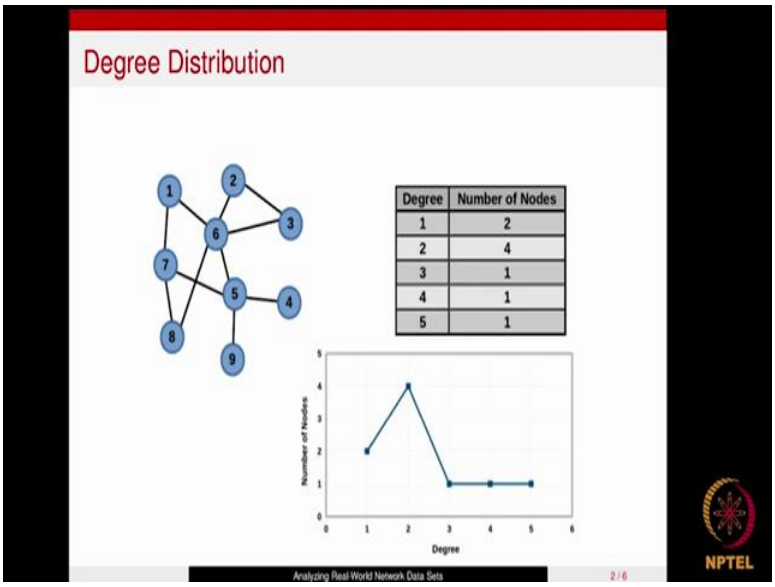
(Refer Slide Time: 14:23)



So, here the all the nodes are arranged along a circle and the edges between them are shown like this. So, this is called circular layout, there are number of other layouts as

well for example, spectral layout and spring layout. So, you can just read the documentation about them. So, we have visualized the network let us close this and let us go back to our program and we can do some basic analysis on this, one of the thing that we can check on this network is we can check the degree distribution.

(Refer Slide Time: 15:00)

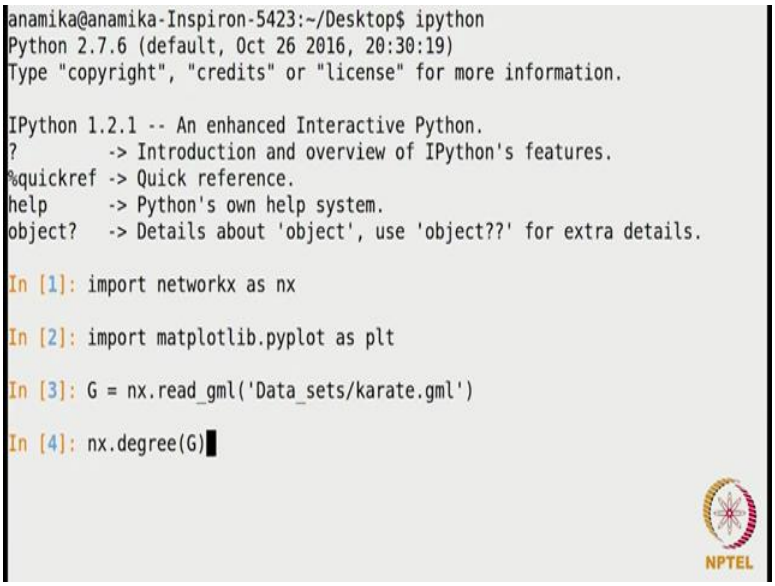


Now, what is degree distribution? Degree distribution basically tells us how many nodes there in the network are, that have a degree. So, this is done for all possible degrees that a node can have in the graph; a let us take this example graph. So, here how many nodes are having degree 1? So, node number 4 and node number 9 they are having degree 1. So, corresponding to 1 will have 2 and similarly we can check how many nodes are having degree 2, how many nodes are degree 3 4 and 5. So, these are all the possible degrees that nodes can have in this graph and corresponding to the degree we have the number of nodes that have that particular degree. So, this basically is called the degree distribution of the nodes in a graph. Now it is always nice idea to plot the degree distribution to get a better idea of the graph.

So, when we plot this, we get this kind of distribution. So, on the x axis we maintain the degree and on the y axis we maintain the number of nodes that have that particular degree. So, this is the kind of plot that we get for this example graph, we can check for our datasets what kind of degree distribution to the exhibit. So, let us go back to our program and let us try to check what kind of degree distribution this karate network has.

For that purpose, I am going to create a function to plot the degree distribution of this graph G. So, let me create a function here, before we implement function, I want to show you a few things on the ipython console. So, let me copy this and let me open the ipython console here.

(Refer Slide Time: 17:03)



So, basically, I am I just copied those first 2 statements here and let me also copied this statement here ok.

So, we have the karate network in the object G; now if I want to see the degree of each node in this graph, I can make use of this function nx.degree(G).

(Refer Slide Time: 17:33)



(Refer Slide Time: 17:36)



So, what this function does this degree function it basically returns a dictionary, where the key is the number of the node and the value is the degree of that node. So, here we get the dictionary for all 34 nodes, we are getting the degrees of these nodes. Now let us go one step back and recall what is our (Refer Time: 17:56) is to have the degree distribution of nodes in the graph.

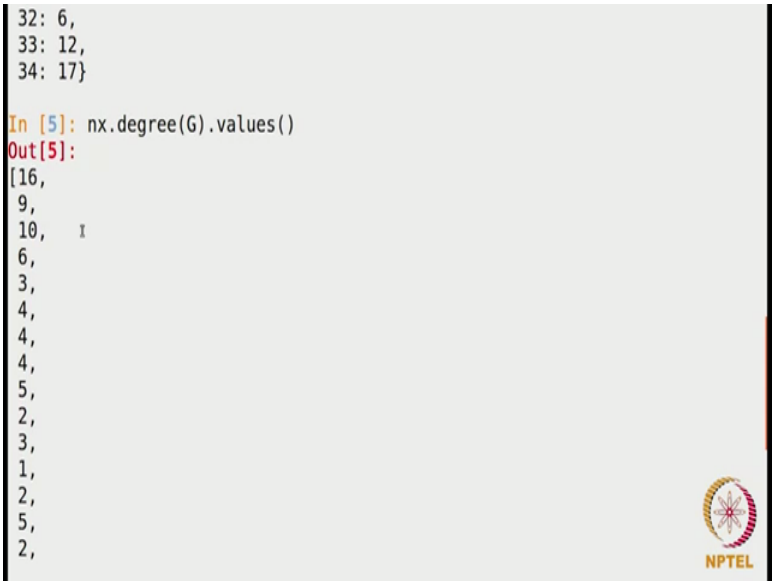
So, basically, we want that for particular degrees how many nodes are there in the network that are have in that degree. So, we basically first of all want to get the possible

degrees that the nodes are having in this network. So, we are getting the dictionary here where we are getting all the possible values of the degrees that is in that the nodes can have. So, what we are interested here in is basically the values. So, what I can write is nx.degree(G). So, this is going to give me a dictionary all I am interested in is the value. So, I am going to write dot values here.

(Refer Slide Time: 18:41)



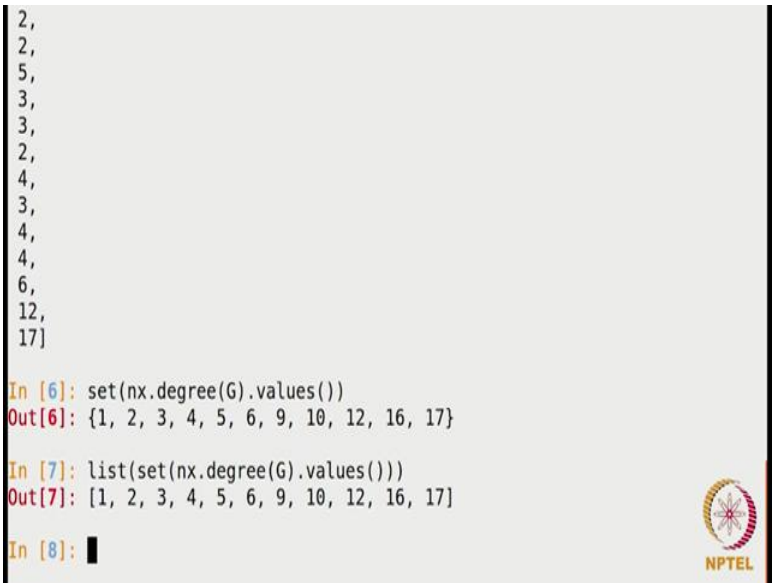
(Refer Slide Time: 18:43)



So, it should return me earliest which is having all the values. So, what is get here is basically all the possible degrees that the nodes are having. Now my aim is get the

possible degrees that the nodes can have; here you see there are lot of reputation. So, I want to get the unique degrees. So, in that case what I can do is, I can just write all this inside of function set.

(Refer Slide Time: 19:13)



So, what it will do is, it will convert the output into a set and we know that in set there cannot be in reputations. So, what we get is the unique values, basically the unique degrees that the nodes can have in this particular network. Now a list is more flexible data structure as compared to set because we can perform lot of operation. So, I can further convert this into list if I want; this is basically up to you how or you want to handle the data.

So, I convert this into a list now what I get finally, is a list of all the unique values of degrees that the nodes can have in this network. So, I should you here so that we can see what is going on in the function. Now let us get back and use these functions in the function that we are creating. So firstly, I want all the degrees. So, I will write nx.degree(G), I want all the values only I do not want the dictionary. So, I will write dot values. So, here I get all the degrees let me comment all the degrees.

Now, I am also interesting getting all the unique degrees so that I can see what the possible degree values is. So, what I am going to do the same thing that I did on the console, I am going to pass all degrees here. So, here I will get all the unique degrees. Now to get the degree distribution what I basically want is I want to out of all the values

that are there in this list unique degrees, I want to see how many nodes are having that particular degree. So, basically what I will do is I will fetch one element out of this list that is unique degrees, and I will see how many occurrences of that value there in all degrees are right.

So, probably I can start for loop here. So, I will write for i in unique degrees sorry for i. So, what I want to check is how many occurrences of i are available in all degrees. So, I can start a variable probably x is equal to all degrees dot count. So, we have this function count in a list, which tells us the occurrence number of occurrences of a particular element in that list. So, x will be telling us the number of occurrences of the degree i in the list all degrees. So, probably we can keep a track of all these values. So, we can create another list count of degrees I will I started a empty list and so I am going to append the x values to this list. So, basically the occurrences of the first degree are now stored in the list count of degrees.

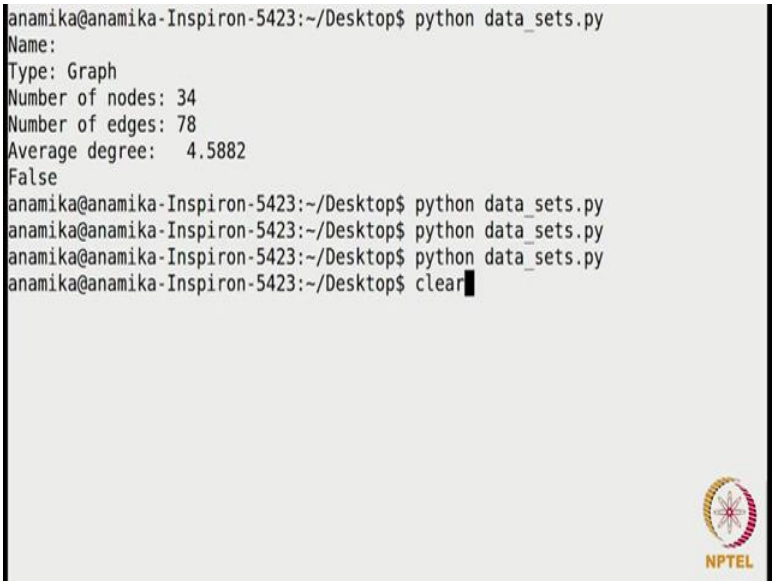
So, after this for loop is finished, we will have all the degree distributions in this list that is called count of degrees, and then we can plotted at. So, let us try plotting is it. So, plt.plot as you might be knowing there are 2 parameters that have to be passed here. So, on the x axis we want all the unique degrees, and on the y axis we want have many nodes have that degree which we have stored in count of degrees. So, will pass that here. So, let us try plotting it. So, I will write plt.show we have not call this function x.

(Refer Slide Time: 23:20)

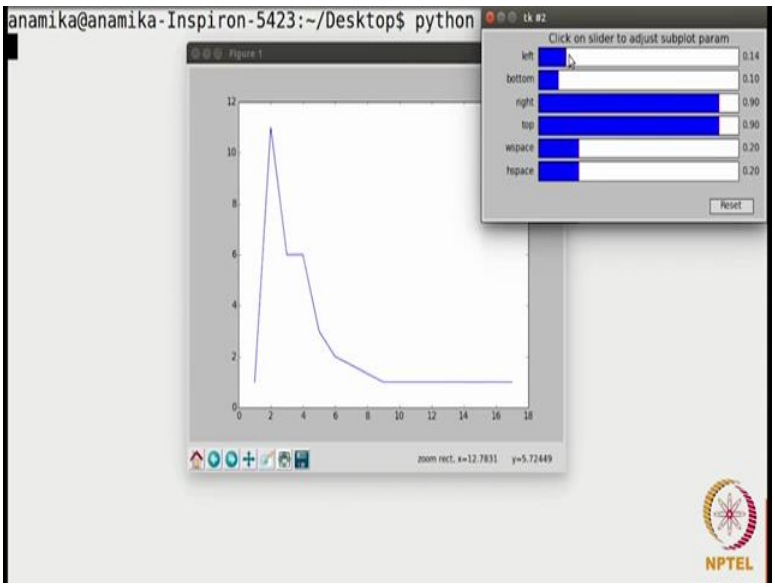


So, let me call this function here and comment this. So, I will call this function plot degree distribution and I will pass this G here ok.

(Refer Slide Time: 23:38)



(Refer Slide Time: 23:40)

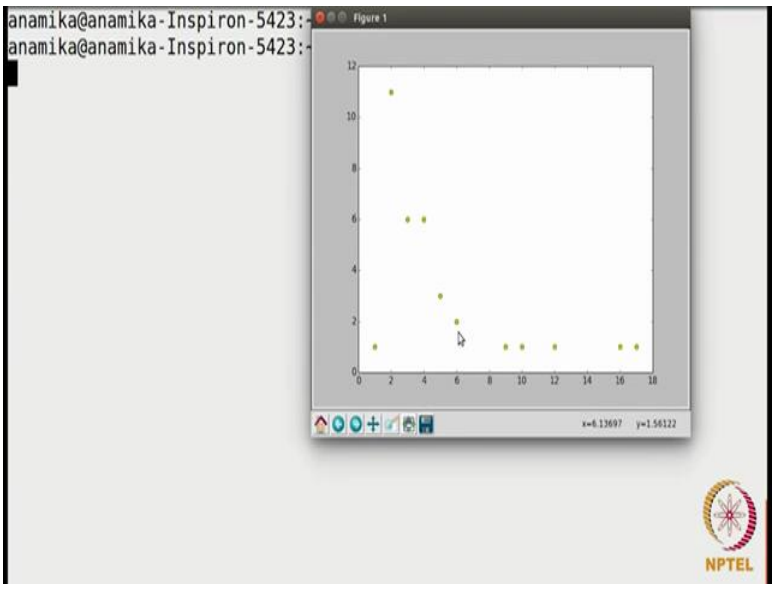


Let us go back here not here will go will try (Refer Time: 23:39) this. So, this is the degree distribution that we are getting for the karate club ah network.

So, this is the x axis where the possible degrees are there and on the y axis the number of nodes having that degree are there, and we you can just again play around with this plot as I told you, you can just use this to move the plot, you can use this to zoom of

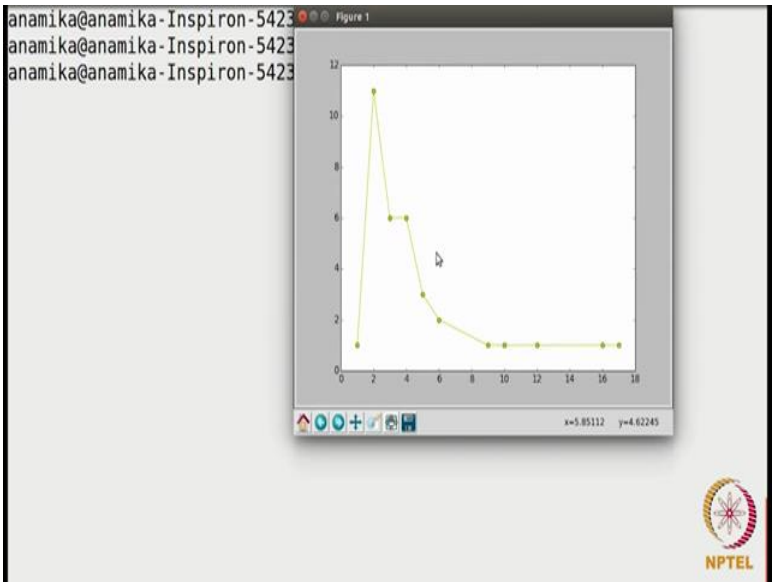
particular part and you can just go back as well and this is a feature that you can use here basically you can just increase or decrease the x and y margins. So, this is up to you whatever way you want it, and you can always reset it is well. So, let me close it. So, this is basically up to you how you can how you want to visualize it. Let be close it there was no x and y axis as you see. So, we can do that. So, let me just quickly do that, before that you can also change the way this plot is appearing so, if I put these dots you will get the plot in the form of these dots ok.

(Refer Slide Time: 24:56)



So, you get yellow dots here and you can also put line here and dots is well. So, we will get both the things.

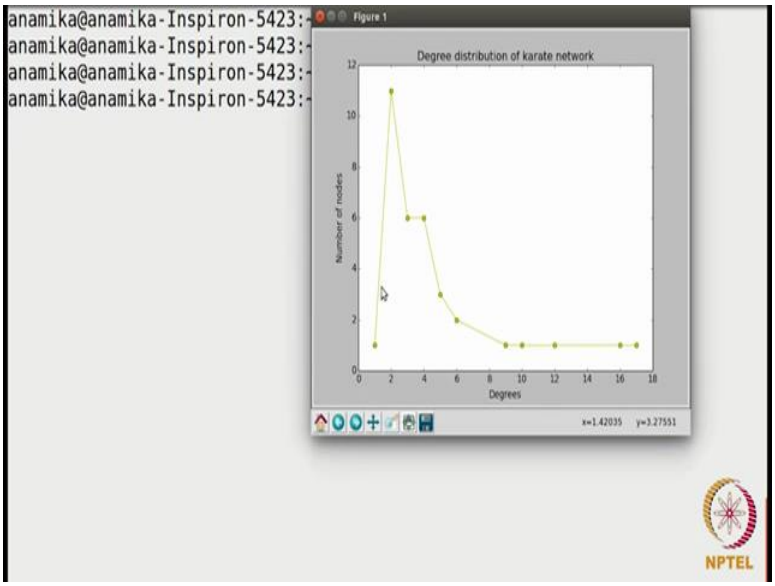
(Refer Slide Time: 25:06)



So, this is plot that you getting; one thing to observe here is that most of the real world networks exhibit power law degree distribution, which means that there are very few nodes which have very high degree and there are lot of nodes which have very less degree. So, the same is being followed in this small network is well, apart from one exception of this node. So, that might happen in some cases, but in general real-world networks have this power law degree distribution ok.

So, let us go back and add a few more things may be let us add the x label. So, let us add degrees here and then we can add the y label as well may be number of nodes and maybe we can add the title as well and degree distribution of karate network.

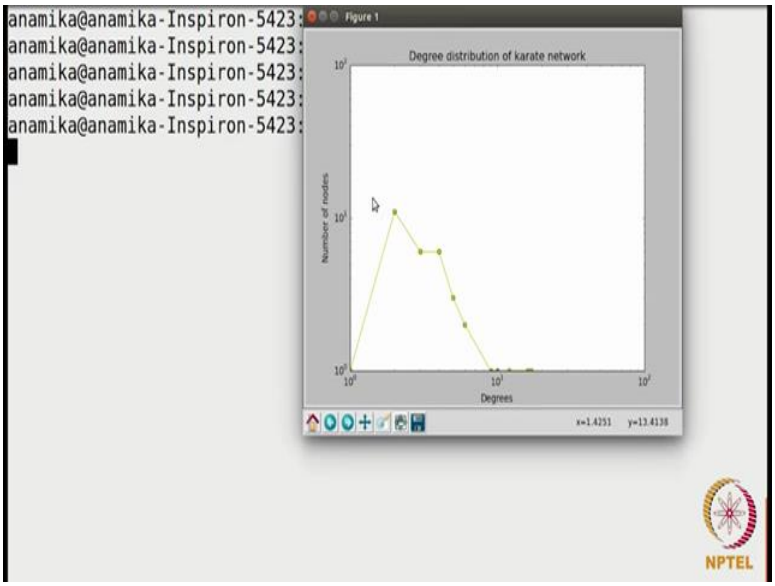
(Refer Slide Time: 26:17)



So, let us sum this you get the title and the x and y labels as well. So, you can just use more features more functions and decorate this plot the way you want. There is one more thing that usually is done in case of power law degree distribution, we can also check the log log plot of that.

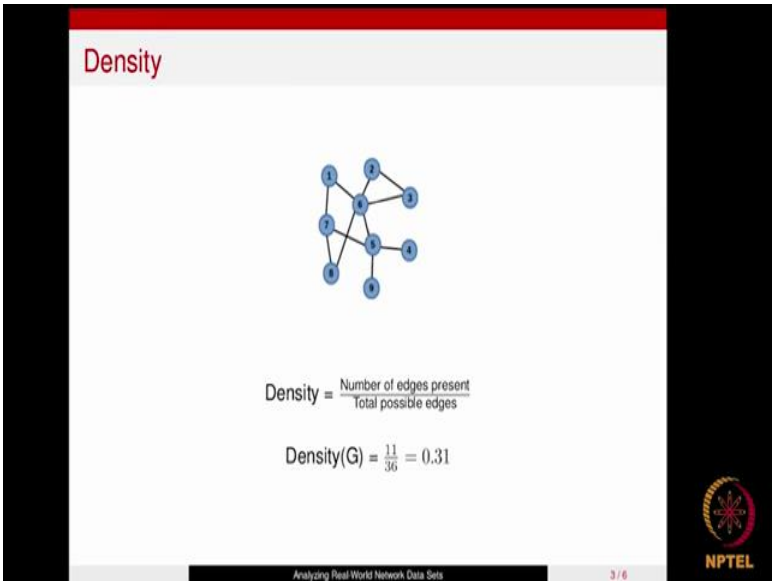
So, how can we do that we can just replace this plot by log log. So, in that case it will give as a log log plot which basically means in take the log of x axis it take the log of y axis, and if network is following complete power log the log log plot should be in a straight line. So, let us see what kind of plot we get in this case.

(Refer Slide Time: 27:01)



So, here firstly, we had section here. Secondly, it was not perfect power law. So, we have this kind of line which is not exactly straight. So, this is the kind of log log plot that we are getting in this case. So, this one about the degree distribution, now I am closing it let us go back here and see some more properties that we can analyze on this network we have done with degree distribution let us go ahead. So, next thing that we can check is density.

(Refer Slide Time: 27:27)



Density value of graph basically tells us whether it is a parts graph or it is a dense graph with respect to the number of edges present. So, if there are n nodes in a network, the total possible edges that that network can have will be ${}^{n}C_2$. Out of these ${}^{n}C_2$ edges how many what is the fraction of the edges that are present in the graph is basically what is stored by the density value.

So, if it is a simple graph the density value between will be between 0 and 1, and if it is empty graph the density will be 0 if it is complete graph density will be 1; however, if it is a multi graph where more than one edges are allowed between 2 nodes, in that case density value will be more than one can be more 1 for example, in this diagram you see simple graph with 9 nodes. So, the total possible edges that can be there in this network will be 9 choose to which is 9 into 8 divided by 2 that is 636. Now the number of edges that are present in this graph are 11. So, the density is going to be 11 divided by 36 and that is equal to 0.31. So, the graph is not very dense. So, this is the kind of indication that the density value gives us.

(Refer Slide Time: 29:06)

```
In [9]: G = nx.complete_graph(100)

In [10]: nx.density(G)
Out[10]: 1.0

In [11]: H = nx.Graph()

In [12]: H.add_nodes_from([1,2,3,4])

In [13]: nx.density(H)
Out[13]: 0.0

In [14]:
```

So, we can go back to the console and see the density value for few networks for example, let me go here and let us create (Refer Time: 29:04). Let us create a complete graph. So, I am going to write G = nx.complete_graph of say 100 nodes. So, since it is a complete graph what should be the density value let us check nx.density. So, this is the function density, which is available in networkx, which will give us the density value;

obviously, in case of complete graph it is going to be 1. Let me now create in other graph let me repeat nx.Graph. So, I have not added an a edges into it let me add few nodes here. So, I am going to pass list. So, I am passing only 4 nodes.

Let us check the density value of this network. So, since we have not added any edge; obviously, the density value was going to be 0; and let us go back here and see what is going to be the density value for our network karate network. So, what I will write is print (Refer Time: 30:26) density is. So, n x dot density sorry and I am to pass G here let us go back (Refer Time: 30:40) density is 0.139.

(Refer Slide Time: 30:40)



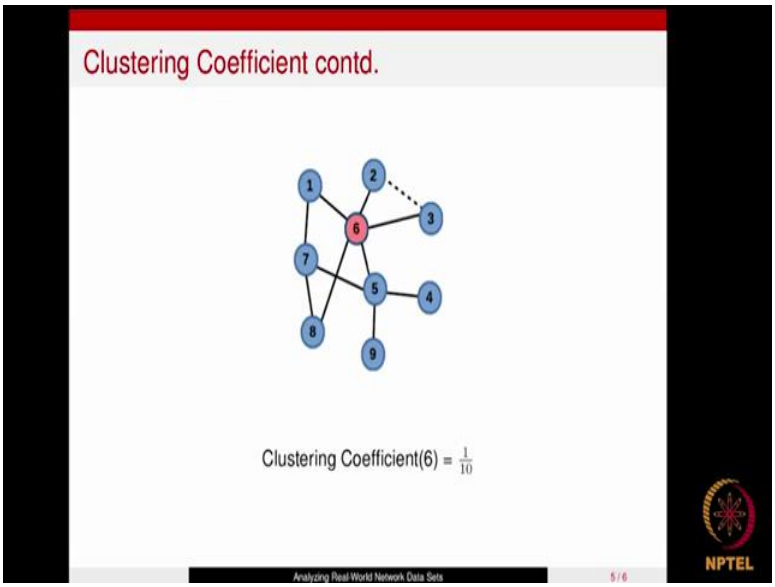
So, basically it is sort of parts graph, so that we can check. So, that is about the density, let us go back and see few more things that we can check on these networks.

(Refer Slide Time: 30:55)



Next you can see clustering coefficient. So, for a given node clustering coefficient basically tells us the number of lengths that are present amongst the neighbors of this node with respect to the total number of lengths that can be possible.

(Refer Slide Time: 31:15)

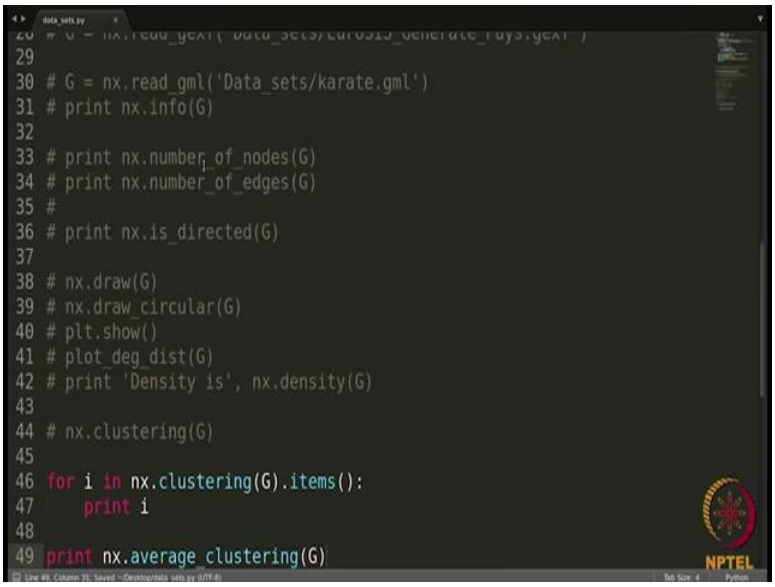


I will show you using example let us take this network let us try to find out the clustering coefficient for these nodes 6. So, as you can see this node as 5 neighbors write these are 1 2 3 5 and 8. So, there are 5 neighbors. So, what we have to check here is the number of

links that are present amongst these neighbors, that is the number of links amongst 1, 2, 3, 5 and 8.

As you can see, we see only one link that is there between 2 and 3; there is no other link present amongst these neighbors. So, on the numerator we will put 1 and, on the denominator, we will put the total possible links that that can be there amongst these 5 nodes. So, amongst 5 nodes total possible links can be $^{5}C_{2}$ which is 5 * 4 divided by 2 that that is 10. So, in this case the clustering coefficient of this nodes 6 will be 1/10. So, in case of friendship network, this clustering coefficient basically tells us how closely net the friends of node are. So, we can calculate the clustering coefficient value for every node and then we can find the average as well. So, average clustering coefficient for they tell us the amount of clustering present amongst the nodes in the graph.

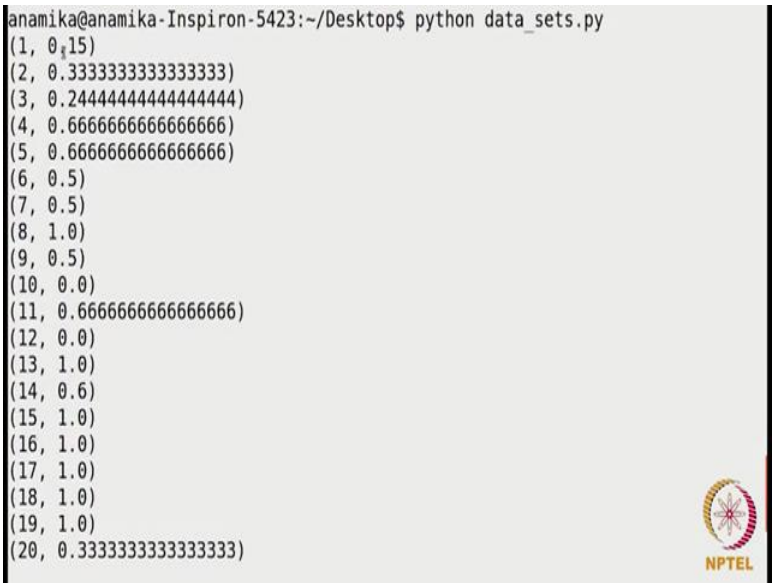
(Refer Slide Time: 32:44)



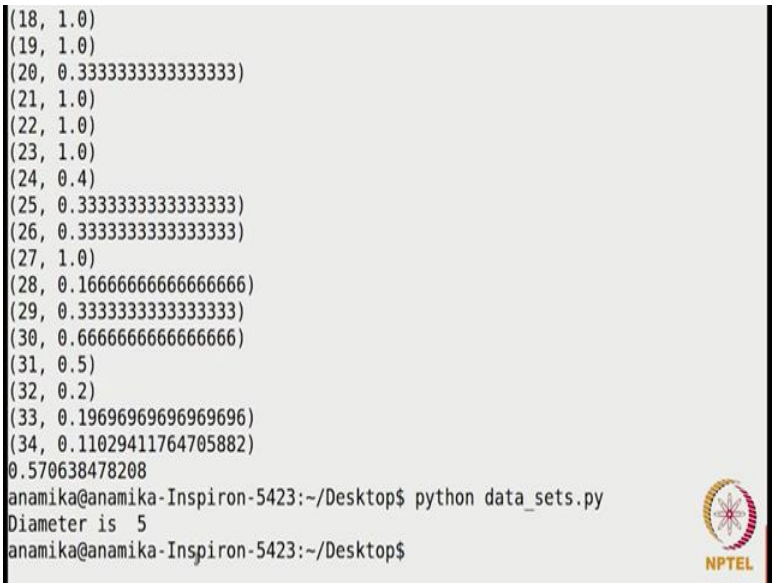
So, let us go back and try to check the same for our network. So, I am going to comment this, the function that we can use for finding the clustering is nx.clustering; however, this function basically returns a dictionary which gives the clustering coefficient value for every node. So, you can always I treat over this dictionary. So, what I will do is, for i in nx.clustering G. So, I am interested in all the items. So, I will write dot items I want to print i. So, I am going to comment this.

So, what we are doing here is this nx.clustering is returning a dictionary, which contains the clustering coefficient values for all the nodes, we are just going to print them. So, let us run this.

(Refer Slide Time: 33:38)



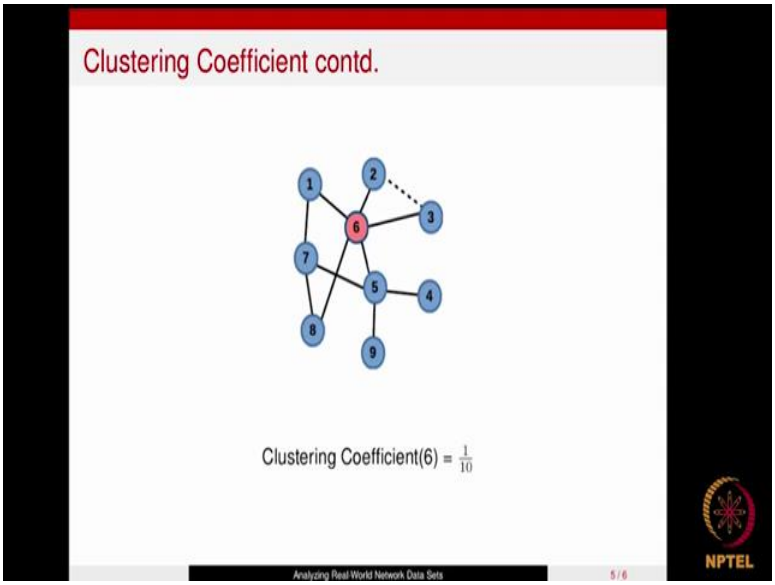
(Refer Slide Time: 33:45)



So, we are getting this dictionary, where for every node we are getting the clustering coefficient value. So, if you want the average clustering coefficient, we can either average these values or we can directly make use of another function which is n x dot average I am sorry average clustering. So, this should tell us the average clustering

present in the network. So, you see 0.57 is an average clustering. So, more this value more the clustering and more title in it the people in the friendship network r.

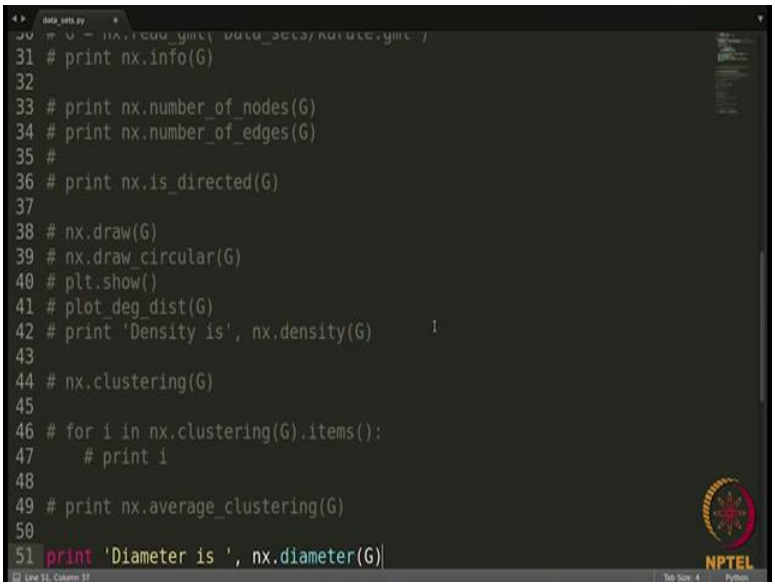
(Refer Slide Time: 34:26)



That was about the clustering coefficient let us go back and check few more properties. So, what is the diameter of a network? Diameter is basically the maximum shortest path that we have to travel to go from one node to the other for example, if you know about all pair shortest path algorithm, it basically the returns the metrics where the values are the length of the shortest path being the 2 nodes. So, is it as that for every pair of the nodes? So, whatever is a maximum value in that metrics will be the diameter of the network.

In other words, it is the shortest path between 2 most distant nodes in the network. So, for example, if you see node 1 and node 9. So, if you have to go from 1 to 9, you will have to traverse this path 1 to 6 to 5 to 9, there is no other shortest path between them. So, the length of this this path is 3, and we do not see any other shortest path which is longer than this 3, if you want to go from 1 to 4 again the length to the path is 3, if you want to go from 3 to 9 the length is 3. So, we do not see any other shortest path which is more than 3. So, that is why the diameter of this network will be 3, we can check the diameter of r network here is well.

(Refer Slide Time: 35:54)



So, I am going to comment this and let us check the diameter. So, I will write diameter is. So, I will the function that we can use is nx.diameter(G).

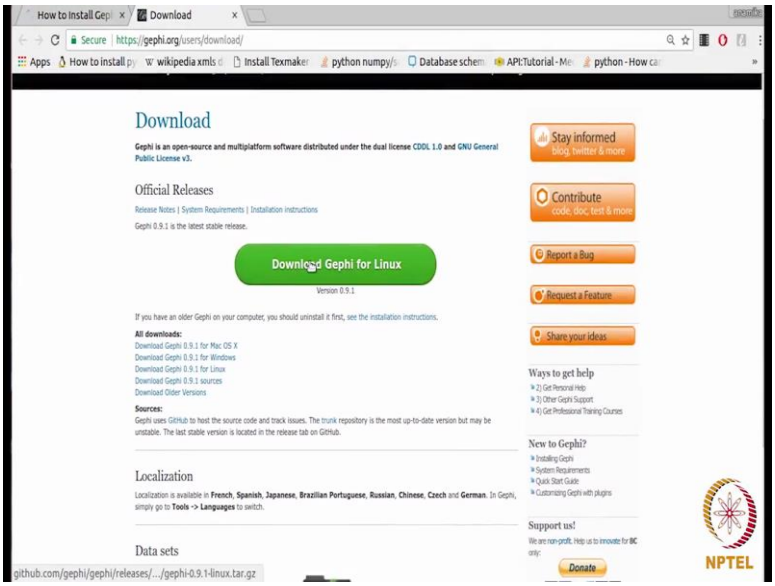
So, it should give us the diameter. So, diameter is 5. So, so there are 34 nodes here, and the diameter is 5; it is basically observe that in real world networks the diameters is basically very less because the nodes are connect to each other and that makes the distance between them very small and let us how the diameter reduces. So, these were just the few point of an analysis that we performed on the networks that we downloaded. That the main thing to notice here is that once you get the network in the networkx graph object you can just play around with it you can apply all the functions that are available, you can just read the documentation and apply the functions which are relevant for that network and you can go ahead with your analysis.

Social Networks
Prof. S. R. S. Iyengar
Department of Computer Science
Indian Institute of Technology, Ropar

Lecture - 22
Handling Real World Network Datasets
Datasets: Analyzing using Gephi

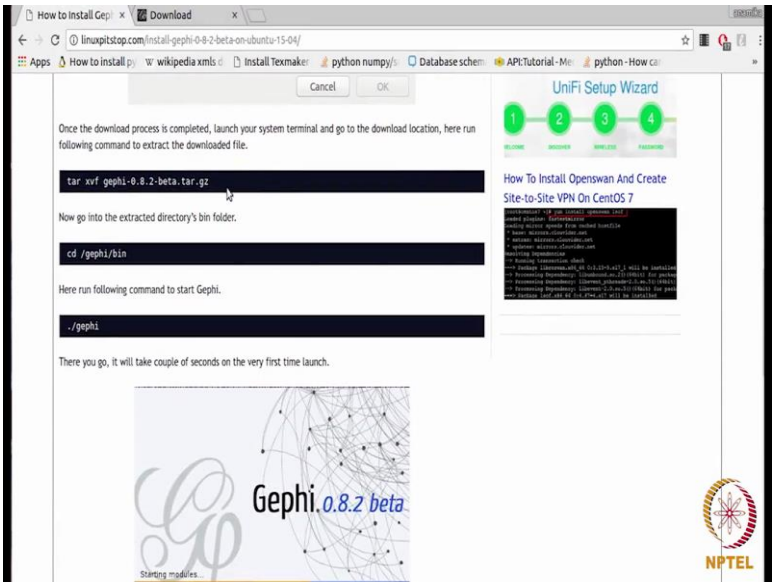
Hi everyone. In the previous video we saw how we can use networkx package on Python to analysis network data sets. In this video we are going to learn some basic features of software called Gephi which is also used for the analysis and visualization of networks. Gephi is in open source tool, which is written in Java, although networkx provides a number of functions to analysis networks. However, when it comes to visualization Gephi provides much more flexibility. So, let us get started and look at the interface of Gephi software, I all ready have a Gephi installed in my system which is a one two, but in case you want to know how we can do that I quickly show you that.

(Refer Slide Time: 00:52)



So, it is pretty simple I will just download Gephi for one two. So, we can follow the first link. So, we have to download a file I will click this link and this while that we have to download the version is 0.9.1.

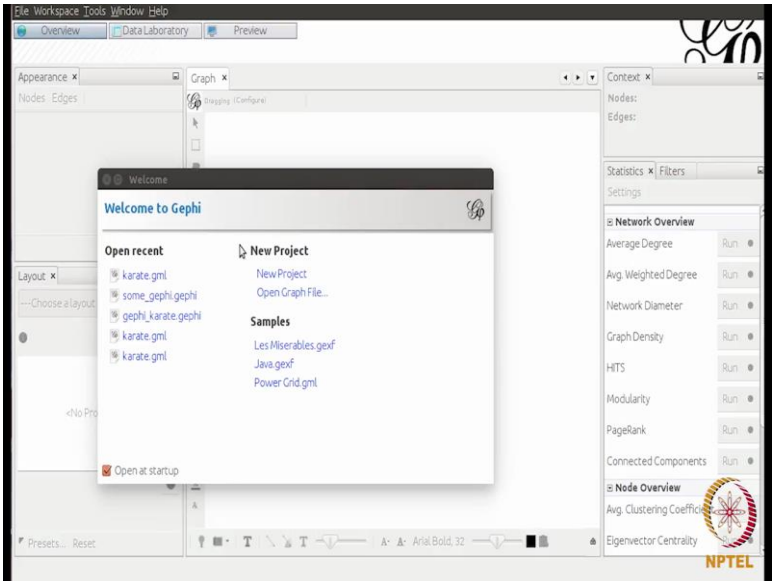
(Refer Slide Time: 01:14)



So, we are going to download this file after you download it you where ever you have saved it you will open the terminal there in that folder, and you will run this command to extract the Gephi folder out of it and after that you will get Gephi folder extracted out of the file, and you will go inside bin and after that you will run dots slash Gephi that is how you will run this software.

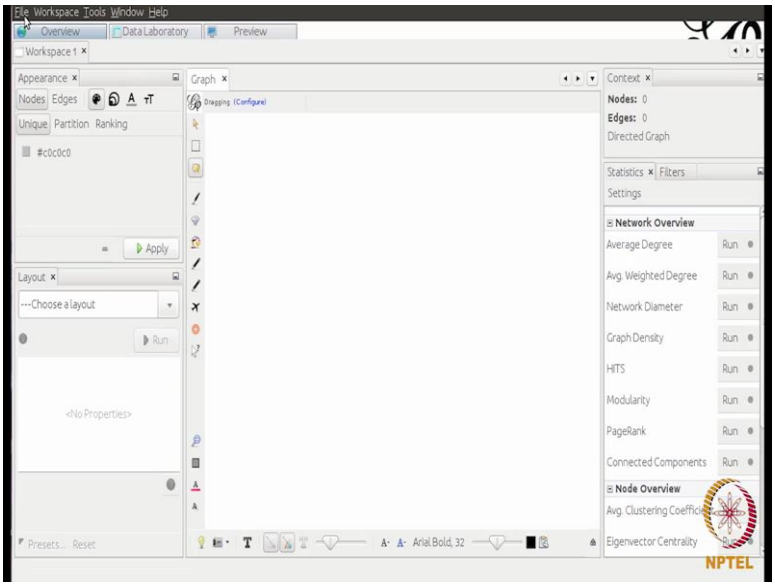
So, I already have a downloaded in my systems. So, I am going to show you how it can started; I am going to open the terminal here.

(Refer Slide Time: 01:49)



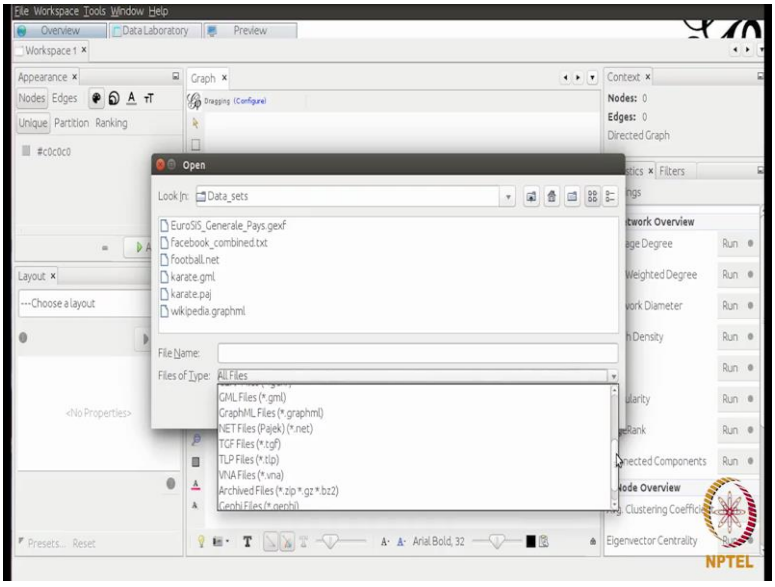
Since, I mean Gephi two to bin and then I will write .\ Gephi. So, that is how we start the software this is the interface of Gephi we are going to start a new project. So, I will click here.

(Refer Slide Time: 02:05)



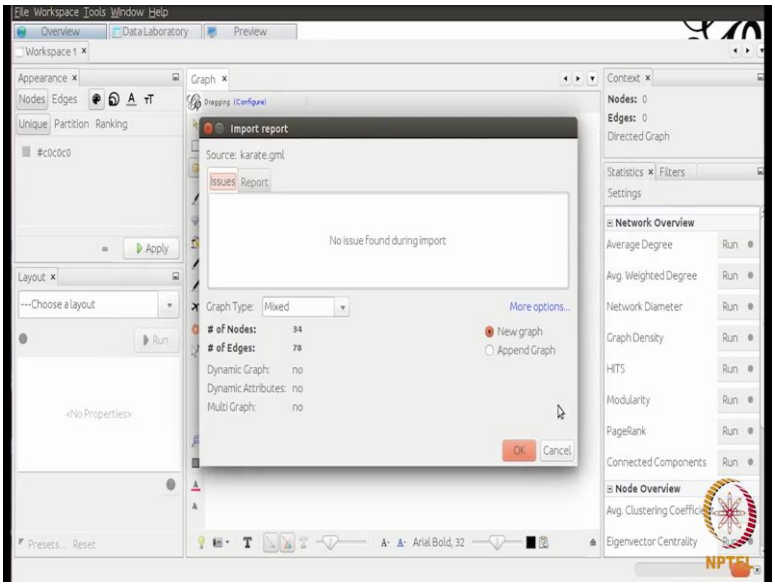
Now, this are the various options; now in order to analyze a network in Gephi we have to first open that the file of that network here. The most commonly used format by Gephi community is dot gexf. However, there are lots of other formats which are also supported in Gephi.

(Refer Slide Time: 02:29)



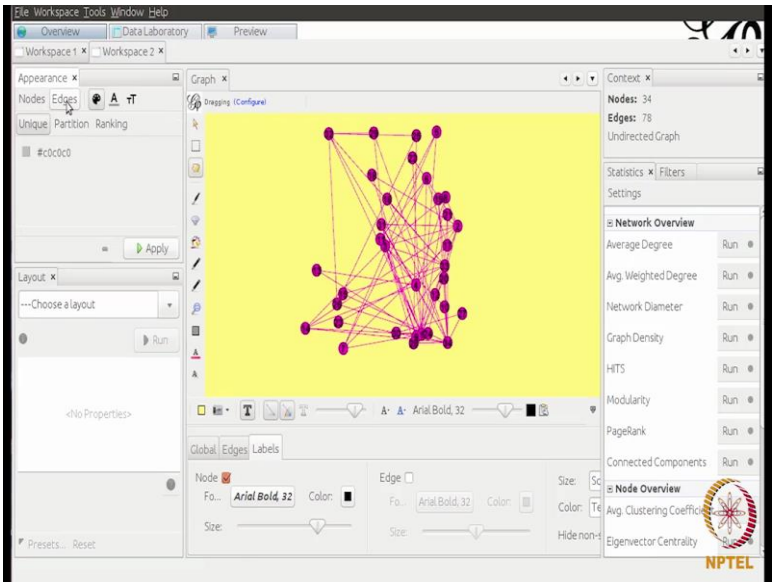
For example, let me show you when you click open here this are the formats that are supported by Gephi. As you can see there is a CSV, there is GXF there is GML GraphML L.net, Pajek files. So, number of formats are fortunately supported by Gephi; in this video we are going to take a small network in we are already downloaded this karate dot GML in the previous video. So, we are going to continue with that. So, I am selecting this file and I am opening it. So, this is what I get.

(Refer Slide Time: 03:01)



So, this is graph type which is mixed here since it is not mentioned anywhere what kind of graph it is whether it is there it is directed or undirected. Since we know that karate is undirected graph I am going to select undirected here then it showing me the number of nodes and edges and it is not a dynamic graph, and it is not a multi graph I am I want a new graph. So, I am going to click.

(Refer Slide Time: 03:30)



So, as I do that this is the graph that I get let me click here and see more options here. So, on this interface there are few options in the reference side there are few option about bottom there are few options on the right hand side, my aim is to introduce you to this software that we can use it for your analysis.

So, let me quickly start this, on the bottom you see some of the options for example you can change the background color. So, you can always change that in case you want then there is an option of zooming, let me zoom in this network. So, that I can visualize it better and then you can go to this edges tab. Here you have this pointer which you can use to in case of thickness of the edges. So, I am going to keep it like this, and then you can go to labels I am telling you the most used features out here. So, when you go to labels and if you click this node, you get the label of the nodes here and this is how you can change the size of the labels as well.

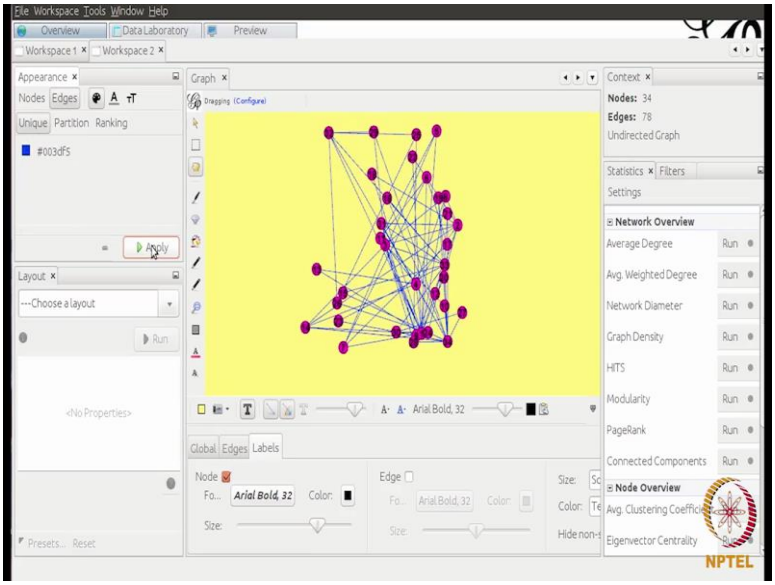
Now this is not looking so good because the size of the node is very small, we can always change that I will show you how we can do that as well and here let me show you can change you can also display the labels of the edges, in this graph there is no label sign to the edges. So, it is not going to make any difference. So, these are the main features of available of here. So, this are some short cuts that you can use for changing the thickness and this is what changing this font size of the label of the nodes. So, that is about it.

Now, let us go to the left hand side panel. So, here we have this appearance panel and under that we have nodes tab and edges tab. So, once this nodes tab is clicked we have some four options over here, the first option is for the color and second option is for the size. Since you want to change the size of the nodes so I am going to show you this first. So, right now the size is 10 let me make it 30, let us say and then I apply and this, what I see. So, labels have gone because the color of the nodes is black. So, we can change that I am going to click this color pallet.

And this is the color which is there by default we can change it let me give some color here and when I apply you see the color of the nodes as well as the edges as change by this is by default it changes the color of the edges as well, we can change that further I will show you the and this for the color of the labels let us keep it black itself as of now and this for the size of the labels.

Now, let us click the edges tab and see what options are here. So, we have the color of the edges since we have to change it, let us change it to some bluish color and apply and the color of the edges as change.

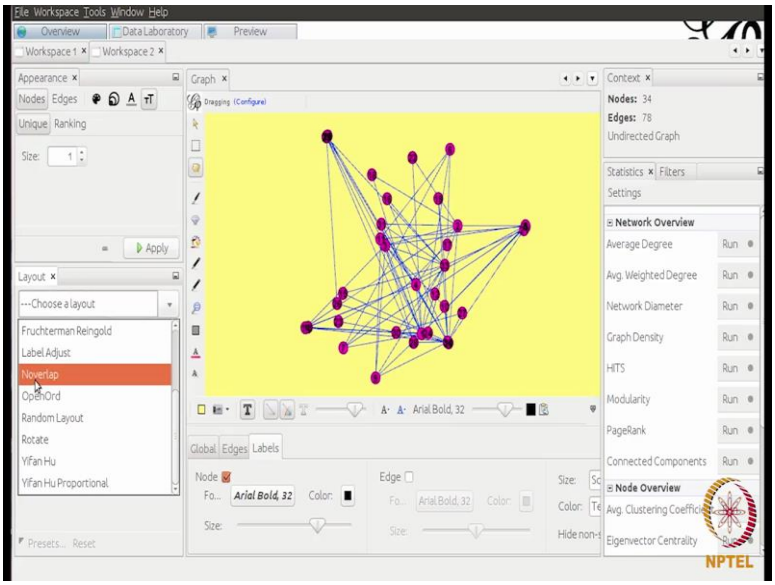
(Refer Slide Time: 06:55)



Now, this is for changing of the labels of edges since there are no labels; so would not make any difference, and this is for changing the size of the labels alright. Now in the bottom left corner out here you see an option for changing the label layouts. So, layouts basically tell us the way the node should be position and how the graph should look like.

So, there are number of layouts here and we can change it, I am you can just pair with them I am going to show you one of the layouts here which may be used full in certain scenarios. Sometimes what happens that these nodes are overlapping? So, one node is hidden behind the other so that way you cannot see them properly.

(Refer Slide Time: 07:56)

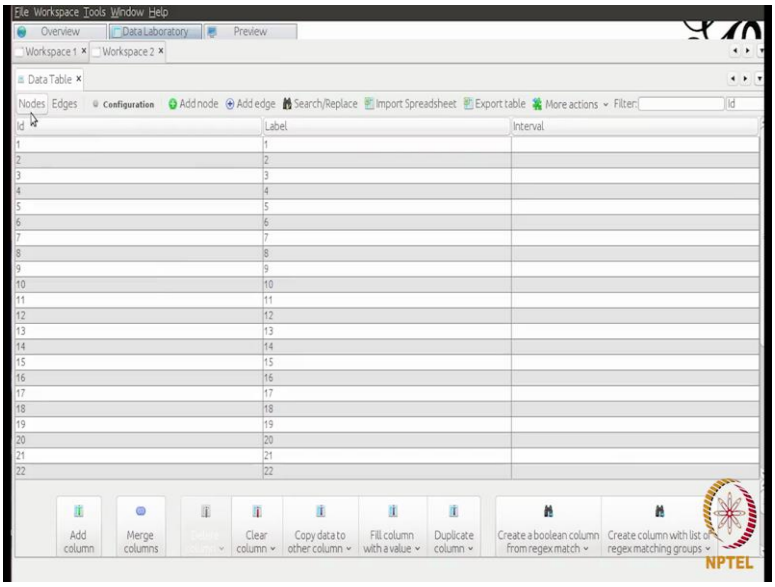


So, I am going to show you one format that can help such a scenario, before that I have to show you one of the nice features of Gephi and that is you can manually position the nodes. Now this is something that you cannot do n network x, if you have to change the positioning you have to right code for that you cannot simply do this.

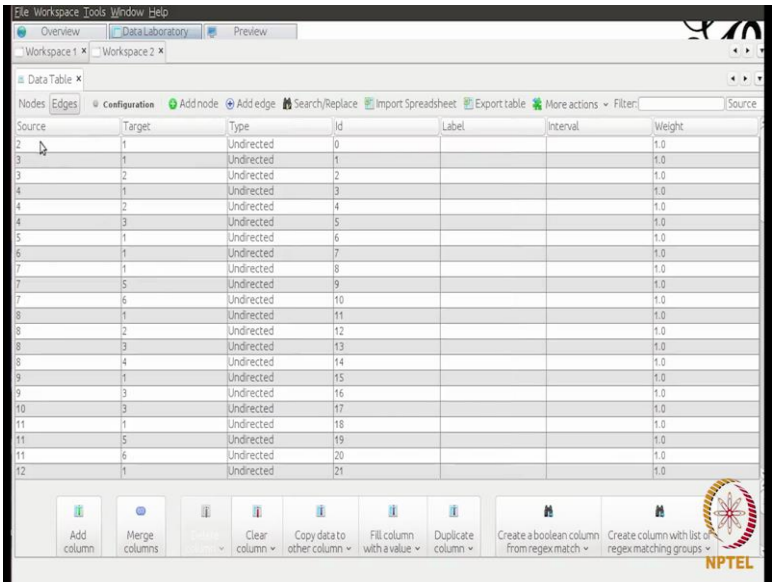
So, this is one of nice features of Gephi. Now as I was telling you sometimes nodes overlap each other I am expressively over lapping some of the nodes so that I can show you functioning of one of the layouts. So, I am just expressively putting some nodes on top of the other nodes. So, let us go here and see there is one layout that is called no overlaps. So, I am taking it and I am joining it and you see the nodes have scattered, the once which are overlapping they are no more over lapping. So, that is what is the effect of this layout. There are a number of other layouts as well so you can just read the documentation of these layouts and see what they do. Let us go to the right panel now on the top you see it just face a number of nodes number of edges and it is an undirected graph. As you go down you have a number of functions that we can apply on the given network.

Now, before we get into these functions, I want to show you two things over here. So, we are currently working in this overview tab, there are two more tab on the on the top left corner. So, one of them is data laboratory.

(Refer Slide Time: 09:39)



(Refer Slide Time: 09:47)

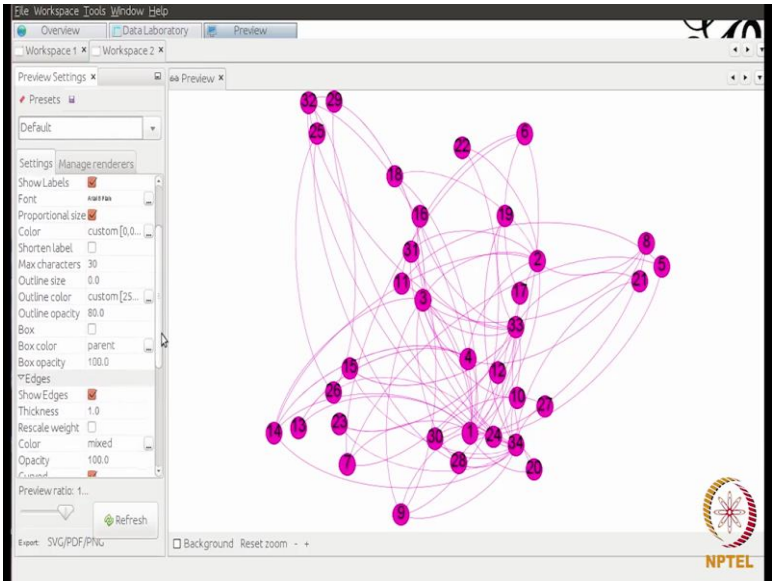


Let us see what is there. So, here you see the id of the nodes and the label of the nodes and when you click on the edges you see all the edges in terms of source and target. So, this is the basic information about the network in the form of a spread sheet. So, there are

some other features also available over here for example, you can add new nodes here you can add edges here and you can also export this table.

So, one more feature that should be noted here that when you apply some sort of operation on the network the values that will be calculated out of that operation will also be added to this spread sheet. So, once you apply lot of operations and then you want to save the details, you can just export the table and you can make use of that table in some other tool as well. So, that is nice feature.

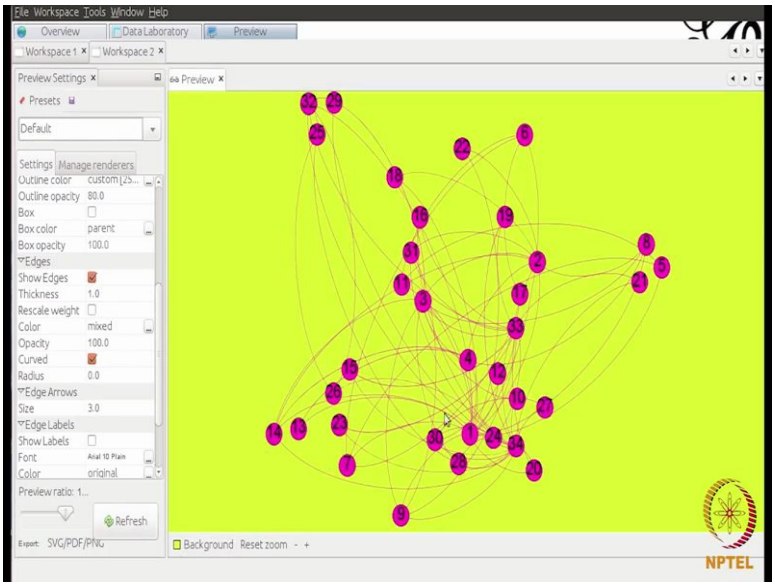
(Refer Slide Time: 10:42)



So, let us get back to the over view and let me show you the third tab is well the preview tab, when you click this and you press refresh your network appears over here with a very nice rendering, which you can download and use it the way you want. So, you can export it this formats that is SVG, PDF, PNG you can also change various properties of this network for example, if you want to show the labels you can click here and you can change the forms after clicking it you have to press refresh, now you see the labels here you can also change the font.

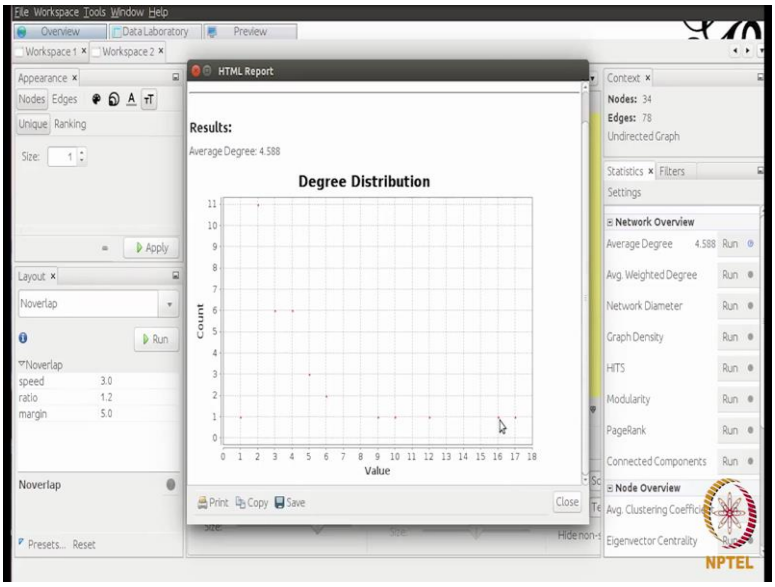
For example; let me reduce the forms to 8 because it is not looking so nice here let me press refresh and label as reduced. So, you can change it the way you want there are lot of other features as well for example, you want to show the labels or show the edges or not of course, you want to show the edges you can just play around this features and you can also change the back ground color let me just show you this features.

(Refer Slide Time: 11:48)



So, this is how you can play around here and then you can download it and you can use it. Let us go back to the overview tab and we were checking the functions that Gephi provides here. So, we have this function average degree let us click on run.

(Refer Slide Time: 12:09)

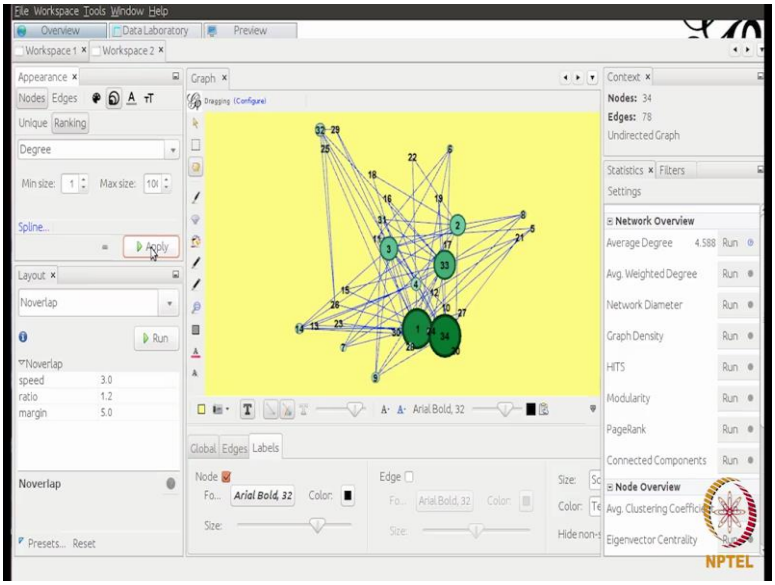


So, just like a button and the average degree and number of other things will be calculated. In this network average degree is open filed and it also given the degree distribution automatically as you can see this is sort of power loss plot as we (Refer Time: 12:26) previous video as well. Now you can save this plot you can copy it you can

print it. So, it is up to you want to use this information. Once you calculate the degree there are a couple of more features that you can see on the left-hand side, now let me show you that. So, here nodes tab is clicked, and you see one more option here that is ranking.

So, sometimes what we want; we want to change the appearance of certain nodes based on certain properties for example, we want that nodes which have high degree should appear little different be it in terms of color or be it in terms of the size of that node. So, such features are easily available in this software. So, we have this node clicked here and let me click this color here, and when I clicked the ranking here it is asking me how we want to ram the nodes.

(Refer Slide Time: 13:31)

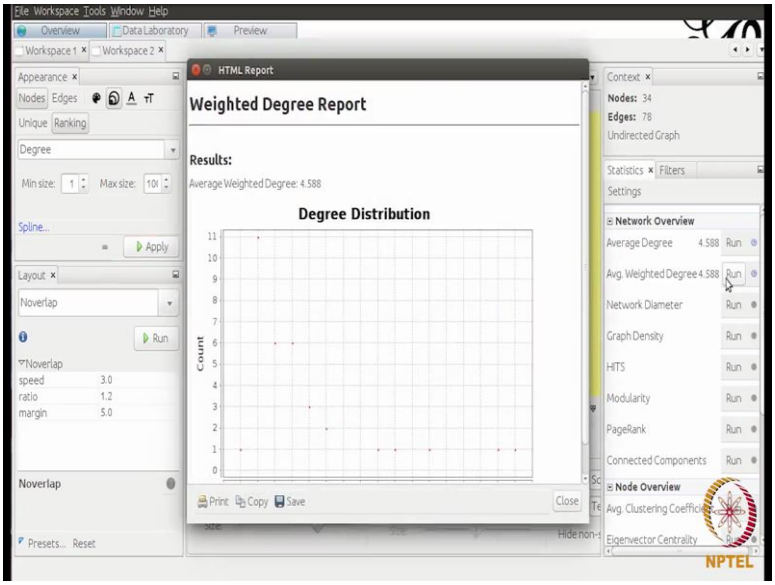


Now, I selected degree here, after this when you press apply you see the color of nodes as change the nodes which have high degree or darker in color and nodes which have less degree are lighter in color. So, you can quickly see by looking at the network you can see which are the nodes that are having high degree. So, this is one visualization feature, similarly if you want that the nodes which have high degree should appear bigger in size, you can also do that. So, I am clicking this button here and I will. So, as of now the size is unique that is 30 right.

So, we want to change it based on degree. So, I will select degree here and the minimum size is one and maximum size is 100, I will go with a head with that press apply. So, this

is what you see the size is of the nodes as change based on the degree. As you can see there are two nodes two three nodes which have very high degree compared to the other nodes, so according the size of the nodes as change. So, this is one another nice visualization features so that is about degree, let us check out more features over here we have average weighted degree.

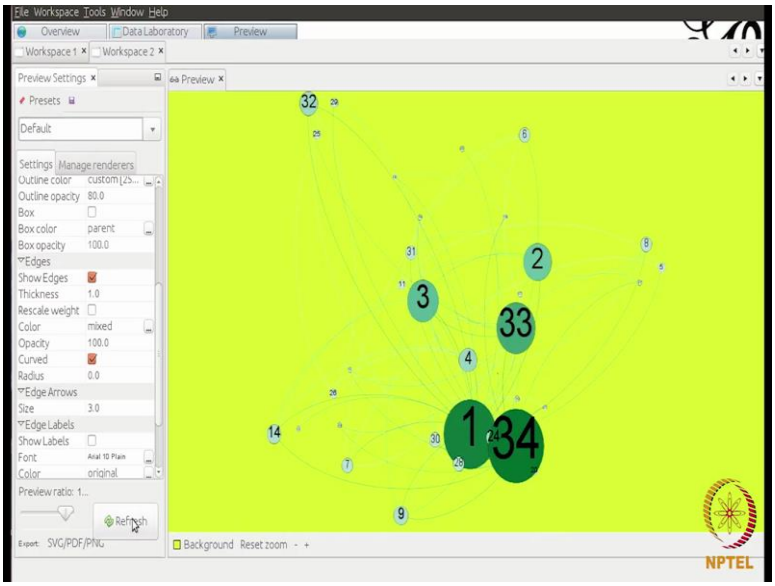
(Refer Slide Time: 14:47)



Since this graph is unweighted. So, this is now going to make difference. So, we getting same average degree here, this is because the weight is one for all the edges. We also have a number of other functions here for the example diameter we have density, where modularity, page rank, connected components clustering coefficients. So, when learn the other concepts in the throughout the course, you might want to come back here and then analyze your network based on that parameters that we have learnt. So, that will be nice because now you see what all features this software provides.

So, as when you learn about that feature that you can come back here and import here graph file and then see the features so that will be a good start for you. Let us go back here and as I want to show you one more thing in detail laboratory, since we calculated degree of the nodes that degrees as come over here. So, another column as be added over here where degrees of the nodes have been displayed. So, you can export this table and you can use it the way you want if you go the preview tab again if you press the refresh button you get that network over here.

(Refer Slide Time: 16:04)



So, this is the final rendering of the network and from the left-hand side panel you can change the various properties of this network. So, I think that was pretty much basic introduction to Gephi software and may be a good starting point for you.

Thank you.

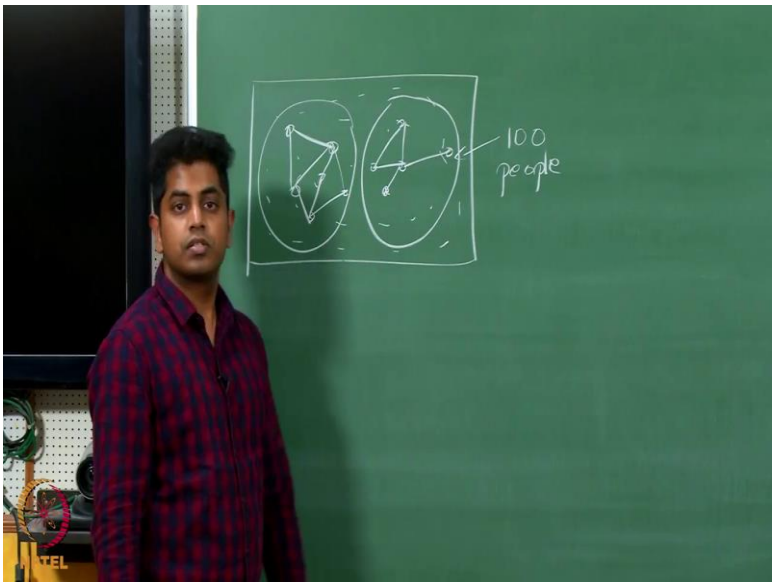
Social Networks
Prof. S. R. S. Iyengar
Department of Computer Science
Indian Institute of Technology, Ropar

Lecture – 23
Handling Real-world Network Datasets
Introduction: Emergence of Connectedness

So, we saw an introduction to datasets, importance of crunching datasets and thankfully as I said we are in era where lot of datasets are available when we can try to make sense out of it. But there are times when we may have to not worry so much about data sets to unravel some mysteries of networks in general. I am going to teach you right now in the forth coming few lectures on how one can actually keep a side data sets and look at graphs in general and make sense out of it.

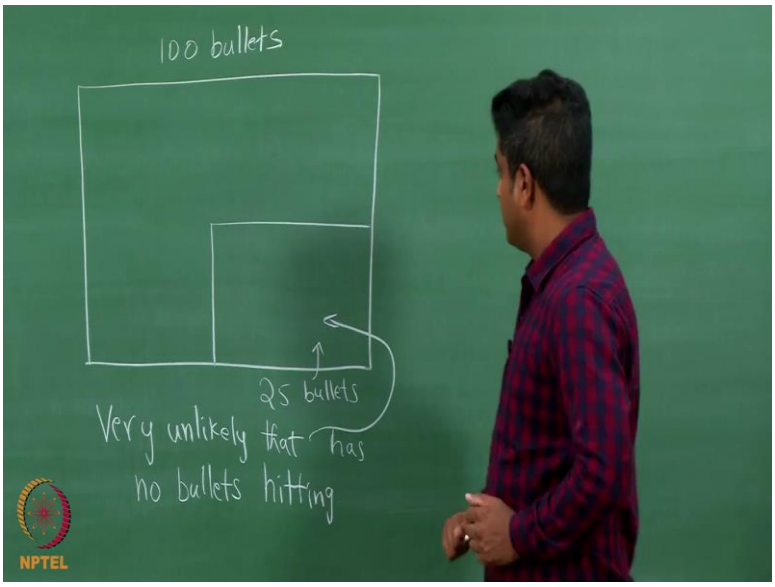
So, what will do is will revisit or age old puzzle which we have been talking from a long time of how does a graph become connected, why is it that almost always a graph is connected in the real world. Given a class room of 100 people as friendships start happening when will you observe the graph becoming connected, will it at all become connected, what if there is a bunch of people this side a bunch of people this side such that nobody knows each other across where friendships only happen within, what do I mean by this? By this I mean following.

(Refer Slide Time: 01:31)



Assume this is my class room and a class room full of some let us say 100 people, let me call this 100 people and you look at the friendships between them some random friendships like this friends with him, friends with him, friends with him etcetera. Will you see a bunch of people this side a bunch of people this side where connections are only within, there are no connections across can this even happen? Let me give you an intuition.

(Refer Slide Time: 02:23)



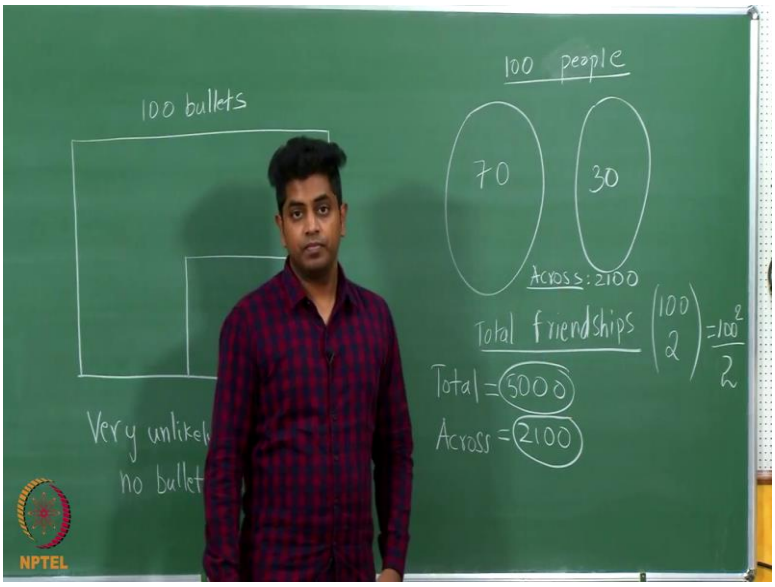
By looking at a similar example look at this assume I am giving you a shooting target by a shooting target I mean I will give you a gun and ask you to shoot within this square only within this is rectangle only you can go on shooting as many bullets as you want right, but then I will give you one quadrant of this rectangle let us say this is one-fourth of the total rectangle correct, in this one-fourth of this rectangle small rectangle your bullet will definitely fall unless you aim really well you are a great archer, you may not throw your arrows only in a particular place.

The assumption here is that you are shooting uniformly random your blind folded and simply shooting and all your bullets assuming here coming within this rectangle I will ask you how many bullets came here, let us say you shoot 100 bullets, how many of those bullets came and fell here? If you are shooting uniformly it random is not it very obvious that this captures one-fourth the total area. So, one-fourth the bullet should come and fall here right. If you are shooting 100 bullets 25 bullets should fall inside can it. So,

happen its very straight forward and intuition for all of us can it. So, happen that none of the 100 bullets fell inside this and all of them fell outside this, let me remind you, you are not shooting it with any intend you are uniformly at random shooting at it, your blind folded you cannot see where to shoot you just hold a rifle and keep shooting. It is very unlikely that no bullets come inside.

Let me write that down very unlikely that this place has no bullets at all, no bullets hitting is not this very unlikely, it is very unlikely correct. Now, if you have this intuition I am going to ask you a question on this it is exactly the same as this observation, what do I mean by that?

(Refer Slide Time: 05:12)



If I take 100 friends, 100 people they are not friends 100 people - let me say 70 people are this side, 30 people are this side totally there are 100 people, total possible friendships, total possible friendships is clearly $^{100}C_2$ as you all know $^{100}C_2$ which is roughly $100^2/2$, $100^2/2$, it is actually (100 * 99)/2 for you know from day one I have been writing it as square by 2 because 99 can be seen as 100 its almost all close to 100. So, there total number of friends which is $100^2/2$ which we all know is 5000 these are the total possible friendships out of which there are some friendships within, within there are some friendships across. Let me see how many friendships are across, across friendships are simply 70 * 30 - 2100 right. What do I mean by this? Let us take a pause and

understand what I have done. So, far total possible friendships are 5000, but the friendships across are 2100 in number.

What I am saying? Here is the situation where there was a possibility for people to make 5000 something friendships and every single friendship that they choose did not fall into this, did not fall into the across friendship, there were so many possible ways in which they could have had an across friendship they did not, all of them fell within only this is very unlikely.

The intuition is this do you see this is roughly half of this, this is roughly half of this. There are let us say twenty balls in a basket 10 black, 10 white you pick some balls from this basket all of them are black, you are picking them uniformly it random and all of them are black is this even possible, it is not possible, it is not impossible it is improper. So, I repeat if you take a bunch of 100 people and look at friendships it is very unlikely that there are no friendships across because a whole lot of possibilities for that to happen and you are saying none of this possibilities happened.

So, what did I just say? Let me connect the analogy properly when you start putting a lot of edges it is as good as you choosing the edges out of 5000 edges it choosing some edges, half the edges are between and when you are choosing none of these edges are chosen, half the edges are across between you choose not to touch even one of them and you are picking the edges look at the analogy I gave you of the basket and balls. There is a basket full of balls half of them are black half of them are white, you blind fold yourself and then pick and you see there all the balls that you are picking are all black. It is unlikely when you are blind folded you should pick uniformly it random white as to come somewhere. So, the black balls all the balls here or all the edges here, the white balls are the edges across, and the black balls are the edges within. Edges across are roughly half the total number of edges, black balls are half the total number of balls, white balls is half the total number of balls, but you when you are picking you are only picking black balls you are not picking white balls and your blind folded look this is very unlikely.

And I am just using argument to say this is very unlikely that you have two components most of them are out of 5000. Half of them are across and you are not choosing even one of them right. Same thing here one-fourth of the place is this part and all your bullets are

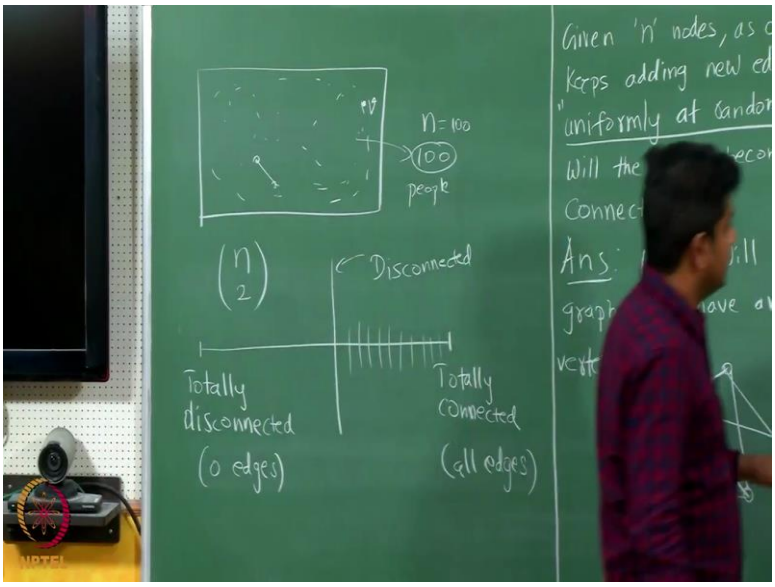
not falling here its falling out side this only that is unlikely some bullets should fall here, some balls should be white, some edge should be across that is the intuition correct. When you create a graph, you cannot see two compartments like this when you have sufficiently many edges by sufficient many edges I mean when you pick one ball it can so happen that it is black only. Second ball can be black only when you pick some 50 balls all of them being black is very unlikely. So, when you put a lot of edges all these edges being within this only is very unlikely.

Social Networks
Prof. S. R. S. Iyengar
Department of Computer Science
Indian Institute of Technology, Ropar

Lecture – 24
Handling Real-world Network Datasets
Advanced Material: Emergence of Connectedness

Let us see something nice and deep with some mathematical analysis, let us continue with our question of when exactly we see a connected graph as we keep putting edges.

(Refer Slide Time: 00:21)



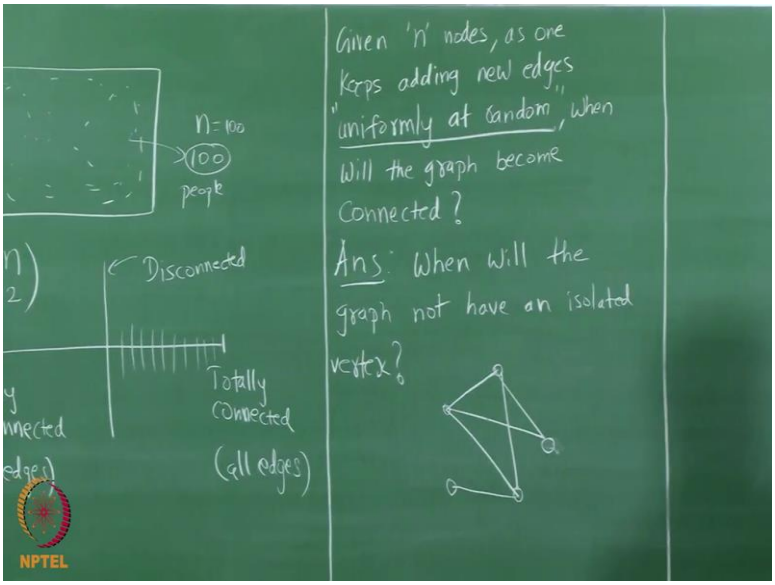
So, the question is this, the same question that we been discussing so far, assume there are 100 people or let us say n people n = 100 here, a particular example there are n people here, if you put all the edges what is all the edges? ${}^{N}C_2$ or let us say ${}^{100}C_2$, if you put all the edges; obviously the graph becomes connected.

Now, when you put all the edges and remove just 1 edge it remains connected. If you keep plugging out edges 1 at a time from this complete graph of ${}^{N}C_2$ edges, there is a time when it becomes disconnected. I repeat take 100 nodes put all possible edges between any 2 nodes, which happens to be 100 choose to and from this you start removing 1 edge at a time remove and then throw it, keep doing this you will finally, get a disconnected graph somewhere at some point. The question is the investigation of when exactly you will get disconnected graph as you keep removing edges.

So, complete graph no completely disconnected graph; complete graph means all edges completely disconnected graph means absolutely no edges, as you keep removing edges you come closer and closer to this, there is a point when the graph becomes disconnected right. So, I am going to use the word totally disconnected which stands for no edges at all and then I am going to say totally connected which means it is a complete graph. 0 edges no edge at all, all edges $^{N}C_{2}$ edges.

As you keep removing edges you will observe that there is a point when the graph becomes disconnected, what is that point should you remove half the edges or only a few edges when removed will make the graph disconnected or should you remove a lot of edges? This question is same as take no edges keep putting 1 edge uniformly at random, when exactly will the graph become connected. So, we will write this down for clarity sake, given n nodes as 1 keeps adding new edges.

(Refer Slide Time: 03:26).



I use the word uniformly at random by this I mean, I do not have any preference for the edges I keep putting edges uniformly at random randomly basically. As I keep putting adding edges uniformly at random, when will the graph become connected? Now a question like this does not have an answer, we always say mostly if you put so many edges becomes connected; whatever mean what do you mean this? If you say I need to lose weight, I need to lose 5 kilo grams of weight, you will say then you start running half an hour a day for the next 1 month, this is just guess work and you will say an

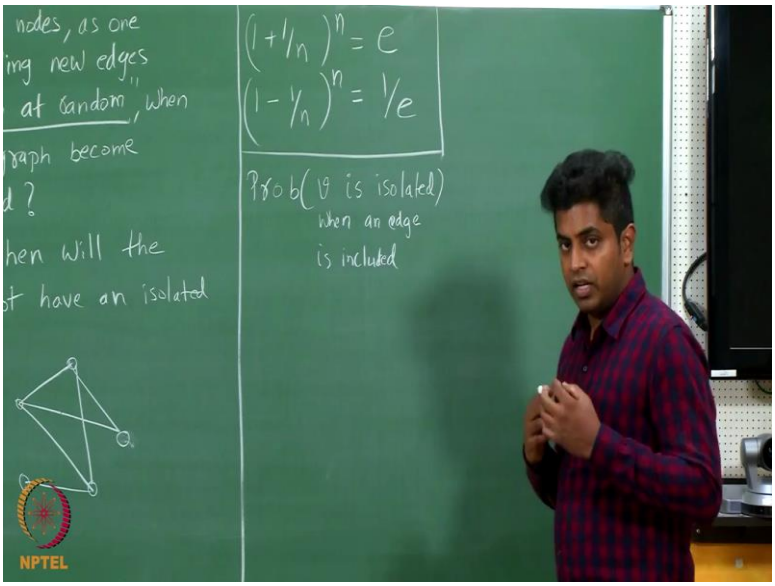
average if you run half an hour day in let us say 1 month time, you probably lose 2 to 3 kilograms of weight I am not a dietitian I am not a exercise expert, I am just telling you an example right.

Similarly, I am asking this question as you keep putting edges what do you guess? Will be the number of edges it takes for you to make the graph become connected. Let us answer this question with a small observation. I will answer this question by asking this question when will the graph not have an isolated vertex. To answer this question I am asking another question, I will ask you this question as you keep putting edges when is the time you do not have even a single point without edges, what do you mean by that?

So, let me take 1 2 3 4 5 vertices here, as you know how many possible edges can put; little bit of observation tells you 10 edges totally. Let me put 1 edge when I put 1 edge this is isolated, this is isolated, this is isolated, this was isolated does not continue to be isolated because you have put an edge. Now I will put another edge let us say this there are 2 isolated vertices now, still 2 isolated vertices only 1 isolated vertices vertex only 1, none there are no isolated vertices right now correct? After putting 1 2 3 4 5 6 edges isolated vertices came to an end, but then I should be putting this edges uniformly at random, I should not try to put edges just in order to make the graph appear connected without any isolated edges.

Please note you cannot have isolated vertex sorry; you cannot continue to have an isolated vertex by simply putting all possible edges within the remaining vertices, not touching our vertex that is all possible. But the question is uniformly at random when you start putting edges, when does the graph become connected? Let us answer this question.

(Refer Slide Time: 07:46)



So, we have all seen this result which says 1 plus 1/n whole to the n, is as n become bigger and bigger, this quantity is between 2 and 3 this is called e we have all seen this, it is some basic calculus limits basically. As n increases $(1 + 1/n)^n$ becomes e for example, if n is 10, 1 + 1/10 which is 1.1 to the power of 10, 1.1 to the power of 10 is around 2.7 which is e value. 1 + 1/20 whole to the power 20 is roughly equal to a number between 2 and 3.

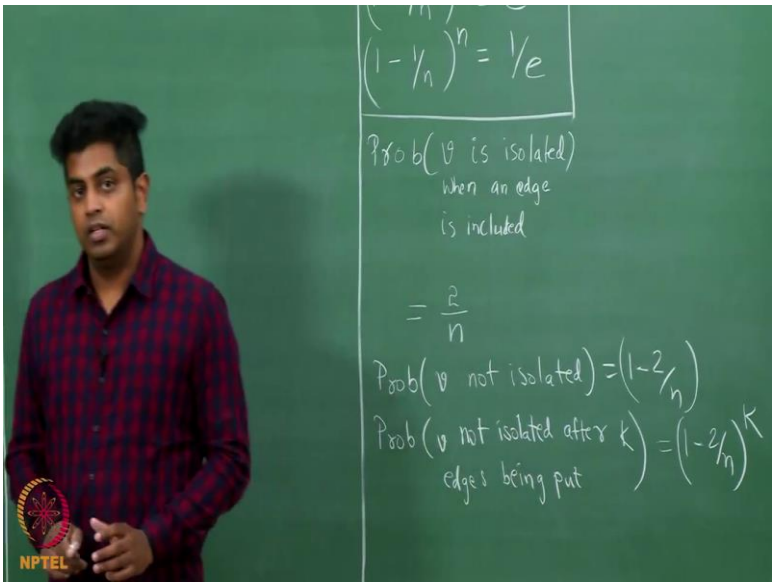
So, it is observed that this is the limit of this is a constant. Little bit of observation says we have all observed this are 11 12th standard mathematics, this is 1/e do not worry much if you we recollect this things or I am trying to say is $(1 + 1/n)^n$ as n becomes bigger and bigger becomes a quantity between 2 and 3 which is e, $(1 - 1/n)^n$ let us say n is 10 then this becomes 1 – 1/10 which is 0.9, 0.9 to the power of 10 is 1 over 2.7. As n is very bigger it becomes very accurate now we are going to use only this 2 things ok.

What is the probability let us get back to the question, what is the probability of you having a vertex v becoming isolated? When an edge is included there are n nodes you put 1 edge and v a vertex v is isolated, what is the probability what you I mean by this? Look at this, here are 100 nodes this is my vertex v, and we put 1 edge what is a probability that v continues to be isolated when you put 1 edge. In this big a graph when you put an edge where there no edges are put an edge, it is very unlikely that this edge

will fall on v, it will be some alpha and beta some 2 vertices it may not be b v, what is the probability that is v?

There are 100 people v is one of them, you choose 2 people to put an edge, what is a probability that it is v? It is 2/100 simple observation, it may be pause and thing for a minute I am going to I am assuming that you know that is 2/100. So, when I take when I put an edge and that edge not being on v on a graph with n vertices is 2/n.

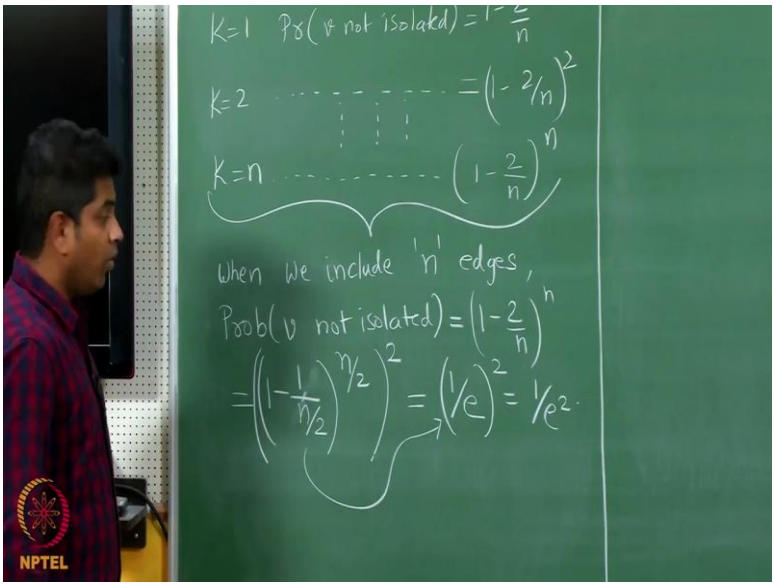
(Refer Slide Time: 11:01).



As I keep doing this you know the probability of me the probability of v having rain today if it let us say 1 by 4, the probability of not raining today is 3 by 4 correct. So, let me say probability of v not being isolated is 1 – 2/n, when you put 1 edge. You see probability of it raining today if its 1 by 4, probability of it not raining today is 3 by 4. The probably the probability of not raining today and tomorrow is 3 by 4 into 3 by 4, the probability of not seeing any kind of rain happening today tomorrow day after is simply product of 3 by 4, 3 by 4, 3 by 4 again straight forward concept of probability right.

So, probability of v not being isolated after K edges being put is, 1 – 2/n, times 1 – 2/n, times 1 – 2/n so on K times right so on K times. So, we saw that the probability is 1 – 2/n whole to the K, the power probability of v not being isolated after K edges being put.

(Refer Slide Time: 13:06)

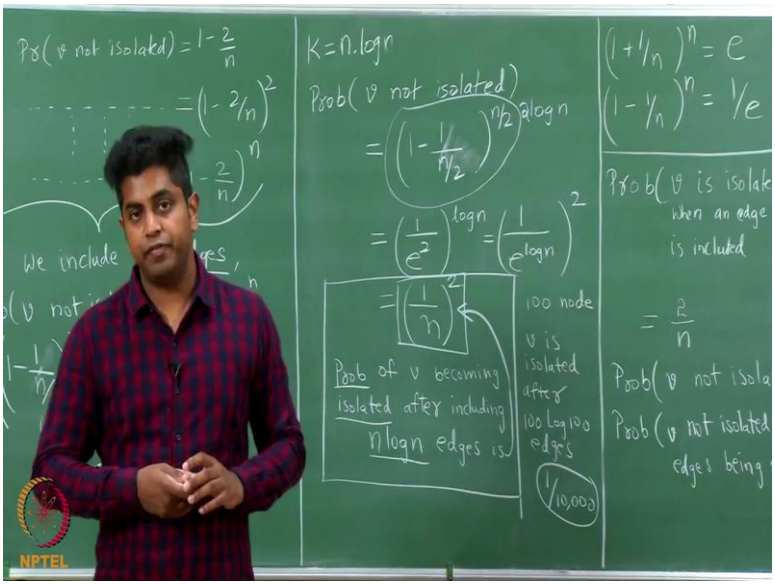


Now, let me ask this question when K = 1, the probability of v not being isolated is $(1 – 2/n)^K$, K is 1 correct. If K = 2 this will be $(1 – 2/n)^2$. If K is 3 it will be $(1 – 2/n)^3$, if let us say K = n then this is going to be $(1 – 2/n)^n$, observe this let me observe this what am I saying. When v include n edges which means K = n, the probability of v not being isolated is $(1 – 2/n)^n$, which is $((1 – 1/(n/2))^{n/2})^2$, 2 goes down and becomes n/2, whole to the power of n I will do a small find tuning here.

I divide this by 2 and multiply it by 2, usual trick we do in mathematics. If something does not look nice you make it appear nice by multiply something on both sides multiplying and dividing by the same number adding something on LHS and RHS we know those tricks. So, I am writing it. So, that this appears like sometime that we know already, in place of n we have n/2 1 minus some 1 over something to the power of the same thing is 1 by e. So, this is 1/e which means is equal to 1/e this e is 1/e, $1/e^2$, what I mean by this? E is a number between of 2 and 3.

So, let me say 2 e is 2, if e is 2 then the probability of e is not being isolated is 1 by 4; is a quarter probability that pause and think what I am saying there is a quarter probability 1 over let us say 2^2 or 2.7^2 actually whatever a constant, there is roughly quarter probability that when you put n edges on n vertices you put n edges, there is some decent probability 1 by 4, but your v is not isolated it is a very small probability, but then I will do a small trick, I will take K = n * log n ok.

(Refer Slide Time: 16:21)



Do not ask me why I am taking n equal to log n, just for fun you can take whatever you want alright there is a reason I am taking this way, but for the time being assume I am taking slightly more than n which is n log n, if K = n log n the probability of v not being isolated happens to be.

Let me rewrite this 1 – 1/(n/2) this thing, instead of n, I put n log n here, n log n here, n log n here and here this becomes same thing I repeat 1 divided by n/2 whole to the n/2 times log n, we know this part is we saw its e correct. So, there is a 2 here, 2 here this becomes $1/e^2$ * log n, which is $(1/e^{\log n})^2$ I just take a log here and put 2 that side, which is equal to 1 over we know $e^{\log n}$ is $(1/n)^2$. Let me take a minute and stare at this and observe what just happens. The probability of a vertex v becoming isolated after including n log n edges is this. I repeat the probability for vertex v becoming isolated after including n log n edges is $1/n^2$; which means if you take a graph with 100 vertices and if you only put 100 log 100 edges in that graph, then you will find a vertex v isolated with 1 over 100 square probability. So, by that I mean 100 node network v is isolated after putting 100 log 100 edges, the probability for this to happen is 1 over 10,000 is what my observation says.

My observation says when you take 100 nodes a vertex v is isolated with very small probability if you put 100 log 100 edges, which means when you take a graph g with n nodes you take a graph g with n nodes by just putting n log n edges, with a very high

probability you will not have any isolated vertices; not having isolated vertices does not guarantee you of a connected graph, but then not having an even isolated vertex, but the graph is disconnected is very unlikely.

I told you before correct you cannot have 2 components; you cannot even have 1 disconnected vertex because it is very small probability. So, this tells you that as you keep putting more and more edges very soon the graph becomes connected; when you put n log n edges the graph with a very high probability does not have any isolated vertices, does not have any isolated components there is no isolated vertex having an isolated component is very unlike correct. With a very high probability the graph is connected when you put just n log n edges.

So, we saw if 1 puts only n log n edges in a graph, the graph becomes connected. You see what n log n is? given a graph with let us say 1 million nodes, 1 million times log 1 million so many edges suffice to make the graph become connected. Point to note log n means number of digits in n. So, you are not multiplying n by a big factor here when I say n log n; by n log n I mean something almost as good as n. So, if you have n number of vertices by putting just a few edges as many as the vertices itself, n times log n you achieve connectedness.

Now, we saw the theoretical framework for it, do not worry much about all the proof that I gave you it is not especially important I just gave it for completeness sake. All that you need to know is the following let me go slowly, all that you need to know is this. Given n nodes you keep putting edges you keep putting n log n edges, then the graph becomes connected; how true is this statement. Whatever I wrote on the broad may not be true the mathematical statements I have derived is in no way a validation for what is going to happen if I am going to program and see it maybe or may not be, I do not know I am at least not convinced. So, let us do one thing, let us take our laptops and try to write piece of code and then check if it is true or not.

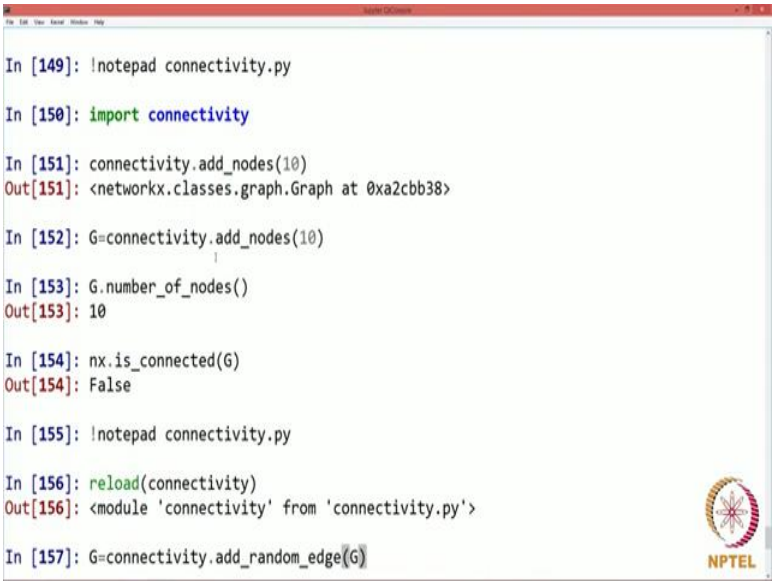
Social Networks
Prof. S. R. S. Iyengar
Department of Computer Science
Indian Institute of Technology, Ropar

Lecture – 25
Handling Real–world Network Datasets
Programming Illustration: Emergence of Connectedness

We just now looked at a very surprising and interesting result which says that on a graph if we have n nodes then one just needs n log n number of edges to make this graph connected that is if we take n nodes and we randomly put n log n edges in this graph, the graph becomes connected. So, you can start at any node in this network and from this node you can find a path to every other node in the network.

Now, we will be doing a screen cast for it we will be writing a piece of code in which we will take random graphs of different sizes and then we will look at the number of edges required to make this graph connected. So, we will do this, we will look at the plot, let us get started.

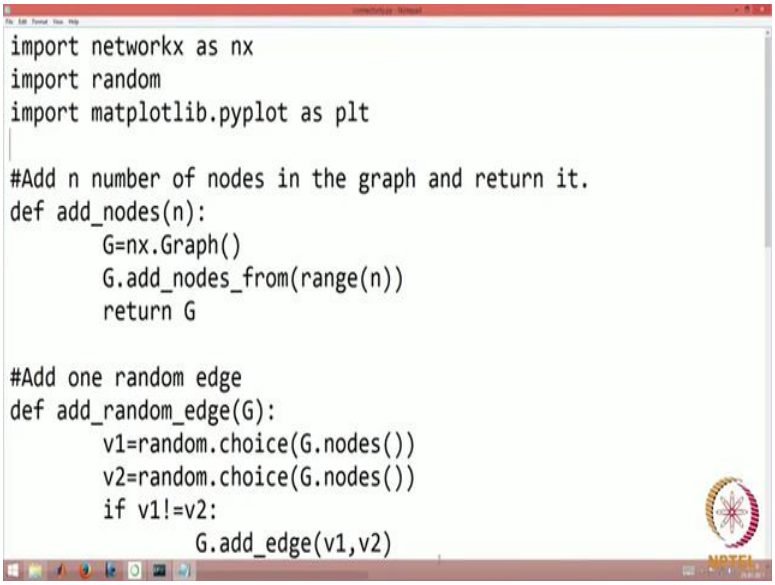
(Refer Slide Time: 00:50)



Let us start from very basic coding. So, what we want to do is we want to have some nodes in the graph and then we add random edges in this network one at a time and our aim is to see after how after addition of how many edges does this network becomes

connected. So, we first create a file for this. So, let us open a new file in notepad let us name it as connectivity.py.

(Refer Slide Time: 01:26)



So, here we have a file oh we first of all import our basic packages which we will need import networkx as nx see I think it is sufficient for the time being. So, what we do? We first define a function to add some specified number of nodes in the network. So, the function already exists in networkx you all know, but just for the sake of simplicity and clarification I will again make a function here and let us name this function as add nodes and you pass a number n as a parameter to this function. So, what this function is going to do is it will create a graph G = nx.Graph and add n number of nodes in this graph. So, we can do G dot I hope you remember the function add nodes from. So, this function adds the nodes in the graph from a list and the list we give here.

So, the list is from number 0 to number n - 1. So, n nodes get added to this graph and we finally, return the graph simple function we will just keep commenting everything side by side. So, we comment it add n number of nodes in the graph and return it perfect. So, we will save it and let us come back to our window.

So, what we need to do now is import this file we import connectivity. So, this gets imported let us call a function to call a function we have the syntax connectivity dot function name our function name is add underscore nodes and me and we had let us say we want to add 10 nodes in the graph you execute it. So, we get a graph here. So, our

function return the graph. So, let us collect the output in a graph name G. So, G = connectivity.add_nodes(10) now I want to see whether 10 nodes have been actually added in the graph or not. So, we can see that using the function G dot number of nodes you see that there are 10 nodes in this graph next, we will write a function for adding one random edge to this network.

So, if you see currently in this network there are just 10 nodes there are no edges. So, this network is disconnected completely disconnected. So, to check that we have command nx.is_connected let say you want to see whether this graph is connected or not let us call. So, before using nx.is_connected parse graph G here which shows us false if the graph is not connected now one by one, we add edges to this network. So, first we write a code for adding one random edge to this network we go back to our file which was connectivity.py and now we add second function here and the function is for add one random edge in this network and one random edge.

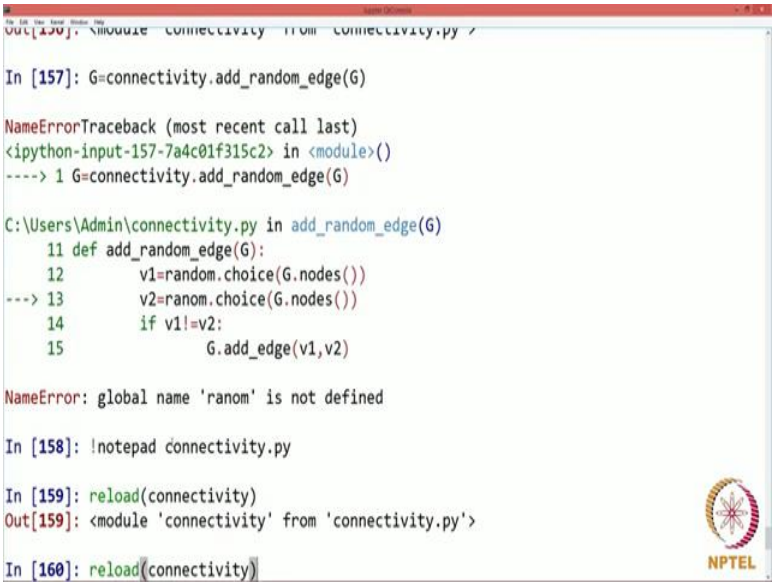
Let us name the function as add random edge and it takes us input the graph G graph G which has been already created with no edges. So, we pass that graph G actually you can pass a graph G with the edges also that is up to us. So, the function takes as input one in graph and adds one extra random edge to this graph define add underscore random edge G, now we need 2 random vertices from the network. So, how do we pick a random vertex? So, to put to pick a random entry we need one package name random. So, we have discussed about it before in the first week what does it do, we are going to use the function random dot choice first vertex v1. So, this is the first random vertex which we want to pick it is random.choice function takes as the input one list and outputs as one random value from that list.

So, the list which we are going to pass here is a G.node that is the list of nodes in the network. So, v1 gets first random node from this network let us choose another node v2 v2 equals to another random node random.choice and again we have here G.nodes we got 2 random vertices now and we want to create an edge between them, but there might be a case where v1 and v2 might be same nodes. So, we would have picked both the random nodes to be same although that is a very rare case that might happened, but we do not want that to happen because we do not want any self loops in our graph. So, we just put a condition here if v1 != v2 only then we add an edge that is these 2 vertices

should be separate. So, if v1 != v2 we go ahead and add an edge G.add_edge from v1 to v2 after adding this edge we return a graph G save it we come back to our console.

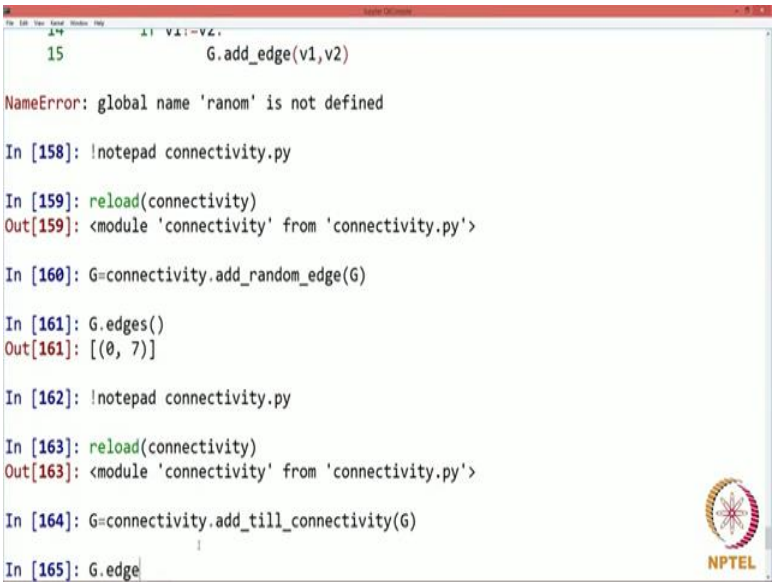
Let us now we not need to reload our file. So, we reload connectivity since we have made changes to it reload connectivity now the spelling is correct reload connectivity. So, it is reloaded we already have a graph here which has 10 nodes. So, what do we do now is G equals to connectivity dot and we call our function add random edge on the graph G itself seems to be some problem?

(Refer Slide Time: 08:07)



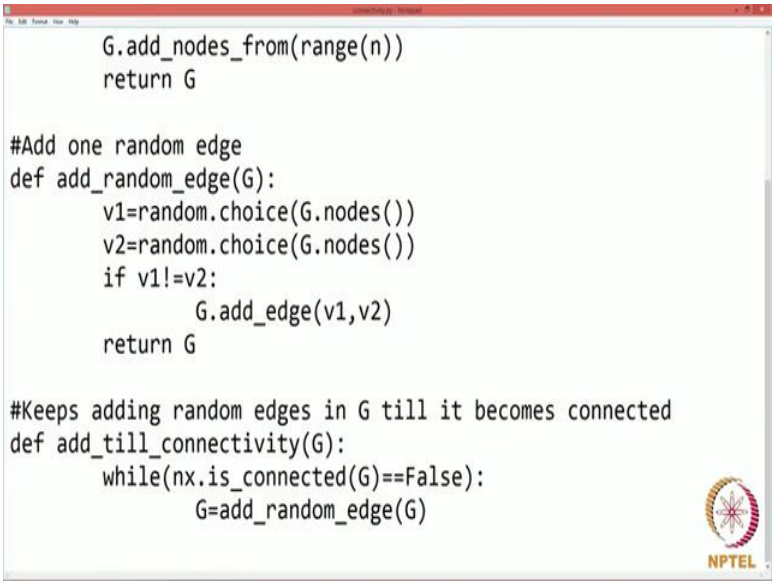
So, it is a name error which says that global name there is a spelling mistake. So, let us go back and correct it if we see here yeah just small spelling error, we correct it save it come back reload it and call the function again it works fine. So, we have we now one random edge should have been added in a graph.

(Refer Slide Time: 08:36)



So, let us see it let us look at the list of edges in a graph. So, we do G dot edges and we can see here that a random edge and edge has been added from 2 vertices 0 to 7. So, we have written the code till now for adding a specific number of vertices in the graph and adding one random edge what is our aim? Our aim is to keep adding these random edges in the network till our network becomes connected and then to look at how many edges were these which made this network connected for doing that we go back to our code.

(Refer Slide Time: 09:22)



And let us create another function here what is function does is it add random edges let rather say keep keeps adding random edges in G till it becomes connected and let us name this function as add till connectivity add till connectivity and it takes the graph G as input. So, what it is going to do till the graph does not become sorry till the graphs become connected.

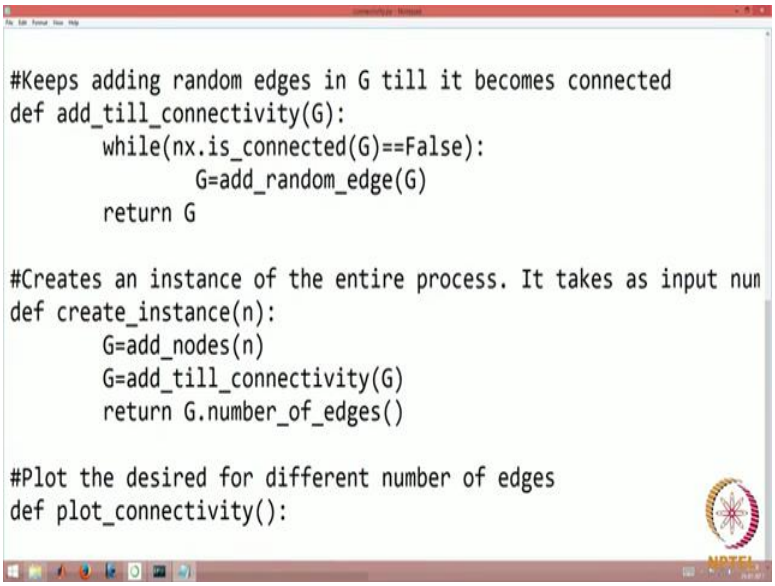
So, while nx.is_connected g. So, this is a function in networkx which returns true if the graph is connected false if it is not connected. So, while nx.is_connected(G) = false it keeps running and what does it do? It adds one random edge to the graph for graph G becomes add random edge G. So, one random edge is added to this graph and it keeps adding more random edges till the graph becomes connected and at the end, let us return the graph. So, let us look at what this code is doing again we reload connectivity and now what we are going to do is G = connectivity.add till connectivity. So, let us see how many how many edges it adds. So, we already had one edge which we have added previously, and we pass the graph G here and let us now look at G.edges.

(Refer Slide Time: 11:19)



So, we see that many edges have been added to this network to be precise the number of edges added is 16.

(Refer Slide Time: 11:36)



So, it seems like in a graph having sixteen nodes just sixteen edges are required to make this graph connected let us also quickly check whether its connected or not. So, nx.is_connected G we can say that it is we can see that it is connected it is true.

Till now what we have done is we have written a small piece of code which takes a graph as input keeps adding edges to this graph till the graph becomes connected next what we want to do is to repeat this procedure for different number of nodes. So, here for example, in this we did it for 10 nodes and we looked at that the number of edges required was 16, now want to do it for 20 look at the number want to do for 30 look at the number and so on.

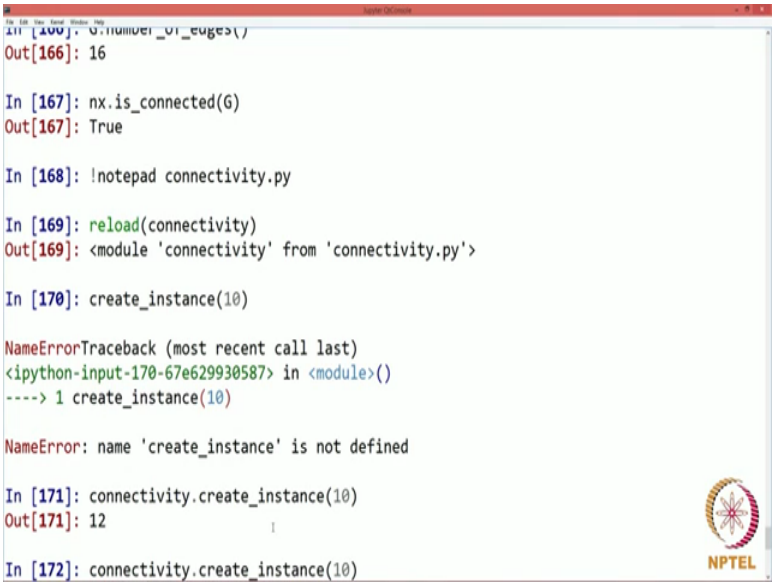
So, for doing that first of all let us create a new function here. So, this function is going to simplify what all we have written here let us name the function create creates an instance of the entire process. So, what do I mean here by instance of an entire process? It takes as input. So, the function will simply take as input number of nodes and returns the number of edges required for connectivity. So, it will make a task easier. So, we will just input a number and a number of nodes and it will output us it will tell us how many edges are required to connect a graph having these many nodes.

So, let us define this function let us name it as create instance and whatever is required inside the code for this function, we have already done that. So, define create underscore instance and let us pass a number and here what we do next is first of all we have to add

nodes. So, G = add_underscore_nodes and we add n nodes. So, we called the first function which we defined previously. So, whatever we have done till now in Ipython that only we are replicating here in the file.

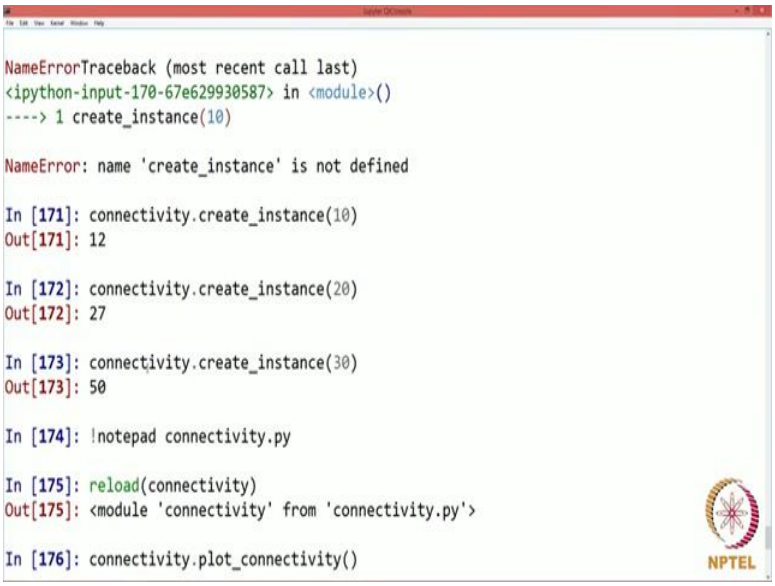
So, we first call this function add underscore nodes and which returns us the graph and next we can simply call G equals true keep adding till the graph gets connected. So, add till connectivity we again pass here G and it will return us what it will return us the number of edges return G.number_of_edges.

(Refer Slide Time: 14:50)



Let us see how it does? how is it working reload the file and let us call create underscore instance and let us pass the number 10 here. So, it is in our file connectivity.py. So, connectivity.create_instance(10). So, we see that if there are 10 nodes, we need 12 edges.

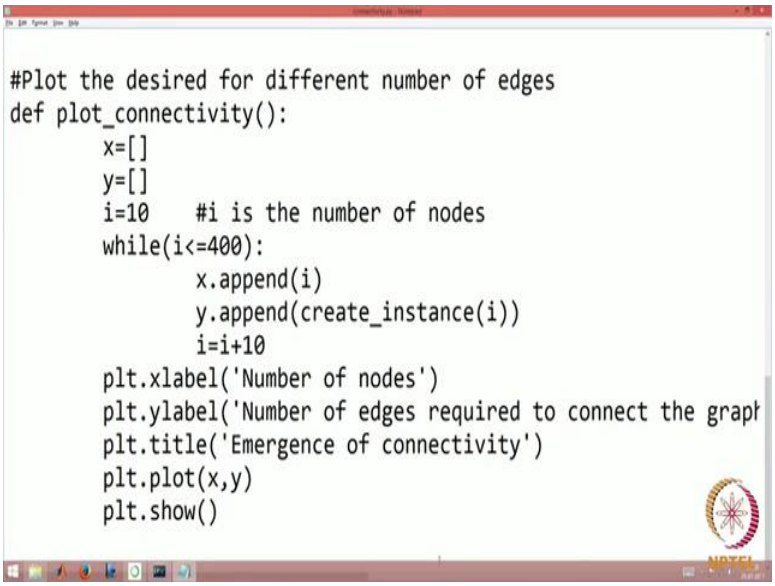
(Refer Slide Time: 15:14)



Let us make it twenty nodes we need 27 edges we have thirty nodes it is 50 edges and so on and what we are interested in I want to plot this and visualize how does this plot looks like. So, we just need to we just need to plot what we have done. So, for plotting it we go back to the file, we can define a function here. So, what is this function do plot the desire let say we know what is desired. So, desired is the number of edges required for connectivity plot the desired for different number of edges and let us define a function and let us call it plot and let us call it plot connectivity.

Let us define this function plot connectivity and we it does not require as input anything we are not taking any input for simply for different number of nodes will be plot it. So, for plotting always we need the arrays corresponding to a 2 axis; x axis and y axis.

(Refer Slide Time: 16:36)

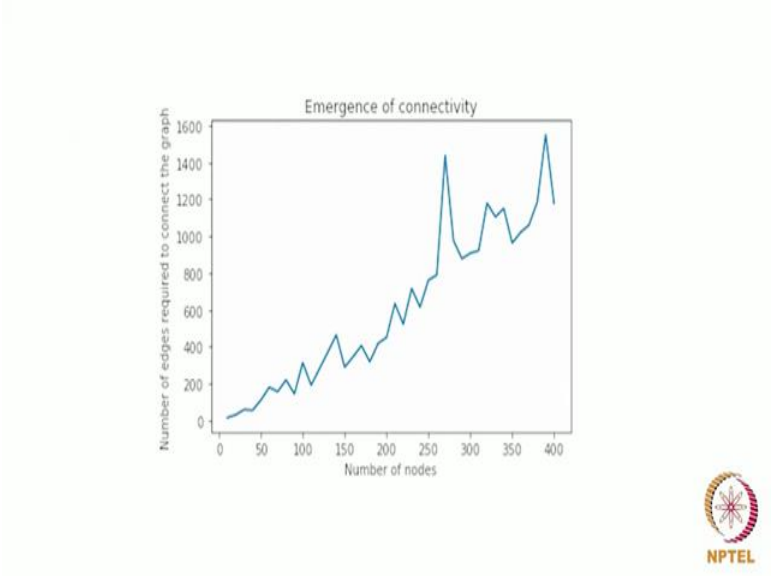


So, let us the array associated with x axis be x and the array associated with y axis be y. So, we have here x and y then we take a number i = 10. So, i what does i tells us i is the number of nodes for which we are experimenting. So, i is the number of nodes while and let us take the number of nodes till 100 or let us rather make it 400. So, while i <= 400 what is loop will do in x axis, we append the number of nodes which is nothing, but i and in y axis we can append or do we append the number of edges required to connect these many nodes which we can get from our function create instance. So, y.append create instance and we pass i here and we need let us increment i with 10 every time i = i + 10.

Now, we need to plot this for plotting this we need the package matplotlib, we import matplotlib.pyplot as plt come back. So, what do we do plt.plot what we want to plot is x against y and plt.show. So, let us just do a little bit of decoration here before plotting it let us add some labels and the title for the curve let us say plt.xlabel which is the label for the x axis is nothing, but the number of nodes plt.ylabel label for the y axis is the number of edges required to connect the graph and plt.title let us give a title to this curve and let us title it as emergence of connectivity it done.

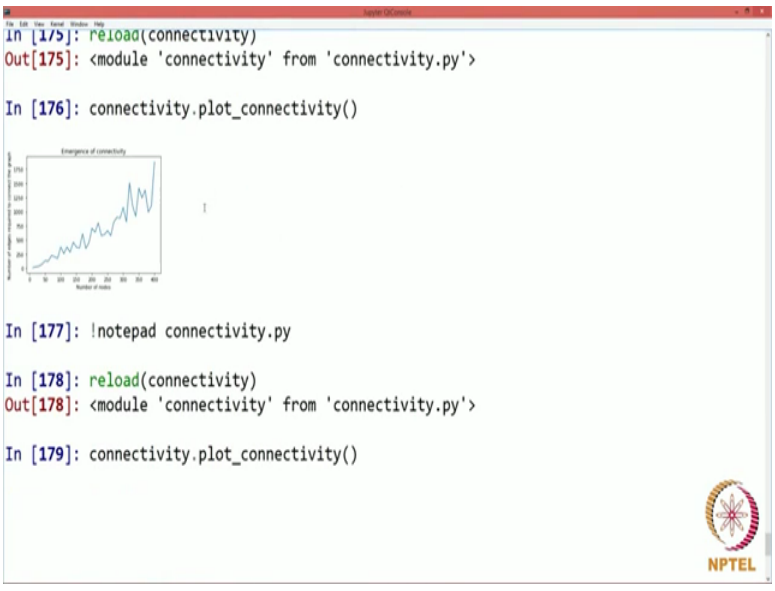
Let us look at how does this plot look like save the file we will reload it and then we call connectivity dot plot connectivity I will take some time to execute yes.

(Refer Slide Time: 19:33)



So, here we can see the curve here we have the number of nodes here we have the number of edges and we can see that as the number of nodes increases the number of edges required to connect it also increases although we see that although we see this plot where number of nodes with number of edges we see a lot of spikes in this curve.

(Refer Slide Time: 19:49)



So, why are these spike occurring it is because our experiment is a random experiment which gives a different output every time you execute it whenever you execute a random random experiment it will give you a slightly different outputs. So, one time it can throw

a value three other time 5 other time 4, but if you take the expected value it gives you a better estimate. So, to remove these spikes in the plot instead of taking one value corresponding to one instance we might want to average it out. So, let us how can we do that we go back. So, what are we doing here is corresponding to one value of I that is corresponding to one particular instance given a particular number of nodes stand here we execute this code only one time?

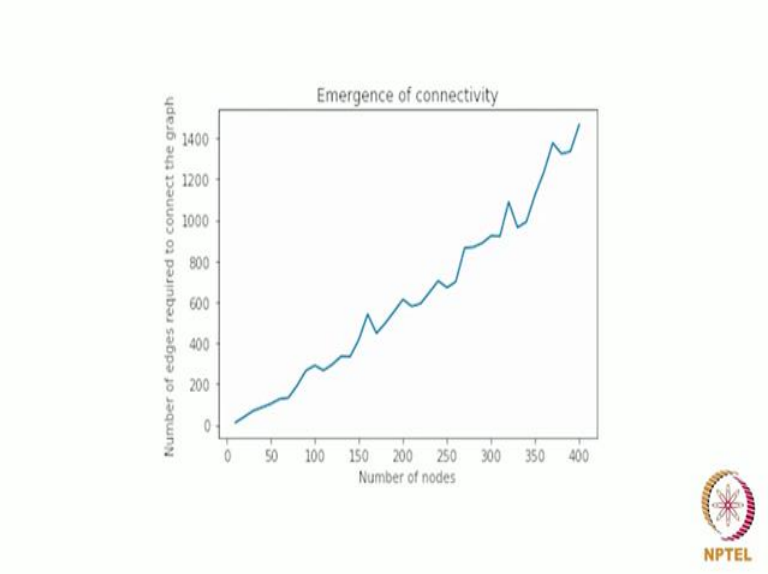
So, we take a graph having 10 nodes we will look at how many edges are required to connect it and then we return the number of edges what will give us a better result is if we do it multiple times. So, we take a graph having 10 nodes then look at the number of edges we again do this again do this do it 10 times and take an average over all those experiments that will give us a better estimate. So, let us try doing that. So, doing that is very easy. So, in this create instance. So, create underscore instances for one instance you give it one number of nodes and it gives you number of edges.

Let us average it over some particular number of instances let us average it over 4 instances. So, n see what I am going to do here is we define a new function n and let us name it create average instance n and what it does is it call the function create underscore instance 4 times and takes an average over them. So, we create a list here list one and for i in range 0 to 4. So, its repeated 4 times what does it do is list one dot append what does it append to list one is create underscore instance n. So, it appends it to list and what it returns is the average. So, it return numpy.average of list one.

So, we have yet node imported numpy. So, let us import numpy here import numpy. So, its returns numpy dot average list one. So, what we are going to do is while plotting in plotting connectivity instead of using creating underscore instance now we create we use create underscore average underscore instance. So, the value is averaged over 4 particular instances now let us look at the change in the curve it creates we reload connectivity and we again call this function.

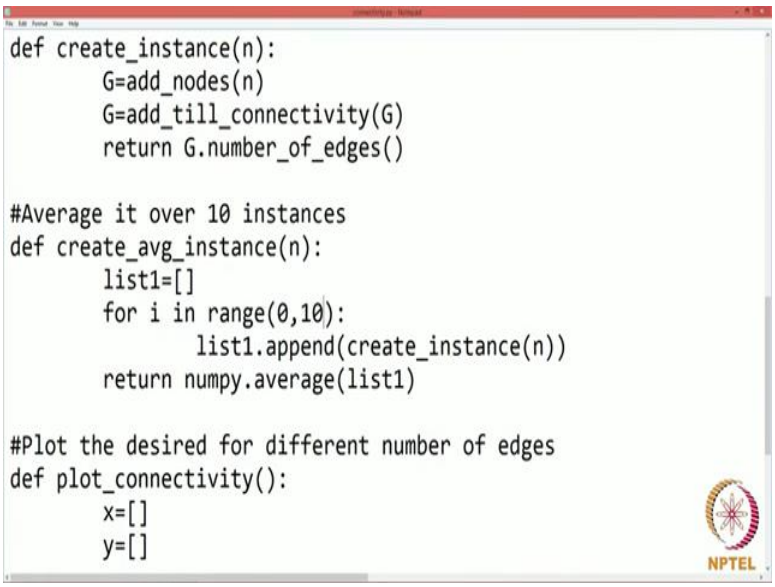
So, since it is doing it 4 times it takes some time and you can see you can say that the difference in 2 curves is visible this curve is more smoother because we have averaged out the values over 4 instances where it was only one instance.

(Refer Slide Time: 23:21)

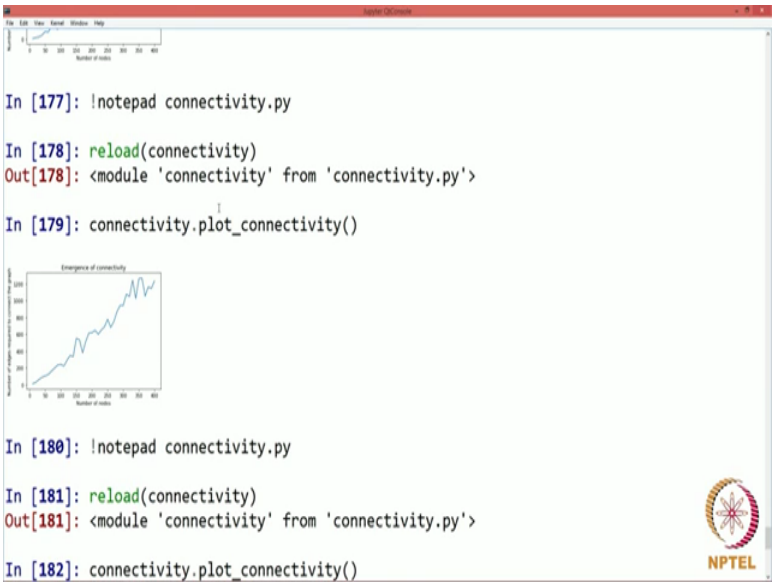


In this case let us play around with it a little bit further and let us change this 4 to let say 10 we do an average over 10 instances we make this 4 as 10.

(Refer Slide Time: 23:45)

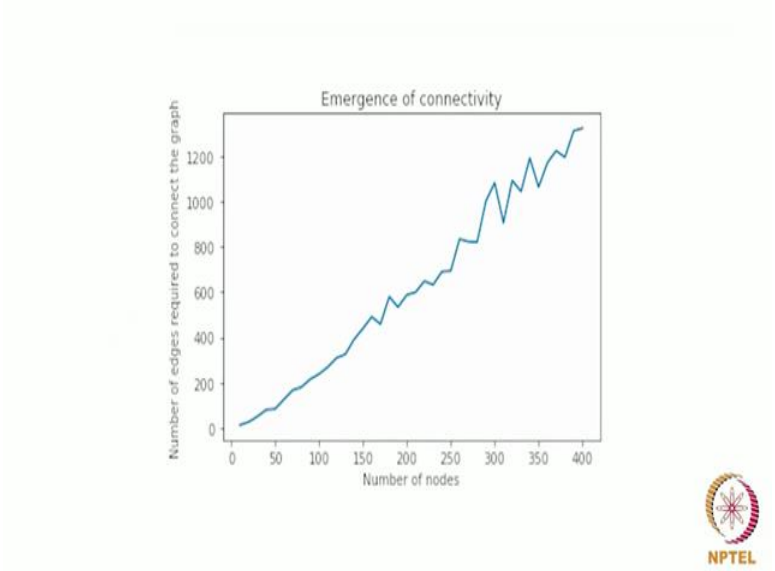


(Refer Slide Time: 23:52)



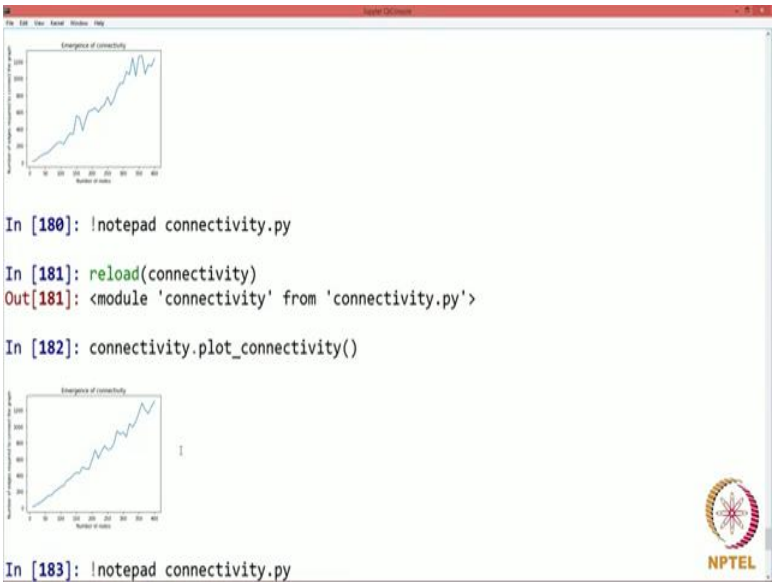
We reload it and we call the function again. So, let us wait for the output.

(Refer Slide Time: 24:01)



So, again you can see a visible change in the curve it gets all the more smooth. So, you can change this value from 10 to 20 to 3200 and the curve will become more and more smooth.

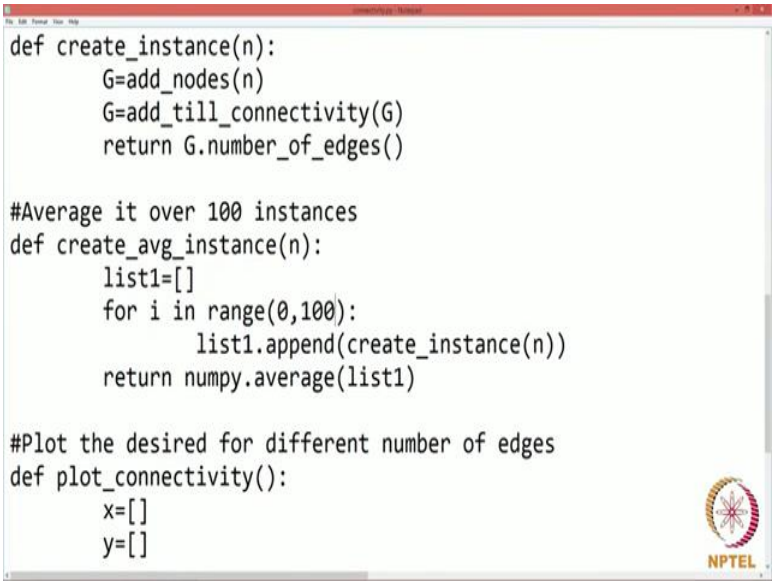
(Refer Slide Time: 24:12)



So, we can try it for higher values now the final and the biggest question what was our claim was that if there are n nodes in the network it takes just n log n number of edges to connect this network the question is are these number of edges n log n. So, if I look at this curve is it the curves of n log n oh let us write a piece of code and check that as well.

So, what I am going to do is I am going to draw this plot. So, I am going to have 2 plots in the same figure one plot is the plot which we have achieved from our code which is the actual data and let us plot it against n log n and look at what is the result.

(Refer Slide Time: 24:58)



So, I want a very smooth curve. So, for a very smooth curve what I do is I average it to hundred instances. So, I average it 400 instances and now I want to plot it against the n log n curve. So, what I do is after doing this before plotting I do exactly the same procedure, which is written above, but here I am going to plot n log n.

So, whatever I am going to do here is simply a replication of the above code with a little bit of change and you can write it as a separate function as well and I am writing it here.

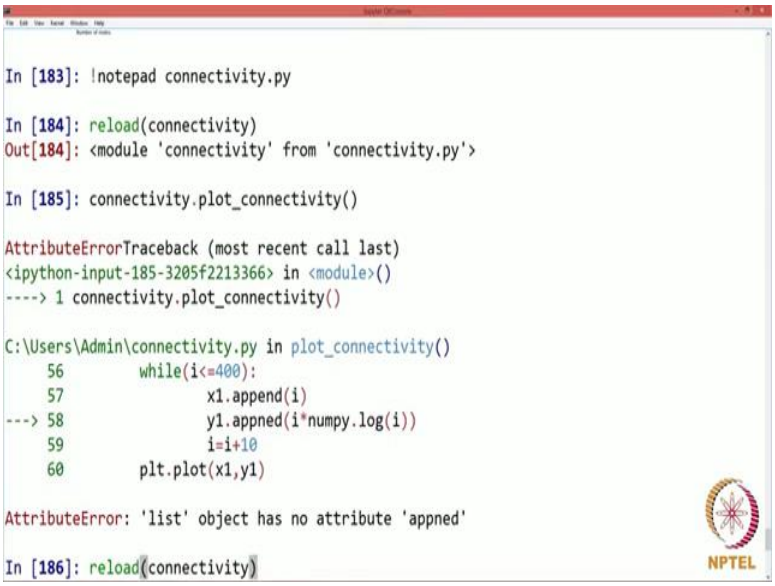
(Refer Slide Time: 25:38)

```
            i=i+10
    plt.xlabel('Number of nodes')
    plt.ylabel('Number of edges required to connect the graph
    plt.title('Emergence of connectivity')
    plt.plot(x,y)

    x1=[]
    y1=[]
    i=10
    while(i<=400):
            x1.append(i)
            y1.appned(i*numpy.log(i))
            i=i+10
    plt.plot(x1,y1)
    plt.show()
```

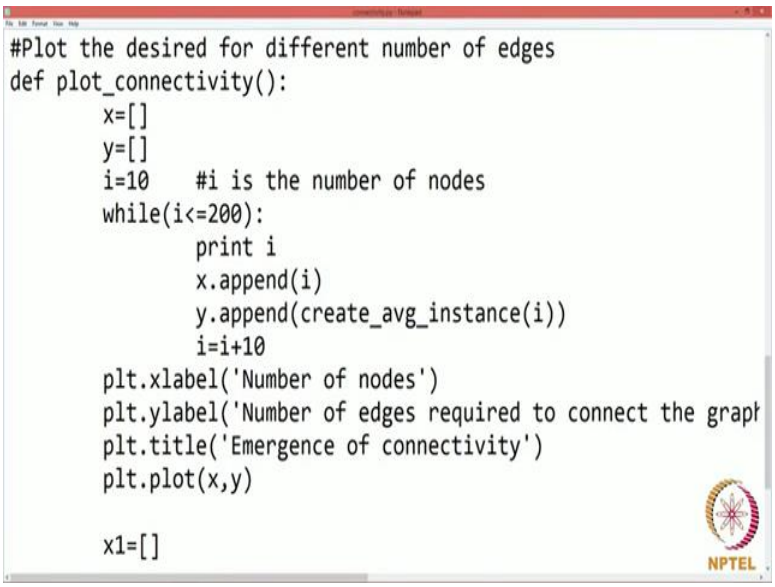
So, I take a new array x1 and a new array y one and then again i set i = 10 which is the number of nodes and while i <= 400 what I do is x1.append(i). y1.append is the curve I want and the curve which I want is i * log(i), sorry, it is x1.append and it is y1.append(i * numpy.log(i)) and then I increment i with 10 and I do plt.plot(x1, y1). So, it is going to show me 2 plots in the same figure 1 is for xy which is the actual data number of nodes and the number of edges required to connect them and second is the x1 y 1 which tells me a number on the x axis and if the number is I, it tells me i log i on the y axis and again reload the file and let us run it.

(Refer Slide Time: 27:08)



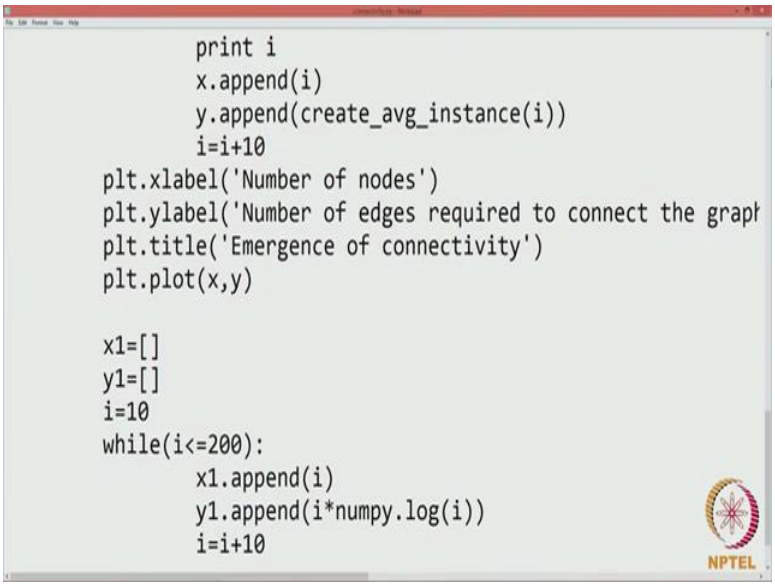
And let us give it some time to execute since it is averaging over hundred instances it will take some decent amount of time the code here ends within a error and next again a spelling mistake let us go back and correct it. So, just correct it here and what we do here is I have added an extra statement here print i which keeps showing us how many iterations has the code complete it.

(Refer Slide Time: 27:34)



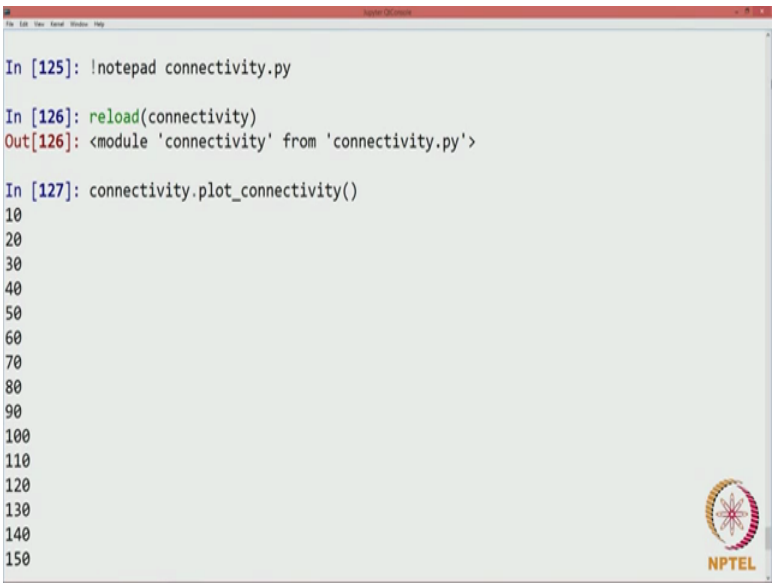
So, for the time being let us make this I 2 hundred since for 400 it takes a lot of time. So, let us see how it works for 200, but if we see a code below.

(Refer Slide Time: 27:59)



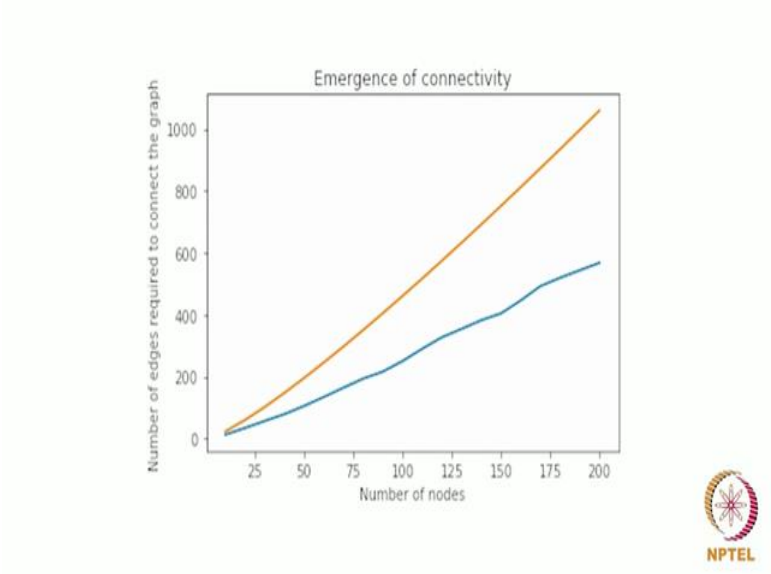
So, for the second plot it is still plotting till the till 400 nodes. So, we also change this and make it here 200 and now we just save this code and let us run it.

(Refer Slide Time: 28:09)



So, let us reload our file we call function connectivity.plot_connectivity. So, here you can see 2 lines in this plot one is the blue line which corresponds to our data where we took different number of nodes and looked at the number of edges required to connect the corresponding network and the red line it corresponds to the n log n plot. So, the lines are not totally overlapping in this case.

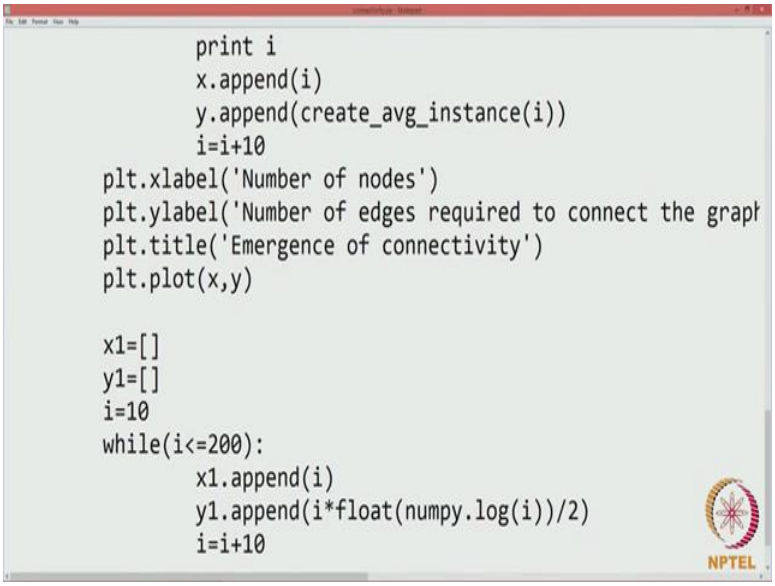
(Refer Slide Time: 28:25)



So, here I would like to remind you that according to the analysis that we have done the number of edges required to connect the graph was not exactly n log n it is of the order of n log n.

So, let us do a small; a little bit of tweak in our code let us go back to a notepad file.

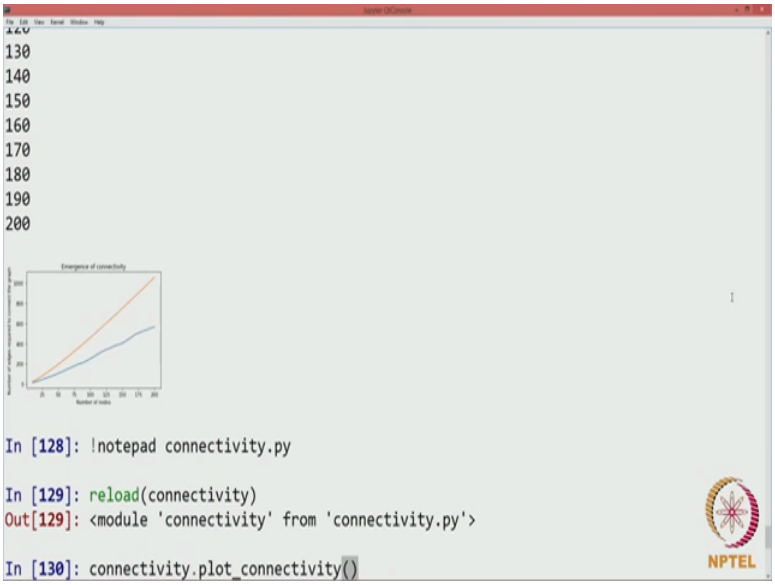
(Refer Slide Time: 29:06)



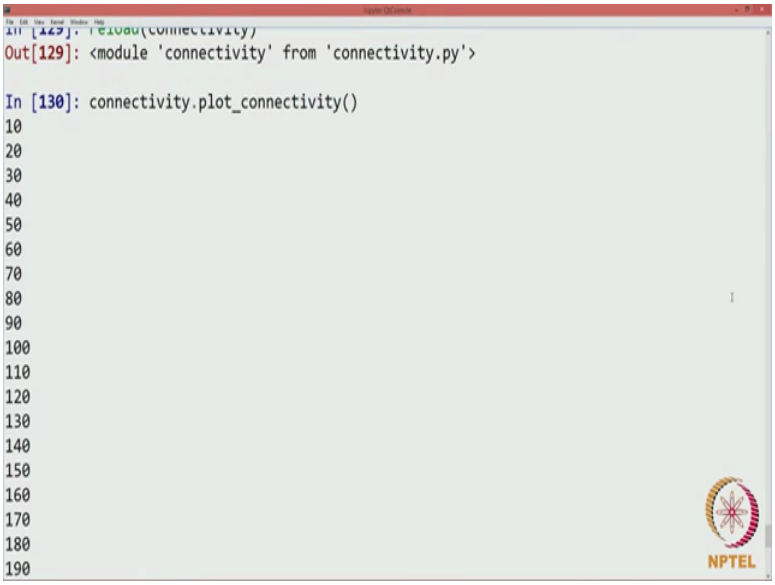
And what I am going to do here is instead of plotting n log n what I plot here is n log n divided by two. So, please note that even if I divide this term n log n by 2 in the final

analysis the number of edges required to connect the network still remains of the order of n log n. So, let us try doing that. So, what I do here is I divide the term log n by 2.

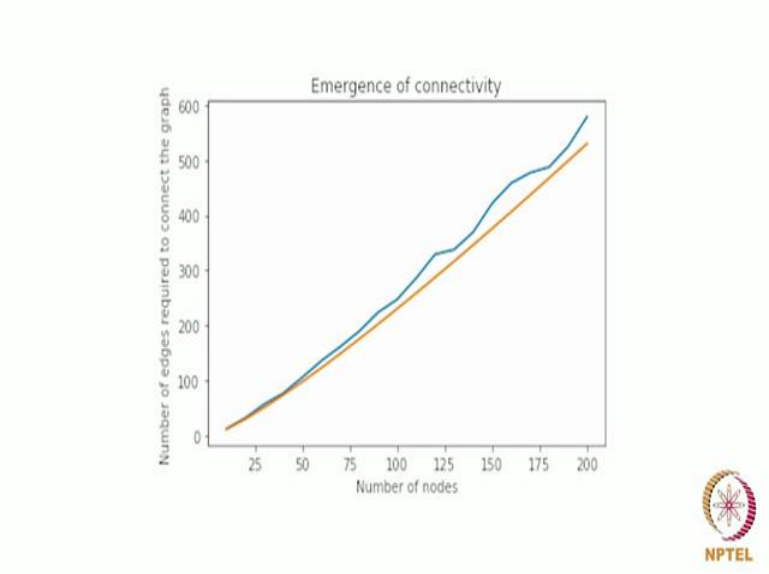
(Refer Slide Time: 29:35)



(Refer Slide Time: 29:38)



(Refer Slide Time: 29:46)



And let us see how our plot changes let us plot it. So, here you can see that both the curves become very close to each other establishing the fact that it is essentially n log n by 2 number of edges almost of the order of n log n which are required to make our graph connecting hence satisfying a previous claim.

Social Networks
Prof. S. R. S. Iyengar
Department of Computer Science
Indian Institute of Technology, Ropar

Lecture – 26
Handling Real-world Network Datasets
Summary to Datasets

This week we saw the side of with introduction to datasets we saw why datasets, the fact that this error has helped us collect a whole lot of datasets and even we have good computational facilities to make sense out of these data sets. We should you using python networkx like APIs, how one can write a piece of code and infer make inferences from these data sets. We also told you a powerful tool like Gephi where you input a graph and it beautifully shows you the different nodes and edges of the graphic, we can visualize it in different ways, it appears very nice.

Gephi is not just used to visualize graphs it is also used to analyze graphs especially for people who are not from programming background. If you were to analyze a given a graph quickly and you want to make sense out of it in let us say no time, python is not your destination it has to be Gephi like tool. But python again gives you the flexibility because to write your own code the way you want a Gephi does not it is sort of standardized. Nevertheless we saw a tool like Gephi which we realized is both powerful and fun to use and then we moved on and saw a piece of puzzle which we have been discussing from the first weeks first lecture which is the emergence of connectedness. We saw how a graph becomes connected when you take a bunch of nodes without any edges and you start putting edges one edge at a time and you will see that there is a movement when it becomes connected, when does it become connected? If you remember it is when you put n log n number of edges.

Then we also saw that it is not just mathematically true that its n log n our program also says it is something to do with a little more than n number of edges for a graph with n vertices to become connected and that talks about the power of what is called synthetic data sets. Synthetic means you synthesize a dataset of your choice for example, you have a huge dataset of 2000 nodes or 2 million nodes let say, but you want to observe what happens on a graph of 100 nodes say you would like to create your own network of 100 nodes. We this the experiment with which we concluded this week was a typical example

of how one can do this. We took, you saw the plot of as you take vertices and you look at the emergence of a connectedness and the plot looks plot has something to tell you right. So, that is a good example of how you can make your own data sets as and when required.

There are many ways to do this we will cover that in the forthcoming chapter. To put the entire chapter in a nutshell we learnt about datasets we saw the emergence of connectedness and we saw the importance of synthetic data sets that us with this week. So, next week onwards we are going to shift our gates completely, the past two weeks is only been playing around with python and understanding some basics of a network x and crunching datasets and things like that. Next week onwards we are going to see some hard-core results in network science. So, all that I told you in the first weeks lectures on do you think you should watch your company, how does Google work, how do you predict who is your next friend so on and so forth all those things are coming next.

So, next week especially we are going to start off with a clip of Harry Potter and we will see what has one to learn from this clip.